

Programming from the Ground Up(조나단 바틀렛)

남궁명수

이 문서는 GNU 자유 문서 라이선스 버전 1.1 또는 자유 소프트웨어 재단이 이후에 발표한 모든 버전의 조건에 따라 복사, 배포 및 수정이 가능하다. 불변 섹션, 앞 표지 텍스트 및 뒷 표지 텍스트는 존재하지 않는다. 또한 저자에 대한 별도의 크레딧 없이도 코드 예제를 어떤 목적이든 자유롭게 사용할 수 있다. 문서에 언급된 모든 상표는 각각의 소유자에게 귀속된다.

이 책은 참고서가 아니라 입문서이다. 따라서 x86 어셈블리 언어를 전문적으로 프로그래밍하는 방법을 배우기에는 적합하지 않다. 학습 과정을 원활하게 하기 위해 일부 세부 사항이 생략되었다. 이 책의 목적은 학생이 어셈블리 언어와 컴퓨터 프로그래밍이 어떻게 작동하는지를 이해하도록 돕는 것이지, 주제에 대한 참고서가 되는 것은 아니다. 특정 프로세서에 대한 참고 정보는 해당 프로세서를 제조하는 회사에 문의하여 얻을 수 있다.

1 장 서론

[1] 프로그래밍에 오신 것을 환영한다.

나는 프로그래밍을 사랑한다. 단순히 작동하는 프로그램을 만드는 것 뿐만 아니라, 그것을 아름답고 스타일있게 만드는 도전을 즐긴다. 프로그래밍은 시와 같다. 프로그램은 컴퓨터뿐만 아니라, 이를 읽고 수정하며 사용하는 사람들에게도 메시지를 전달한다.

프로그램을 통해 우리는 자신의 규칙에 따라 하나의 세계를 만든다. 문제와 해결책에 대한 우리의 관점이 곧 그 세계의 모습이 된다. 뛰어난 프로그래머는 시나 에세이처럼 명확하고 간결한 코드로 세계를 창조한다.

위대한 프로그래머 도널드 커누스(Donald Knuth)는 프로그래밍이란 컴퓨터에게 무엇을 시키는 것이 아니라, 사람이 컴퓨터에게 무엇을 시키는지 설명하는 행위라고 말했다. 즉, 프로그램은 컴퓨터보다도 사람이 읽기 위해 만들어진다.

당신의 프로그램은 당신이 떠난 뒤에도 다른 사람들에 의해 수정되고 확장될 것이다. 따라서 프로그래밍은 컴퓨터와의 소통인 동시에, 미래의 프로그래머와의 소통이다. 프로그래머는 문제 해결자이자 시인이며, 동시에 가르치는 사람이다. 우리의 목표는 문제를 해결하되, 균형과 미감을 갖추고, 그 해결책을 다음 세대에게 전달하는 것이다. 이 책이 컴퓨팅을 흥미롭게 만드는 시와 마법의 일부를 전해 주길 바란다.

많은 프로그래밍 입문서들은 여전히 “컴퓨터는 실제로 어떻게 동작하는가?”라는 질문에 명확히 답하지 못한 채 끝난다. 중요하지만 어려운 주제들을 피하는 경우가 많기 때문이다. 이 책은 그러한 어려운 문제들을 정면으로 다룬다. 그것이 진정한 프로그래밍을 숙련으로 가는 유일한 길이기 때문이다.

이 책의 목표는 독자가 아무런 배경지식이 없는 상태에서 출발해, 프로그래머처럼 사고하고, 작성하고, 학습하는 방법을 이해하도록 돕는 것이다. 모든 것을 알게 되지는 않더라도, 각 요소들이 어떻게 연결되는지에 대한 큰 그림을 얻게 될 것이다. 이 책을 마칠 무렵, 독자는 프로그램이 어떻게 동작하고 서로 상호작용하는지 이해할 수 있어야 한다.

이 책을 통해 독자는 다음과 같은 능력을 기르는 것을 목표로 한다.

- 프로그램이 어떻게 동작하고 서로 상호작용하는지 이해하기
- 다른 사람이 작성한 프로그램을 읽고 그 구조와 동작 방식을 이해하기
- 새로운 프로그래밍 언어를 빠르게 습득하기
- 컴퓨터 과학의 고급 개념을 효율적으로 학습하기

이 책이 컴퓨터 과학의 모든 것을 다루지는 않는다. 컴퓨터 과학은 매우 방대한 분야이며, 특히 이론과 실제 프로그래밍이 결합될 때 그 깊이는 더욱 커진다. 다만 독자가 원하는 방향으로 나아갈 수 있도록 탄탄한 기초를 마련하는 것이 이 책의 목표이다.

특히 프로그래밍, 그 중에서도 어셈블리 언어를 배우는 과정에는 ‘닭과 달걀’과 같은 어려움이 존재한다. 배워야 할 내용이 많고, 각 개념이 서로를 전제로 하기 때문이다. 이 과정에서 자신과 컴퓨터 모두에게 인내심을 가지는 것이 중요하다. 처음에 이해되지 않는 부분이 있다면 다시 읽고, 그래도 어렵다면 일단 넘어갔다가 나중에 다시 돌아오는 것이 좋다. 경험이 쌓일수록 개념은 점차 명확해진다. 낙담하지 말자. 이 여정은 길지만 그만큼 가치가 있다.

각 장의 끝에는 세 단계의 복습 문제가 제공된다.

첫 번째는 개념을 되짚는 문제로, 배운 내용을 이해했는지 확인한다.

두 번째는 응용 문제로, 배운 지식을 활용해 문제를 해결할 수 있는지를 점검한다.

세 번째는 사고를 확장하는 문제로, 시야를 넓히고 깊이 있는 사고를 유도한다.

이 중 일부 질문은 책의 뒷부분에서야 답할 수 있을 수도 있고, 외부 자료 조사가 필요한 경우도 있다. 또한 정답이 하나로 정해지지 않은 질문도 많다. 그러나 이는 중요하지 않다는 뜻이 아니다. 문제를 탐구하고, 답을 찾는 방법을 배우며, 미래를 내다보는 능력은 모두 프로그래머에게 필수적인 자질이다.

[2] 당신의 도구

이 책은 x86 프로세서와 GNU/Linux 운영 체제를 대상으로 한 어셈블리 언어를 다룬다.

모든 예제는 GNU/Linux 표준인 GCC 도구 세트를 사용하여 작성하고 실행된다.

GNU/Linux 나 GCC 도구에 익숙하지 않더라도, 필요한 내용은 이후에 설명될 것이다.

리눅스가 처음이라면 기본적인 사용법을 익히는 것도 도움이 된다. 특히 명령줄에서의 탐색 방법과 pico, emacs, vi 와 같은 텍스트 편집기 중 하나를 사용할 수 있으면 충분하다. 보다 자세한 내용은 *rute* 가이드를 참고할 수 있다.

리눅스를 처음 접하는 사용자라면 지역 GNU/Linux 사용자 그룹에 참여하는 것도 좋은 방법이다. 사용자 그룹의 구성원들은 설치부터 효율적인 방법까지 친절하게 도와주는 경우가 많다. GNU/Linux 사용자 그룹 목록은 linux.org 에서 확인할 수 있다.

이 책의 모든 프로그램은 Red Hat Linux 8.0x 환경에서 테스트되었으며, 다른 x86 기반 GNU/Linux 배포판에서도 정상적으로 동작해야 한다. BSD 와 같은 비-Linux 운영 체제에서는 동작하지 않지만, 여기서 배우는 개념과 기술은 다른 시스템으로도 쉽게 옮겨 적용할 수 있다.

만약 GNU/Linux 머신에 직접 접근할 수 없다면, 리눅스 셸 계정을 제공하는 호스팅 서비스를 이용할 수도 있다. 이 경우 명령줄 기반 환경에서 실습을 진행하게 된다. 저렴한

서비스가 많지만, 반드시 x86 아키텍처에서 동작하는 리눅스 환경을 제공하느라 확인해야 한다. 경우에 따라 지역 GNU/Linux 사용자 그룹의 구성원이 셸 계정을 제공해 줄 수도 있다.

셸 계정을 사용하려면 인터넷 연결과 텔넷 프로그램이 필요하다. Windows 사용자의 경우 기본적으로 텔넷 클라이언트가 포함되어 있으며, 시작 -> 실행 -> telnet 을 입력하면 사용할 수 있다. 다만 Windows 기본 텔넷 클라이언트에는 불편한 점이 있으며, 일반적으로 PuTTY 를 사용하는 것이 더 좋다.

Macintosh 환경에서도 다양한 텔넷 프로그램을 사용할 수 있으며, 그 중 Nifty Telnet 은 추천할만한 선택지이다.

만약 GNU/Linux 환경도 없고 셸 계정 서비스도 구할 수 없다면, knoppix 를 대안으로 사용할 수 있다. knoppix 는 CD 로 부팅되는 GNU/Linux 배포판으로, 하드디스크에 설치할 필요가 없다. 사용을 마친 후 재부팅하고 CD 를 제거하면 기존 운영체제로 그대로 돌아갈 수 있다.

GNU(GNU's Not Unix)

유닉스와 호환되지만 완전히 자유로운 소프트웨어로 만들어진 운영체제(OS)이자 소프트웨어 모음이며, 리처드 스톨먼이 주도한 자유 소프트웨어 운동의 핵심 프로젝트이다.

GCC(GNU Compiler Collection)

GNU 컴파일러 모음을 의미하며, C, C++, Fortran 등 다양한 프로그래밍 언어의 소스 코드를 컴퓨터가 이해할 수 있는 기계어로 번역해주는 강력하고 널리 쓰이는 오픈소스 컴파일러이다.

GNU/Linux

UNIX 를 모델로 설계된 운영체제이다. 여기서 GNU 는 GNU 프로젝트에 비롯된 것으로, 우리가 사용할 대부분의 프로그램과 개발 도구를 포함한다. 특히 이 책에서 사용하는 GCC 도구 세트 역시 GNU 프로젝트의 산물이다. GCC 도구 세트는 다양한 프로그래밍 언어로 프로그램을 작성하고 컴파일하는 데 필요한 핵심 도구들을 모두 포함하고 있으며, 소프트웨어 개발을 위한 표준적인 환경을 제공한다. 이러한 GNU 도구들과 리눅스 커널이 결합되어 GNU/Linux 운영체제가 구성된다.

리눅스(Linux)는 운영 체제 전체를 가리키는 말이 아니라, 그 중에서도 커널(kernel)의 이름이다. 커널은 운영 체제의 핵심으로, 시스템 전반의 동작을 관리하고 조율하는 역할을 한다. 커널은 일종의 울타리이자 관문이다. 관문으로 커널은 프로그램들이 하드웨어에 일관된 방식으로 접근할 수 있도록 중개한다. 커널이 없다면, 프로그램마다 각기 다른 장치 모델을 직접 처리해야 할 것이다. 커널은 이러한 장치별 상호작용을 대신 처리해 주기 때문에, 개발자는 하드웨어의 세부사항을 신경 쓰지 않아도 된다.

또한, 커널은 파일 접근과 프로세스 간의 상호작용도 관리한다. 예를 들어 사용자가 키보드로 입력을 할 때, 그 입력은 바로 편집기도 전달되지 않는다. 먼저 커널이 하드웨어로부터 입력을 받아 처리한다. 키보드는 스캔 코드를 커널에 전달하고, 커널은 이를 문자, 숫자, 기호와 같은 의미 있는 입력으로 변환한다.

그래픽 환경을 사용하는 경우(예: Microsoft Windows 또는 X Windows System), 윈도우 시스템은 커널로부터 키 입력을 받아 현재 화면에서 포커스를 가진 프로그램에 전달한다. 이처럼 커널은 하드웨어와 사용자 프로그램 사이에서 모든 상호작용을 조정하는 중심적인 역할을 한다.

[3] 예제: 컴퓨터가 키보드 신호를 처리하는 과정

- 키보드 -> 커널 -> 윈도우 시스템 -> 응용 프로그램

커널은 키 입력 처리뿐 아니라, 프로그램 간 정보 흐름 전체를 제어한다. 프로그램들이 외부 세계와 데이터를 주고받을 때마다, 그 메시징 과정은 모두 커널을 거친다. 앞선 키보드에서도 커널은 윈도우 시스템이 키 입력을 응용 프로그램에 전달하도록 중개하는 역할을 한다. (여기서 윈도우 시스템은 위에서 설명한, 창을 띄워 사용하는 시각적 환경을 의미한다.)

또한 커널은 보호 장치로서의 역할을 수행한다. 프로그램이 서로의 데이터를 실수로 덮어쓰거나, 권한이 없는 파일이나 장치에 접근하지 못하도록 막는다. 이를 통해 잘못 작성된 프로그램이 다른 실행 중인 프로그램에 끼칠 수 있는 피해를 제한한다.

이 책에서 다루는 환경에서 커널은 리눅스다. 그러나 커널만으로는 아무것도 할 수 없다. 커널만으로는 컴퓨터를 부팅할 수도 없다. 이를 수도관에 비유할 수 있다. 수도관이 없으면 수도꼭지가 작동하지 않지만, 수도꼭지가 없으면 수도관 역시 쓸모가 없다.

사용자 응용 프로그램(주로 GNU 프로젝트 및 기타 자유 소프트웨어)과 리눅스 커널이 함께 결합되어 완전한 운영체제인 GNU/Linux 를 이룬다.

이 책의 대부분은 컴퓨터의 저수준 언어인 어셈블리 언어를 다룬다. 본질적으로 프로그래밍 언어는 다음과 같이 세 가지 유형으로 나눌 수 있다.

기계어(Machine Language)

컴퓨터가 실제로 이해하고 처리하는 언어이다. 컴퓨터가 인식하는 모든 명령은 숫자 또는 숫자의 연속된 형태로 표현된다.

어셈블리 언어(Assembly Language)

어셈블리 언어는 기계어와 동일한 기능을 수행하지만, 숫자로 된 명령어 대신 사람이 기억하기 쉬운 문자 기반의 기호를 사용한다. 또한 프로그래머가 작업하기 쉽도록 몇 가지 편의 기능이 추가되어 있다.

고급 언어(High-Level Language)

고급 언어는 프로그래밍을 보다 쉽고 직관적으로 만들기 위해 존재한다. 어셈블리 언어가 기계의 구조와 동작 방식에 직접 맞추어 작성되는 반면, 고급 언어는 문제와 해결책을 보다 자연스러운 표현으로 기술할 수 있게 해준다. 일반적으로 고급 언어의 하나의 명령은 여러 개의 어셈블리 명령으로 변환된다.

이 책에서는 주로 어셈블리 언어를 다루지만, 고급 언어에 대해서도 일부 살펴볼 것이다. 어셈블리 언어를 배우는 과정은 프로그래밍의 본질과 컴퓨터가 실제로 어떻게 동작하는지에 대한 이해를 한단계 더 깊게 만들어 줄 것이다.

2장 컴퓨터 아키텍처

프로그래밍을 배우기 전에, 컴퓨터가 프로그램을 어떻게 해석하고 실행하는지를 이해하는 것이 중요하다. 전기공학 학위가 필요할 정도의 지식은 아니지만, 몇 가지 기본 개념은 반드시 알고 있어야 한다.

현대의 컴퓨터의 대부분은 폰 노이만(von Neumann) 아키텍처를 기반으로 설계되었다. 이 아키텍처는 컴퓨터를 두 가지 핵심 구성 요소로 나눈다. 하나는 CPU(중앙처리 장치)이고, 다른 하나는 메모리이다. 폰 노이만 아키텍처는 개인용 컴퓨터뿐 아니라 슈퍼 컴퓨터, 메인 프레임, 그리고 휴대폰에 이르기까지 거의 모든 현대 컴퓨터 시스템에서 사용되고 있는 기본 구조이다.

[1] 컴퓨터 메모리의 구조

컴퓨터가 메모리를 어떻게 인식하는지 이해하기 위해, 지역 우체국을 떠올려 보자. 우체국에는 보통 번호가 매겨진 PO 박스가 가든 착 방이 있다. 이 PO 박스들은 모두 같은 크기를 가지며, 일련의 번호로 구분된다. 이러한 구조는 컴퓨터 메모리와 매우 유사하다.

예를 들어, 256MB의 메모리를 가진 컴퓨터는 약 2억 5천 6백만 개의 고정 크기 저장 위치를 가지고 있다고 볼 수 있다. 이는 곧 2억 5천 6백만 개의 PO 박스가 있는 것과 같다. 각 메모리 위치에는 고유한 번호(주소)가 있으며, 모든 위치는 동일한 크기를 가진다.

PO 박스와 컴퓨터 메모리의 중요한 차이점은 저장되는 내용이다. PO 박스에는 다양한 물건을 넣을 수 있지만, 컴퓨터 메모리의 각 저장 위치에는 단 하나의 숫자 값만 저장할 수 있다. 이런 방식의 메모리 구조는 구현이 단순하고 비용이 적게 든다는 장점이 있다. 만약 메모리의 크기가 제각각인 위치들로 구성되거나, 서로 다른 형태의 데이터를 직접 저장할 수 있도록 설계되었다면 구현은 훨씬 복잡하고 비싸졌을 것이다.

컴퓨터 메모리는 다양한 용도로 사용된다. 모든 계산기의 결과는 메모리에 저장되며, 우리가 흔히 말하는 ‘저장된다’는 모든 정보를 결국 메모리에 기록된다. 집에 있는 컴퓨터를 떠올리며, 현재 메모리 안에 어떤 정보들이 담겨 있는지 상상해보면 이해가 더 쉬울 것이다.

예를 들어, 컴퓨터 메모리에는 다음과 같은 정보들이 저장된다.

- 화면에서 커서의 현재 위치
- 각 창의 크기와 위치
- 사용 중인 글꼴에서 각 글자의 모양
- 각 창에 포함된 모든 컨트롤의 배치 정보
- 톨바 아이콘에 사용되는 그래픽 데이터
- 오류 메시지와 각종 대화 상자의 텍스트
- ...

이러한 데이터 뿐만 아니라, 폰 노이만 아키텍처에서는 컴퓨터의 동작을 제어하는 프로그램 자체도 메모리에 저장되어야 한다고 규정한다. 즉, 컴퓨터 메모리에는 단순한 데이터뿐 아니라 실행 중인 프로그램의 코드도 함께 존재한다.

실제로 컴퓨터 내부에서는 프로그램과 그 프로그램이 사용하는 데이터 사이에 본질적인 차이가 없다. 둘 다 동일한 방식으로 메모리에 저장되며, 동일한 방법으로 접근된다. 차이는 오직 컴퓨터가 해당 정보를 어떻게 해석하고 사용하는지에 있을 뿐이다.

[2] CPU(중앙처리장치)

컴퓨터가 단순히 데이터를 저장하는 것만으로는 충분하지 않다. 데이터에 접근하고, 이를 처리하며, 이동시키기 위해서는 CPU가 필요하다. CPU는 컴퓨터의 두뇌로서 실제 연산과 제어를 담당한다.

CPU는 메모리에 저장된 명령어를 하나씩 읽고 실행한다. 이 과정을 가져오기-실행 주기(fetch-execute cycle)라고 한다. 이를 수행하기 위해 CPU는 다음과 같은 주요 구성 요소들을 포함한다.

- 프로그램 카운터(Program Counter)
- 명령어 디코더(Instruction Decoder)
- 데이터 버스(Data Bus)
- 범용 레지스터(General-Purpose Register)
- 산술 및 논리 장치(ALU, Arithmetic and Logic Unit)

프로그램 카운터는 CPU가 다음에 실행할 명령어가 메모리의 어디에 있는지를 알려준다. 앞서 언급했듯이, 프로그램과 데이터는 메모리에 동일한 방식으로 저장되며, 차이는 CPU가 그것을 어떻게 해석하는지에 있다. 프로그램 카운터는 실행될 다음 명령어의 메모리 주소를 저장한다.

CPU는 프로그램 카운터가 가리키는 주소를 확인한 뒤, 해당 위치에 저장된 숫자를 메모리에서 가져온다. 이 숫자는 명령어로 해석된다. 가져온 명령어는 명령어 디코더에 의해 분석된다. 디코더는 해당 명령이 어떤 작업을 수행해야 하는지, 그리고 그 작업에 어떤 메모리 위치나 레지스터가 사용되는지를 결정한다. 일반적으로 하나의 컴퓨터 명령어는 실제 수행할 동작과, 그 동작에 필요한 메모리 위치들의 목록으로 구성된다.

명령어가 해석되면, 컴퓨터는 데이터 버스를 통해 계산에 필요한 메모리 위치의 데이터를 가져온다. 데이터 버스는 CPU와 메모리를 연결하는 통로로, 실제로는 이 둘을 물리적으로 연결하는 전선들의 집합이다. 컴퓨터의 메인보드를 살펴보면 메모리에서 이어지는 전선들이 바로 데이터 버스에 해당한다.

메인 메모리 외에도, 프로세서 내부에는 레지스터(Register)라고 불리는 매우 빠른 특수 메모리 공간이 존재한다. 레지스터는 크게 두 가지로 나뉜다.

- 일반 목적 레지스터

실제 연산이 수행되는 공간으로, 덧셈, 뺄셈, 곱셈, 비교 등 대부분의 연산이 여기에서 이루어진다. 그러나 개수가 매우 제한적이다.

- 특수 목적 레지스터

특정한 기능을 수행하기 위해 사용되는 레지스터로, 프로그램 카운터와 같은 제어 정보를 관리한다.

일반적으로 대부분의 데이터는 주 메모리에 저장되어 있다가, 연산이 필요할 때 레지스터로 옮겨진 후 처리된다. 연산이 끝나면 결과는 다시 메모리에 저장된다. 이후 장에서 이러한 특수 목적 레지스터들에 대해 더 자세히 다룰 것이다.

CPU가 연산이 필요한 모든 데이터를 가져오면, 해당 데이터는 산술 및 논리장치(ALU)로 전달되어 추가 처리가 이루어진다. 이 단계에서 명령어가 실제로 실행된다.

연산이 완료되면, 그 결과는 다시 데이터 버스를 통해 이동하며, 명령어에 지정된 대로 메모리나 레지스터의 적절한 위치에 저장된다. 이로써 하나의 명령어 실행 과정이 마무리된다.

물론 이는 매우 단순화된 설명이다. 실제 현대 프로세서는 훨씬 더 복잡한 구조를 가지고 있다. 기본적인 동작원리는 동일하지만, 성능 향상을 위해 캐시 계층, 슈퍼스칼라 구조, 파이프라인, 분기 예측, 비순차 실행, 마이크로코드 변환, 코프로세서 등 다양한 최적화 기법이 사용된다. 이러한 용어들이 지금은 익숙하지 않더라도 걱정할 필요는 없다. 필요하다면 추후에 하나씩 찾아보며 이해해도 충분하다.

정리하자면,

프로그램의 실행은 프로그램 카운터(Program Counter)에서 시작된다. 프로그램 카운터는 CPU가 다음에 실행할 명령어가 저장된 메모리 주소를 보관하고 있다. CPU는 이 주소를 참조하여 메모리에 접근하고, 데이터 버스(Data Bus)를 통해 해당 위치에 저장된 명령어를 가져온다. (Fetch 단계)

가져온 명령어는 명령어 디코더(Instruction Decoder)로 전달된다. 명령어 디코더는 이 명령어를 분석하여 어떤 연산을 수행해야 하는지, 그리고 그 연산에 어떤 범용 레지스터(General-Purpose Register)나 메모리 위치가 사용되는지를 결정한다. (Decode 단계)

명령어 해석이 끝나면, 필요한 데이터들이 메모리로부터 데이터 버스를 통해 범용 레지스터로 로드된다. 범용 레지스터는 실제 연산이 이루어지기 직전에 데이터를 저장하는 매우 빠른 저장 공간이다.

이후 레지스터에 저장된 데이터는 산술 및 논리 장치(ALU, Arithmetic and Logic Unit)로 전달되어 명령어에서 요구한 산술 연산이나 논리 연산이 수행된다. 이 단계에서 명령어가 실제로 실행된다. (Execute 단계)

연산이 완료되면, 그 결과는 다시 범용 레지스터나 명령어에서 지정한 메모리 위치로 저장되며, 이 과정에서도 데이터 버스가 사용된다. (Store 단계)

마지막으로 프로그램 카운터는 다음 명령어를 가리키도록 갱신되고, CPU는 동일한 과정을 반복하면서 프로그램을 순차적으로 실행한다.

[3] 몇 가지 기본 용어

컴퓨터 메모리는 고정 크기의 저장 위치들이 번호 순서대로 나열된 구조이다. 각 저장 위치에 부여된 주소를 주소(address)라고 한다. 하나의 저장 위치가 담을 수 있는 데이터의 크기를 바이트(byte)라고 하며, x86 프로세서에서 한 바이트(8bit, 2^8)는 0부터 255까지의 값을 저장할 수 있다. 컴퓨터가 0부터 255 사이의 숫자만 저장할 수 있다면, 어떻게 텍스트나 그래픽, 심지어 큰 숫자까지 표현할 수 있는지 의문이 들 수 있다.

먼저, 그래픽 카드와 같은 특수 하드웨어는 각 숫자를 자신만의 방식으로 해석한다. 화면에 문자를 표시할 때 컴퓨터는 ASCII 코드 표를 사용하여 메모리에 저장된 숫자를 문자로 변환해 출력한다. 이 방식에서 각 숫자가 정확히 하나의 문자에 대응된다.

예를 들어, 대문자 A는 숫자 65, 숫자 '1'은 49로 표현된다. 따라서 화면에 "HELLO"를 출력하려면 컴퓨터에 72, 69, 76, 79라는 숫자들의 순서를 전달해야 한다. 마찬가지로, 문자열 "100"을 출력하려면 49, 48, 48이라는 숫자를 사용한다. 숫자를 ASCII 문자로 해석하는 것 외에도, 프로그래머는 숫자에 원하는 의미를 부여할 수 있다. 즉, 숫자는 단순한 값이 아니라 특정 개념이나 대상을 표현하는 수단이 될 수 있다.

예를 들어, 상점을 운영한다고 가정한다. 판매하는 각 상품은 하나의 숫자로 표현될 수 있으며, 이 숫자는 상품이 스캔될 때 화면에 표시될 ASCII 문자 정보와 연결될 수 있다. 또한 가격, 재고 수량과 같은 정보 역시 숫자로 저장되고 관리된다. 이처럼 컴퓨터에서 모든 데이터는 결국 숫자로 표현되며, 그 숫자에 어떤 의미를 부여하느냐는 전적으로 프로그래머의 설계에 달려 있다.

그렇다면 255보다 큰 숫자가 필요할 때는 어떻게 할까? 이 경우 여러 바이트를 조합하여 더 큰 숫자를 표현한다.

- 2 바이트(2^{16})는 0부터 65,535까지의 숫자를 표현할 수 있다.
- 4 바이트(2^{32})는 0부터 4,294,967,295까지의 숫자를 표현할 수 있다.

이처럼 여러 바이트를 결합해 하나의 큰 숫자를 만드는 과정을 직접 구현하려면 다소 복잡하고 수학적 계산이 필요하다. 하지만 다행히도 컴퓨터는 이러한 작업을 자동으로

처리해 준다. 실제로 대부분의 경우 4 바이트 정수가 기본적인 숫자 단위로 사용되며, 우리가 일반적으로 다루는 숫자들도 이러한 방식으로 처리된다.

앞서 살펴보았듯이, 컴퓨터에는 일반 메모리 외에도 레지스터(register)라고 불리는 특수 목적의 저장공간이 있다. 레지스터는 CPU가 계산을 수행할 때 직접 사용하는 매우 빠른 저장 장치이다.

레지스터를 책상 위의 공간에 비유할 수 있다. 서랍이나 파일철에는 많은 정보가 저장되어 있을 수 있지만, 실제로 현재 작업 중인 자료는 책상 위에 올려두고 사용한다. 이와 마찬가지로, 레지스터는 CPU가 현재 처리 중인 숫자와 연산 결과를 잠시 보관하는 역할을 한다. 우리가 사용하는 컴퓨터에서 레지스터는 각각 4 바이트 크기를 가진다. 이러한 일반적인 레지스터의 크기를 컴퓨터의 워드(word) 크기라고 한다.

x86 프로세서는 4 바이트 워드를 사용하며, 이는 아키텍처에서 계산이 4 바이트 단위로 이루어지는 것이 가장 자연스럽다는 의미이다. 4 바이트 워드는 약 40 억개(2^{32})의 서로 다른 값을 표현할 수 있다. 메모리 주소 또한 4 바이트(1 워드) 크기이므로 레지스터에 그대로 저장할 수 있다. 이 때문에 x86 프로세서는 충분한 메모리가 있을 경우 최대 4,294,967,295 바이트(4GB)의 메모리에 접근할 수 있다.

이는 주소 역시 다른 숫자와 동일한 방식으로 저장된다는 것을 의미한다. 실제로 컴퓨터는 어떤 값이 주소인지, 단순한 숫자인지, ASCII 코드인지, 혹은 다른 용도로 사용되는 값인지를 구별하지 않는다. 하나의 숫자는 상황에 따라 서로 다른 의미를 갖는다. 화면에 출력하려 할 때는 ASCII 코드로 해석되고, 특정 메모리 위치를 참조하려 할 때는 주소로 해석된다.

이 개념은 매우 중요하다. 컴퓨터 프로그램은 숫자에 어떤 의미를 부여하는지에 따라 전혀 다른 동작을 하게 되며, 이를 이해하는 것이 컴퓨터가 어떻게 동작하는지를 파악하는 핵심이다. 메모리에 저장된 주소 값은 포인터(pointer)라고도 불린다. 포인터는 일반적인 데이터 값과 달리, 메모리의 다른 위치를 가리키는 역할을 한다.

앞서 설명했듯이, 컴퓨터 명령어 또한 메모리에 저장되며, 저장 방식은 다른 데이터와 완전히 동일하다. 컴퓨터가 어떤 메모리 위치의 값이 명령어인지 구분할 수 있는 유일한 기준은 명령어 포인터(instruction pointer)라는 특수 목적 레지스터가 가리키고 있는지 여부이다.

명령어 포인터가 특정 메모리 단어를 가리키면, 그 값은 명령어로 해석되어 로드된다. 그렇지 않다면, 컴퓨터는 해당 값이 프로그램 코드인지, 데이터인지, 또 다른 용도의 값인지 구별할 수 없다. 이처럼 컴퓨터 내부에서는 프로그램 데이터 사이에 본질적인 차이가 없으며, 오직 사용되는 목적에 따라 의미가 달라진다.

[4] 메모리 해석

컴퓨터는 매우 정확하다. 그리고 컴퓨터가 정확하기 때문에, 프로그래머 역시 정확해야 한다. 컴퓨터는 프로그램이 무엇을 해야 하는지 스스로 알지 못하며, 단지 프로그래머가 지시한 대로만 수행한다.

예를 들어, 숫자를 출력할 때 ASCII 코드 대신 일반 숫자를 보내면, 컴퓨터는 그대로 수행하고 화면에는 의미 없는 글자가 나타난다. 컴퓨터는 그 숫자가 ASCII 에서 무엇을 의미하는지 추측하지 않는다. 또 다른 예로, 명령어 대신 데이터가 저장된 메모리 위치에서 실행을 시작하도록 지시하면, 컴퓨터는 그 내용을 있는 그대로 해석하려 시도할 것이다. 결과가 논리적이지 않을지라도, 컴퓨터는 프로그램이 지정한 정확한 순서대로 명령어를 실행한다. 따라서 프로그래머는 항상 정확한 지시와 순서를 제공해야 하며, 컴퓨터의 정확성과 무심함을 이해하는 것이 중요하다.

핵심은, 컴퓨터는 프로그래머가 지시한 대로 정확히 수행한다는 것이다. 설마 그 지시가 논리적이지 않더라도 마찬가지다. 따라서 프로그래머는 메모리에서 데이터가 어떻게 배열되어 있는지 정확히 이해해야 한다.

기억해야 할 점은, 컴퓨터는 오직 숫자만 저장할 수 있다는 것이다. 문자, 그림, 음악, 웹페이지, 문서 등 모든 데이터는 결국 숫자의 긴 시퀀스에 불과하며, 특정 프로그램이 이 숫자를 해석하여 우리가 이해할 수 있는 형태로 보여준다.

예를 들어, 고객 정보를 메모리에 저장한다고 가정한다. 한 가지 방법은 고객 이름과 주소에 최대 크기를 미리 설정하는 것이다. 각 항목을 50 바이트(50 개의 ASCII 문자)로 할당한다고 하면, 그 다음 고객의 나이와 ID 번호를 위한 숫자 데이터를 이어서 저장할 수 있다. 이 경우 메모리 블록은 다음과 같이 구성된다.

- “고객 이름(50 바이트) - 기록 시작”
- “고객 주소(50 바이트) - 기록시작 + 50바이트”
- “고객 나이(1워드 - 4바이트) - 기록 시작 + 100바이트”
- “고객 ID 번호(1 뒤— - 4 바이트) - 기록시작 + 104 바이트”

이렇게 저장된 메모리 구조를 사용하면, 프로그램은 각 항목의 정확한 위치를 알고 데이터를 읽고 쓸 수 있다.

앞서 예시에서는 고객 이름과 주소를 각각 50 바이트로 제한했지만, 크기를 제한하고 싶지 않은 경우에는 포인터(pointer)를 사용하는 방법이 있다. 즉 고객 이름과 주소 대신 각 항목을 가리키는 포인터를 기록에 저장하는 것이다. 이 경우 메모리 구조는 다음과 같다.

- “고객 이름 포인터(1워드) - 기록 시작”
- “고객 주소 포인터(1워드) - 기록시작 + 4”
- “고객 나이(1워드) - 기록 시작 + 8”
- “고객 ID 번호(1 워드) - 기록시작 + 12”

실제 이름과 주소 데이터는 메모리의 다른 위치에 저장된다. 이렇게 하면 각 데이터 항목의 크기를 미리 제한하지 않아도, 기록 시작 위치만 알면 각 필드가 어디에 있는지 쉽게 찾을 수 있다. 만약 기록 내 필드의 길이가 가변적이라면, 다음 필드가 어디서 시작되는지 알기 어려워진다. 기록의 크기가 달라지므로 다음 기록이 어디서 시작되는지 찾기도 어렵다. 따라서 대부분의 기록은 고정 길이로 관리된다. 가변 길이 데이터는 일반적으로 기록의 나머지 부분과 별도로 저장되며, 기록 내에는 그 위치를 가리키는 포인터만 포함된다.

[5] 데이터 접근 방법

프로세서는 메모리나 레지스터에 저장된 데이터에 접근하기 위해 다양한 주소 지정 모드(Addressing Mode)를 제공한다. 주소 지정 모드는 명령어가 데이터를 어디에서, 어떤 방식으로 가져올 것인지를 정의하며, 프로그램의 효율성과 유연성에 큰 영향을 미친다.

[5-1] 즉시 지정 모드(Immediate Addressing Mode)

즉시 지정 모드는 접근할 데이터가 메모리에 존재하지 않고 명령어 자체에 직접 포함되는 방식이다.

예를 들어, 레지스터를 0으로 초기화할 때, 메모리 어딘가에 저장된 0의 주소를 참조할 필요 없이 명령어에 숫자 0을 바로 포함시킬 수 있다. 이 방식은 구조가 단순하고 실행 속도가 빠르기 때문에, 작은 상수 값을 다룰 때 자주 사용된다.

[5-2] 레지스터 주소 지정 모드(Register Addressing Mode)

레지스터 주소 지정 모드에서 명령어가 메모리 주소가 아닌 접근할 레지스터 자체를 지정한다. 연산이나 데이터 이동이 메모리와의 접근 없이 레지스터 사이에서만 이루어지므로, 가장 빠른 주소 지정 방식 중 하나이다. 단, 레지스터의 개수가 제한적이라는 한계가 있다.

[5-3] 직접 주소 지정 모드(Direct Addressing Mode)

직접 주소 지정 모드에서는 명령어 안에 접근할 메모리 주소가 직접 포함된다.

예를 들어, “메모리 주소 2002에 저장된 데이터를 레지스터로 가져와라”와 같은 형태이다. 이해하기 쉽고 직관적이지만, 명령어가 특정 주소에 종속되기 때문에 코드의 유연성이 떨어진다는 단점이 있다.

[5-4] 인덱스 주소 지정 모드(Index Addressing Mode)

인덱스 주소 지정 모드는 기본 메모리 주소 + 인덱스 레지스터 값을 더해 실제 접근할 주소를 계산하는 방식이다.

예를 들어, 기본 주소가 2002 이고 인덱스 레지스터에 4 가 저장되어 있다면, 실제 접근 주소는 2006 이 된다.

이 방식은 배열처럼 연속된 데이터 집합을 순차적으로 처리할 때 매우 유용하다. 인덱스 레지스터의 값을 증가시키면 다음 요소로 자연스럽게 이동할 수 있기 때문이다.

x86 프로세서에서는 인덱스 값에 배수(scale)를 적용할 수 있어, 바이트 단위뿐 아니라 워드(4 바이트) 단위 접근도 효율적으로 수행할 수 있다.

[5-5] 간접 주소 지정 모드(Indirect Addressing Mode)

간접 주소 지정 모드에서는 명령어가 데이터의 주소를 직접 포함하지 않고, 그 주소를 저장하고 있는 레지스터를 지정한다. 즉, 레지스터의 값은 데이터가 아니라 메모리 주소(포인터)로 해석된다.

예를 들어, 레지스터에 값 4 가 들어 있다면, 직접 주소 지정에서는 값 4 를 데이터로 사용하지만, 간접 주소 지정에서는 메모리 주소 4 에 저장된 값을 사용한다. 이 방식은 포인터를 활용한 데이터 접근을 가능하게 하여, 동적으로 변하는 메모리 위치를 다루는 데 적합하다.

[5-6] 기본 포인터 주소 지정 모드(Base Pointer Addressing Mode)

기본 포인터 주소 지정 모드는 간접 주소 지정 모드에 고정된 오프셋(offset)을 더한 방식이다. 레지스터에는 기준이 되는 주소가 저장되고, 오프셋은 그 기준으로부터 떨어진 위치를 나타낸다.

예를 들어, 레지스터에 1000 이 저장되어 있고 오프셋이 8 이라면, 실제 접근 주소는 1008 이 된다. 이 방식은 구조체, 레코드, 스택 프레임처럼 하나의 기준 주소를 중심으로 여러 필드에 접근할 때 매우 효과적이다.

[5-7] 인덱스 주소 지정과 기본 포인터 주소 지정의 차이

두 방식은 형태가 비슷하지만 목적이 다르다.

기본 포인터 주소 지정

- 기준 주소는 레지스터
- 오프셋은 고정 값
- 구조체나 고정된 데이터 필드 접근에 적합

인덱스 주소 지정

- 기준 주소는 고정
- 오프셋은 레지스터 값으로 변화
- 배열과 같은 반복적 데이터 접근에 적합

문제

[1] 개념 이해하기

- {1} 가져오기-실행 주기를 설명하기
- {2} 레지스터란 무엇인가, 레지스터가 없으면 계산이 어떻게 더 어려워질까?
- {3} 255 보다 큰 숫자는 어떻게 표현할 수 있을까
- {4} 우리가 사용할 기계의 레지스터는 얼마나 큰가
- {5} 컴퓨터는 주어진 바이트 또는 메모리의 바이트 집합을 어떻게 해석하는지 어떻게 알까
- {6} 주소 지정 모드는 무엇이며, 무엇에 사용되는가
- {7} 명령 포인터는 무엇을 하는가

[2] 개념 사용하기

- {1} 직원 기록에 어떤 데이터를 사용할 것인가, 메모리에서 어떻게 배치할 것인가
- {2} 위의 직원 기록의 시작 부분에 대한 포인터가 있고, 그 안의 특정 데이터를 접근하고 싶다면, 어떤 주소 지정 모드를 사용할 것인가?
- {3} 기본 포인터 주소 지정 모드에서, 값이 3122 인 레지스터와 오프셋이 20 이 있다면, 어떤 주소에 접근하려고 하는가
- {4} 인덱스 주소 지정 모드에서 기본 주소가 6512 이고, 인덱스 레지스터가 5 이며, 배율이 4 일 때, 어떤 주소에 접근하려고 하는가
- {5} 인덱스 주소 지정 모드에서 기본 주소가 123472 이고, 인덱스 레지스터가 0 이며, 배율이 4 일 때, 어떤 주소에 접근하려고 하는가
- {6} 인덱스 주소 지정 모드에서 기본 주소가 9123478 이고, 인덱스 레지스터가 20 이며, 배율이 1 일때, 어떤 주소에 접근하려고 하는가

[3] 심화

- {1} 계산이 필요한 최소 주소 지정 모드의 수는 얼마인가
- {2} 필요하지 않은 주소 지정 모드를 포함하는 이유는 무엇인가
- {3} 파이프라이닝(또는 다른 복잡한 요소 중 하나)이 가져오는 가져오기-실행 주기에 미치는 영향을 조사하고 설명하기
- {4} 고정 길이 명령어와 가변 길이 명령어 간의 트레이드 오프를 조사하고 설명하기

3장 첫 번째 어셈블리 프로그램: exit

이 첫 번째 프로그램은 아무런 작업도 수행하지 않고 즉시 종료되는 매우 단순한 프로그램이다. 비록 기능은 없지만, 이 프로그램을 통해 어셈블리 언어의 기본 구조, 리눅스 프로그램의 실행 과정, 그리고 시스템 호출의 개념을 살펴볼 수 있다.

프로그램의 파일의 이름은 반드시 `exit.s` 로 지정해야 하며, 이후 이 파일을 어셈블하고 링크하여 실행 파일을 생성하게 된다. 지금 당장 코드의 모든 내용을 이해하지 못하더라도 걱정할 필요는 없다. 이 단계의 목적은 프로그램을 입력하고 실행하는 전체 흐름을 경험하는 것이다.

[1] 소스 코드, 어셈블링, 링크 과정

우리가 작성한 `exit.s` 파일은 소스 코드(source code)이다. 소스 코드는 사람이 이해하기 쉬운 형태로 작성된 프로그램이며, 컴퓨터가 직접 실행할 수는 없다. 이를 실행 가능한 형태로 만들기 위해서는 어셈블링과 링킹이라는 두 단계를 거쳐야 한다.

[1-1] 어셈블링(Assembling)

어셈블링은 어셈블리 언어로 작성된 소스 코드를 기계어(machine code)로 변환하는 과정이다. 이 과정은 어셈블러(as)가 수행하며, 결과물은 객체 파일(object file)이다.

```
as exit.s -o exit.o
```

이 명령어는 `exit.s` 를 어셈블하여 `exit.o` 라는 객체 파일을 생성한다. 객체 파일은 기계어로 이루어져 있지만, 아직 독립적으로 실행할 수는 없다.

[1-2] 링크(Linking)

링크 과정은 하나 이상의 객체 파일을 결합하고, 운영체제가 프로그램을 올바르게 로드하고 실행할 수 있도록 필요한 정보들을 추가하는 단계이다. 이 과정은 링커(ld)가 수행한다.

```
ld exit.o -o exit
```

이 명령어는 `exit.o` 를 링크하여 최종 실행 파일 `exit` 를 생성한다. 이후 프로그램을 실행하려면 다음과 같이 입력한다.

```
./exit
```

프로그램은 즉시 종료되며, 화면에는 아무런 출력도 나타나지 않는다.

[1-3] 종료 상태 코드 확인

리눅스에서 모든 프로그램은 종료 시 종료 상태 코드(exit status)를 반환한다. 일반적으로 0은 정상 종료, 0이 아닌 값은 오류나 특정 상태를 의미한다.

```
echo $?
```

이 명령어를 입력하면 방금 실행한 프로그램의 종료 상태 코드가 출력된다. 우리의 프로그램은 정상적으로 종료되므로 0이 출력된다.

[2] 어셈블리 언어 프로그램의 개요

[2-1] 주석(Comment)의 의미와 중요성

#으로 시작하는 줄은 주석이다. 주석은 어셈블러에 의해 번역되지 않으며, 실행에도 전혀 영향을 주지 않는다. 그 목적은 오직 사람을 위한 설명이다.

대부분의 프로그램은 시간이 지나면 다른 사람(혹은 미래의 나 자신)이 수정하게 된다. 따라서 주석은 "코드가 왜 존재하는지", "어떻게 동작하는지", 그리고 왜 이상해 보이는 방식을 쓰는지를 설명해야 한다.

특히 반드시 주석에 포함해야 할 내용은 다음 세 가지다.

- 코드의 목적
- 처리 과정의 개요
- 특이한 동작과 그 이유

이상한 코드는 보통 이유가 있다. 그 이유를 적어두지 않으면, 미래의 프로그래머는 코드를 깨뜨려 가며 이유를 추측해야 한다.

[2-2] 어셈블리 지시문과 섹션 구조

.section .data와 .section .text처럼 마침표(.)으로 시작하는 줄은 CPU가 실행하는 기계 명령이 아니라, 어셈블러에게 주는 지시이다. 이를 어셈블리 지시문(pseudo-operation)이라고 한다.

.section .data

데이터 섹션은 프로그램에서 사용하는 데이터를 저장할 공간을 정의한다. 이 예제에서는 실제 데이터가 없지만, 프로그램의 구조를 완성하기 위해 포함되어 있다.

.section .text

텍스트 섹션을 실제로 실행될 명령어들이 위치하는 영역이다. 모든 실행 코드는 이 섹션 안에 작성된다.

[2-3] _start, 심볼(symbol), 레이블(label)

`.global _start` 는 어셈블러에게 `_start` 라는 심볼을 절대 제거하지 말라고 지시한다.
이 심볼은 프로그램의 시작 지점을 의미하며,
링커와 운영체제가 "어디서부터 실행해야 하는지"를 알기 위해 반드시 필요하다.

`_start:`

이 줄은 레이블(label)이다.
레이블은 "이 위치의 주소에 이름을 붙인다"라는 의미다.

중요한 점은

- 레이블은 실제 주소를 직접 쓰지 않게 해준다.
- 코드나 데이터 위치가 바뀌어도 자동으로 주소가 갱신된다.

즉, 레이블과 심볼 덕분에 우리가 숫자 주소 대신 이름으로 프로그래밍할 수 있다.

[2-4] 레지스터와 x86 구조

x86 프로세서는 여러 개의 범용 레지스터(General-Purpose Register)를 가진다.

대표적인 32 비트 범용 레지스터는 다음과 같다.

범용 레지스터

- `%eax`, `%ebx`, `%ecx`, `%edx`
- `%esi`, `%edi`

이외에도 특별한 목적을 가진 레지스터들이 있다.

특수 목적 레지스터

- `%ebp`: 스택 프레임 기준점(베이스 포인터)
- `%esp`: 스택 포인터
- `%eip`: 프로그램 카운터(다음 실행할 명령어 주소)
- `%eflags`: 상태 플래그

[2-5] movl \$1, %eax – 즉시 주소 지정

```
movl $1, %eax
```

이 명령어는 숫자 1 을 `%eax` 레지스터에 복사한다.

여기서 중요한 것은 `$` 기호다.

- `$1` -> 값 1 자체(immediate mode)
- `1` -> 메모리 주소 1 에 있는 값(direct addressing)

우리는 숫자 1 자체가 필요하므로 `$` 를 반드시 붙여야 한다.

[2-6] 시스템 호출과 `%eax`, `%ebx`

리눅스에서 운영체제 기능(종료, 파일 처리 등)은 시스템 호출(system call)을 통해서만 수행된다.

x86 리눅스에서는:

- `%eax`: 시스템 호출 번호
- `%ebx`, `%ecx`, `%edx`: 시스템 호출의 매개변수

```
movl $1, %eax #exit 시스템 호출 번호
```

숫자 1 은 exit 시스템 호출 번호이다.

```
movl $0, %ebx #종료 상태 코드
```

`%ebx` 에는 프로그램 종료 상태 코드가 들어간다. 이 값은 셸에서 `echo $?` 로 확인할 수 있다.

[2-7] int 0x80 – 커널로 제어를 넘기는 순간

```
int $0x80
```

`int` 는 `interrupt`(인터럽트)를 의미한다. 이 명령은 프로그램의 흐름을 멈추고, CPU 제어를 리눅스 커널에게 넘긴다.

즉, “지금 레지스터에 설정한 요청을 운영체제가 처리해 달라”라고 신호를 보내는 것이다. 이 명령이 없으면 레지스터에 값을 넣어도 아무 시스템 호출도 실행되지 않는다.

[2-8] 정리 및 전체 코드

.data 와 .text 섹션을 정의하고 _start 를 프로그램의 엔트리 포인트로 노출한 뒤, EAX(시스템 콜번호)에 1(sys_exit)을 넣고 EBX(첫 번째 인자)에 인자(0)을 넣고, int 0x80 을 통해 커널 모드로 진입하여 커널에게 프로세스 종료를 요청한다.

32 비트 x86

```
.section .data
.section .text
.global _start

_start:
    movl $1, %eax    #어떤 시스템 콜을 쓸지 지정
    movl $0, %ebx    #그 시스템 콜의 첫 번째 인자
    int $0x80        #커널에 요청
```

64 비트 x86-64

```
.section .data
.section .text
.global _start:

_start:
    mov $60, %rax
    xor %rdi, %rdi
    syscall
```

[3] 프로그램 계획하기

다음 프로그램에서는 숫자 목록에서 최대값을 찾는 작업을 수행하려고 한다. 컴퓨터는 매우 세부적인 지시를 필요로 하기 때문에, 프로그램을 작성하기 전에 여러 가지 사항을 미리 계획해야 한다. 이러한 세부 사항에는 다음과 같은 것들이 포함된다.

- 원래 숫자 목록은 어디에 저장될 것인가?
- 최대 숫자를 찾기 위해 어떤 절차를 따라야 하는가?
- 그 절차를 수행하기 위해 얼마나 많은 저장 공간이 필요한가?
- 모든 저장 공간을 레지스터에 둘 수 있는가, 아니면 일부 메모리도 사용해야 하는가?

최대 숫자를 찾는 것과 같은 간단한 작업은 많은 계획이 필요하지 않을 것처럼 보일 수 있다. 실제로 사람에게 숫자 목록에서 가장 큰 값을 찾으라고 하면, 큰 어려움 없이 수행할 수 있다. 그러나 우리의 마음은 복잡한 작업을 자동으로 구성하고 처리하는 데 익숙한 반면, 컴퓨터는 이러한 과정을 명확한 지시를 통해서만 수행할 수 있다.

또한 우리는 보통 많은 정보를 동시에 머릿속에 쉽게 유지할 수 있다. 예를 들어, 숫자 목록을 훑으면서 최대값을 찾을 때, 우리는 지금까지 본 숫자 중 가장 큰 값과 현재 목록에서의 위치를 동시에 기억한다. 인간의 두뇌는 이러한 작업을 자동으로 처리하지만, 컴퓨터는 현재 위치와 현재까지의 최대값을 저장하기 위해 이를 명시적으로 설정해야 한다.

이 외에도 언제 작업을 멈춰야 하는지와 같은 문제도 존재한다. 사람이 종이에 적힌 숫자를 읽을 때는 숫자가 끝나면 자연스럽게 멈출 수 있다. 그러나 컴퓨터는 단순한 숫자만을 처리하므로, 별도의 정보가 없으면 마지막 숫자에 도달했는지를 스스로 알 수 없다.

컴퓨터에서는 모든 단계를 명확하게 계획해야 한다. 그러므로 이제 실제로 계획을 세워본다. 먼저, 숫자 목록이 시작되는 주소를 `data_items` 라고 이름 짓겠다. 목록의 마지막 숫자는 0 으로 설정하여, 프로그램이 어디에서 멈춰야 하는지를 알 수 있도록 한다. 또한 다음과 같은 값들을 저장할 공간이 필요하다.

- 목록에서 현재 위치
- 현재 검사 중인 목록 요소
- 지금까지 발견한 최대값

각 값은 레지스터에 다음과 같이 할당한다.

- `%edi`: 목록에서의 현재 위치
- `%ebx`: 지금까지 발견한 최대값
- `%eax`: 현재 검사 중인 목록 요소

프로그램을 시작하면서 먼저 목록의 첫 번째 항목을 살펴본다. 아직 다른 항목을 보지 않았기 때문에, 이 값은 자동으로 현재까지의 최대값이 된다. 또한 현재 위치를 0(첫 번째 요소)으로 설정한다.

그 이후에는 다음 절차를 반복한다.

1. 현재 목록 요소(%eax)가 0 인지 확인하기(종료 요소인지 검사)
2. 값이 0 이면 프로그램을 종료한다.
3. 현재 위치(%edi)를 증가시킨다.
4. 목록에서 다음 값을 현재 값 레지스터(%eax)에 로드한다.
→ 어떤 주소 지정 모드를 사용할 수 있을까, 그이유
5. 현재 값(%eax)을 현재 최고 값(%ebx)과 비교한다.
6. 현재 값이 최고 값보다 크다면, 최고 값을 현재 값으로 갱신한다.
7. 다시 반복한다.

이것이 전체 절차이다. 이 과정에서 "if"라는 개념이 여러 번 등장한다. 이는 결정을 내려야 하는 지점을 의미한다. 컴퓨터는 항상 동일한 명령어 순서를 실행하지 않는다. 조건이 참인지 거짓인지에 따라 서로 다른 명령어 경로를 따르게 된다.

예를 들어, 어떤 반복에서는 현재 값이 최고 값보다 크지 않을 수도 있다. 이 경우 6 단계를 건너뛰게 된다. 프로그램이 복잡해 질수록 이러한 "건너뛰기"는 더욱 빈번해진다.

이러한 "if" 동작은 흐름 제어 명령어라고 불리는 명령어들로 구현된다. 흐름 제어 명령어는 컴퓨터에게 어떤 단계를 실행하고, 어떤 경로를 선택해야 하는지를 알려준다. 이전 프로그램에서는 흐름제어 명령어가 없었다. 선택할 수 있는 경로가 하나 뿐이었다.(바로 종료)

그러나 이번 프로그램은 데이터에 의해 동작이 결정되므로, 훨씬 더 동적이다. 입력되는 데이터에 따라 서로 다른 명령어 경로를 따르게 된다.

이 프로그램에서는 흐름 제어를 위해 두 가지 종류의 점프 명령어를 사용한다.

- 조건부 점프: 이전 비교 또는 계산 결과에 따라 분기
- 무조건 점프: 조건과 관계없이 항상 지정된 위치로 이동

무조건 점프는 처음에는 쓸모없어 보일 수 있다. 하지만 모든 명령어가 일렬로 나열되어 실행되기 때문에, 여러 경로가 다시 하나의 주 경로로 합쳐질 때 반드시 필요하다.

또한, 흐름 제어의 중요한 용도 중 하나는 루프(loop)를 구현하는 것이다. 루프는 프로그램 코드의 일부를 반복 실행하는 구조이다.

이 예제에서 프로그램의 첫 부분(현재 위치를 0으로 설정하고, 첫 값을 최대값으로 설정하는 부분)은 한 번만 실행되므로 루프가 아니다. 그러나 그 이후의 부분은 목록의 모든 숫자에 대해 반복적으로 실행된다. 마지막 요소(0)에 도달했을 때만 반복이 종료된다. 이러한 반복 구조를 루프라고 부른다.

루프는 일반적으로 루프의 끝에서 시작 지점으로 무조건 점프함으로써 구현된다. 단, 반드시 어딘가에 루프를 종료할 조건부 점프를 포함해야 한다. 그렇지 않으면 루프는 영원히 반복된다. 이를 무한 루프라고 한다. 만약 우리가 위의 1 단계, 2 단계 또는 3 단계를 실수로 생략한다면, 프로그램은 절대 끝나지 않을 것이다.

다음 섹션에서는 지금까지 계획한 이 프로그램을 실제로 구현해 볼 것이다. 프로그램 계획은 처음에는 복잡하게 느껴질 수 있다. 실제로 프로그래밍을 처음 배울 때는, 우리가 자연스럽게 수행하는 사고 과정을 컴퓨터가 이해할 수 있는 명확한 절차로 변환하는 것이 쉽지 않다. 우리는 문제를 해결할 때 머릿속에서 사용하는 여러 가지 “임시 저장소”를 종종 인식하지 못한다. 하지만 프로그램을 계속 읽고 작성하다 보면, 이러한 사고 방식은 점점 자연스러워질 것이다.

[3-1] 최대값 찾기

```
# 목적: 이 프로그램은 데이터 항목 집합에서 최대값을 찾는다
#
# 레지스터 사용:
# %edi - 검사 중인 데이터 항목의 인덱스
# %ebx - 지금까지 발견한 최대값
# %eax - 현재 데이터 항목
#
# 메모리:
# data_items - 숫자 목록, 0 으로 종료됨

.section .data
data_items:
    .long 3,67,34,222,45,75,54,34,44,33,22,11,66,0

.section .text
.global _start

_start:
    movl $0, %edi          #인덱스를 0 으로 초기화
    movl data_items(,%edi,4), %eax    #첫 번째 값 로드
    movl %eax, %ebx        #첫 값은 현재 최대값

start_loop:
    cmpl $0, %eax          #현재 값이 0 인가?
    je loop_exit           #0 이면 종료

    incl %edi              #인덱스 증가
    movl data_items(,%edi,4), %eax    #다음 값 로드

    cmpl %ebx, %eax        #현재값 vs 최대값
    jle start_loop         #작거나 같으면 다시 루프

    movl %eax, %ebx        #더 크면 최대값 갱신
    jmp start_loop         #루프 계속

loop_exit:
    movl $1, %eax          #exit 시스템 호출 번호
    int $0x80              #커널 호출
```


다음 명령어를 통해 빌드와 실행을 수행한다.

```
as maximum.s -o maximum.o
ld maximum.o -o maximum
./maximum
echo $?
```

[3-2] data_items 레이블의 의미

```
data items:
    .long 3,67,34,222,45,75,54,34,44,33,22,11,66,0
```

레이블이란

- data_items 는 그 다음에 오는 메모리 위치의 주소를 가리키는 이름
- 즉, data_items = 숫자 3 이 저장된 메모리의 시작 주소

왜 레이블을 쓰는가

- 실제 메모리 주소는 어셈블 시점에 정해짐
- 사람이 주소 숫자를 직접 알 필요 없이 이름으로 접근 가능
- 예: movl data_items, %eax

어셈블러가 자동으로 다음처럼 바꾼다.

```
movl [숫자 3 이 있는 실제 주소], %eax
```

결과적으로 값 3 이 %eax 에 로드된다.

[3-3] .long 지시어와 데이터 저장방식

.long 지시어는 어셈블러에게 4 바이트(32 비트) 크기의 메모리를 예약하고 그 안에 정수를 저장하도록 지시한다. x86 32 비트 환경에서 레지스터의 크기 역시 4 바이트이므로 .long 은 레지스터와의 데이터 이동 및 연산에 적합하다. 프로그램에서는 여러 개의 .long 값이 연속적으로 배치되며, 이들은 하나의 정수 배열처럼 메모리에 저장된다. 총 14 개의 정수가 저장되며, 이는 56 바이트의 메모리를 차지한다.

[3-4] 배열의 끝을 0 으로 표시하는 이유

컴퓨터는 데이터의 개수를 스스로 알 수 없기 때문에, 배열의 끝을 판단할 수 있는 기준이 필요하다. 이 프로그램에서는 숫자 목록의 마지막에 0 을 추가하여 배열의 종료를 표시한다. 프로그램은 각 숫자를 하나씩 검사하다가 값이 0 인 항목을 만나면 더 이상 처리하지 않고 종료한다. 만약 이러한 종료 표시가 없다면, 프로그램은 배열의 범위를 넘어 다른 메모리 영역까지 계속 접근하게 되어 잘못된 결과나 오류를 발생시킬 수 있다.

[3-5] 데이터 지시어의 종류

어셈블리 언어에서는 다양한 크기의 데이터를 저장하기 위해 여러 지시어를 제공한다.

`.byte` 는 1 바이트 크기의 데이터를 저장하며 0 부터 255 까지의 값을 표현할 수 있다. `.int` 는 2 바이트 크기의 데이터를 저장하며 최대 65535 까지의 값을 표현할 수 있다. `.long` 은 4 바이트 크기의 데이터를 저장하며 32 비트 정수를 표현하는데 사용된다.

또한 `.ascii` 지시어는 문자열을 메모리에 저장하며, 각 문자는 1 바이트로 저장되고 문자열의 끝은 `\0` 으로 표시된다.

[3-6] `.globl` 지시어와 심볼의 범위

`_start` 레이블은 프로그램의 시작 위치를 나타내며, 리눅스 커널이 프로그램을 실행할 때 반드시 참조해야 하는 심볼이므로 `.globl` 로 선언되어야 한다. 반면 `data_items` 는 같은 소스 파일 내부에서만 사용되는 데이터 레이블이므로 전역 심볼로 선언할 필요가 없다.

물론 `.globl data_items` 를 선언해도 오류는 발생하지 않지만, 외부에서 참조되지 않기 때문에 필수는 아니다.

[3-7] 프로그램 수정 시 주의사항

소스 코드에서 데이터 값이나 명령어를 변경한 경우, 변경 사항이 실행 파일에 반영되도록 반드시 다시 어셈블하고 링크해야 한다. 또한 리눅스에서 프로그램의 종료 상태 코드는 0 부터 255 까지만 유효하므로, 최대값이 255 를 초과할 경우 `echo $?` 명령으로 확인한 결과 의도와 다르게 나타날 수 있다. 이러한 제한 사항을 고려하여 데이터 값을 설정해야 한다.

[3-8] 변수의 개념과 프로그램에서의 역할

이제 코드를 살펴본다. 주석을 보면 프로그램에서 사용할 계획인 몇 가지 변수가 미리 표시되어 있음을 알 수 있다. 이전 섹션에서 이러한 개념들에 대해 이미 이야기했지만, 당시에는 각각에 이름을 부여하지 않았다. 이 프로그램에는 총 세 가지 주요 변수가 사용된다. 첫째는 지금까지 발견된 최대 숫자를 저장하기 위한 변수이고, 둘째는 현재 검사 중인 목록의 위치를 나타내는 변수로 흔히 인덱스라고 부른다. 셋째는 현재 검사 중인 숫자 자체를 저장하기 위한 변수이다.

[3-9] 레지스터를 이용한 변수 저장

이 경우에는 필요한 변수의 수가 많지 않기 때문에, 모든 변수를 레지스터에 저장할 수 있다. 그러나 더 큰 프로그램에서는 모든 변수를 레지스터에 담을 수 없기 때문에, 일부는 메모리에 저장해 두었다가 필요할 때 레지스터로 옮겨 사용해야 한다. 이에 대해서는 이후에 다시 논의할 것이다. 많은 사람들이 프로그래밍을 처음 시작할 때 필요한 변수의 수를 과소평가하는 경향이 있다. 이는 사람들이 문제를 해결하는 과정의 모든 세부 단계를 명확히 의식하지 않는 데 익숙하기 때문이다. 그 결과, 첫 번째 프로그램에서는 필요한 변수를 빠뜨리는 경우가 자주 발생한다.

[3-10] 인덱스의 의미와 데이터 접근 방식

이 프로그램에서는 `%ebx` 레지스터를 현재까지 발견한 가장 큰 데이터 항목을 저장하는 용도로 사용한다. 또한 `%edi` 레지스터는 현재 보고 있는 데이터 항목의 인덱스를 저장하는데 사용된다. 여기서 인덱스의 개념을 살펴볼 필요가 있다. 우리가 `data_items`에서 읽을 때는 첫 번째 항목(데이터 항목 번호 0)부터 시작하여, 두 번째 항목(데이터 항목 번호 1), 세 번째 항목(데이터 항목 번호 2) 순서로 접근하게 된다. 이때 각 데이터 항목의 번호가 바로 `data_items` 배열의 인덱스이다.

[3-11] 인덱스 초기화

프로그램에서 컴퓨터에 주는 첫 번째 명령어는 다음과 같다.

```
movl $0, %edi
```

우리는 `%edi`를 인덱스로 사용하여 데이터 목록의 첫 번째 항목부터 검사하고자 하므로, `%edi`에 0을 로드한다. 이렇게 함으로써 현재 위치를 데이터 목록의 시작 지점으로 설정한다. 그 다음에 등장하는 명령어는 다소 복잡해 보일 수 있지만, 이 프로그램의 핵심 동작과 직접적으로 관련된 매우 중요한 명령어이다.

[3-12] 인덱스 주소 지정 모드의 이해

다음 명령어를 이해하기 위해서는 몇 가지 중요한 사항을 염두에 두어야 한다.

```
movl data_item(%edi,4), %eax
```

먼저 data_items 는 숫자 목록이 시작되는 메모리 주소를 나타낸다. 또한 이 목록의 각 숫자는 .long 지시어로 선언되었기 때문에 각각 4 바이트의 저장 공간을 차지한다. 이 시점에서 %edi 레지스터에는 0 이 저장되어 있으며, 이는 현재 첫 번째 데이터 항목을 가리키고 있음을 의미한다.

이 명령어가 수행하는 동작을 풀어 설명하면, "data_items 가 가리키는 시작 주소에서 출발하여, 현재 인덱스 값(%edi)만큼 이동한 위치의 데이터를 가져오되, 각 항목이 4 바이트 크기임을 고려하여 주소를 계산한 후, 그 값을 %eax 레지스터에 저장한다"라고 할 수 있다. 즉, 데이터 배열의 현재 인덱스에 해당하는 값을 읽어 오는 역할을 한다.

[3-13] 인덱스 주소 지정 모드의 일반적인 형태

어셈블리 언어에서 이러한 방식의 메모리 접근을 인덱스 주소 지정 모드(indexing addressing mode)라고 한다. 이 주소 지정 방식의 일반적인 형태는 다음과 같다.

```
movl BEGINNINGADDRESS(%INDEXREGISTER,WORDSIZE)
```

이 프로그램에서는 data_items 가 시작 주소(BEGINNINGADDRESS)에 해당하고, %edi 가 인덱스 레지스터(INDEXREGISTER)이며, 4 는 각 데이터 항목의 크기(WORDSIZE)를 의미한다. 이 주소 지정 모드는 배열과 같은 연속된 데이터 구조를 처리할 때 매우 유용하며, 이에 대해서는 '주소 지정 모드' 섹션에서 더 자세히 다룬다.

[3-14] 인덱스 값에 따른 데이터 접근 예시

data_items 에 저장된 숫자 목록을 살펴보면, %edi 가 0 일 때는 첫 번째 숫자인 3 이 %eax 로 로드된다. %edi 가 1 로 설정되면 두 번째 숫자인 67 이 %eax 에 들어가며, %edi 가 2 가 되면 세 번째 숫자인 34 가 로드된다. 이와 같은 방식으로 인덱스 값이 증가함에 따라 목록의 다음 숫자가 차례대로 읽혀진다.

만약 여기서 저장 위치의 크기를 4 가 아닌 다른 값으로 지정한다면, 올바른 주소 계산이 이루어지지 않아 전혀 의도하지 않은 값을 읽게 된다. 이 때문에 데이터의 크기와 주소 계산 방식은 반드시 일치해야 한다. 이러한 작성 방식은 처음에는 다소 어색하게 느껴질 수 있지만, 각 구성 요소가 어떤 역할을 하는지 이해하면 비교적 명확하게 받아들일 수 있다. 더 자세한 내용은 '주소 지정 모드' 섹션을 참고하면 된다.

[3-14] 최초 최대값 설정

다음 명령어는 현재 값 레지스터에 저장된 값을 최대값 레지스터로 복사한다.

```
movl %eax, %ebx
```

앞선 단계에서는 우리는 데이터 목록의 첫 번째 항목을 %eax 에 로드하였다. 이 값은 아직 다른 항목과 비교되지 않았지만, 현재까지 확인한 값 중 유일한 값이므로 자동으로 가장 큰 값이 된다. 따라서 이 값을 %ebx 에 저장하여, 이후 비교 과정에서 기준이 되는 최대값으로 사용한다.

여기서 주의할 점은 movl 명령어가 이름과 달리 실제로는 값을 이동시키는 것이 아니라 복사한다는 점이다. 이 명령이 실행 된 후에도 %eax 와 %ebx 모두 동일한 시작값을 유지한다.

[3-15] 루프의 시작과 역할

이제 프로그램은 루프 구조로 진입한다. 루프란 프로그램의 특정 코드 영역이 한 번 이상 반복 실행될 수 있도록 구성된 구조를 의미한다. 이 프로그램에서는 루프의 시작 위치를 start_loop 라는 기호로 표시하였다.

루프를 사용하는 이유는 처리해야 할 데이터 항목의 개수를 미리 알 수 없기 때문이다. 데이터 항목의 수가 얼마이든지 간에, 수행해야 할 절차는 항상 동일하다. 우리는 목록의 길이에 따라 프로그램을 다시 작성하고 싶지 않으며, 각 항목에 대해 개별적인 비교 코드를 작성하는 것도 바람직하지 않다. 대신, data_items 에 포함된 모든 요소에 대해 반복적으로 실행되는 하나의 코드 블록을 사용한다.

[3-16] 루프에서 수행되는 작업 개요

이전 섹션에서 계획한 것처럼, 이 루프는 다음과 같은 작업을 반복적으로 수행한다.

- 현재 검사 중인 값이 0 인지 확인한다. 만약 0 이라면, 이는 데이터 목록의 끝에 도달했음을 의미한다.
- 목록에서 다음 값을 로드한다.
- 새로 로드한 값이 현재까지 저장된 최대값보다 큰지 비교한다.
- 만약 더 크다면, 최대값을 저장하고 있는 위치에 이 값을 새로 저장한다.

이러한 과정을 통해 프로그램을 데이터 목록의 끝에 도달할 때까지 모든 항목을 검사하고, 최종적으로 가장 큰 값을 찾아낸다.

start_loop:	
cmpl \$0, %eax	#현재 값이 0 인가?
je loop_exit	#0 이면 종료
incl %edi	#인덱스 증가
movl data_items(,%edi,4), %eax	#다음 값 로드
cmpl %ebx, %eax	#현재값 vs 최대값
jle start_loop	#작거나 같으면 다시 루프
movl %eax, %ebx	#더 크면 최대값 갱신

[3-17] 종료 조건 검사와 비교 명령어

다음 명령어는 현재 검사 중인 값이 데이터 목록의 끝을 나타내는지 확인하기 위해 사용된다.

cmpl \$0, %eax
je end_loop

cmpl 명령어는 두 값을 비교하는 명령어이다. 여기서는 즉시 값 0 과 %eax 레지스터에 저장된 값을 비교한다. 이 명령어는 비교 결과를 직접 레지스터에 저장하지 않고, %eflags 레지스터에 저장한다. %eflags 레지스터는 상태 레지스터라고도 불리며, 비교 결과나 연산 결과에 대한 여러 상태 정보를 담고 있다. 이 레지스터는 이후의 흐름 제어 명령어에서 중요한 역할을 하며, 자세한 내용은 나중에 다시 논의된다.

[3-18] 조건부 점프와 상태 레지스터의 활용

비교 결과는 %eflags 레지스터에 저장되며, 다음에 실행되는 조건부 점프 명령어는 이 값을 참조하여 실행 여부를 결정한다. je 명령어는 "jump if equal"의 약자로, 바로 앞에서 수행한 비교 결과가 같을 경우 지정된 위치로 점프하라는 의미이다. 따라서 %eax 의 값이 0 일 경우, 프로그램은 end_loop 레이블로 이동하여 루프를 종료한다.

만약 마지막으로 로드된 값이 0 이 아니라면, 조건이 만족하지 않으므로 점프는 발생하지 않고 프로그램은 다음 명령어를 계속 실행한다.

[3-19] 주요 조건부 점프 명령어

어셈블리 언어에는 다양한 조건부 점프 명령어가 존재하며, 이들은 모두 상태 레지스터에 저장된 비교 결과를 기반으로 동작한다. 자주 사용되는 명령어는 다음과 같다.

- je: 두 값이 같으면 점프
- jg: 두 번째 값이 첫 번째 값보다 크면 점프
- jge: 두 번째 값이 첫 번째 값보다 크거나 같으면 점프=
- jl: 두 번째 값이 첫 번째 값보다 작으면 점프
- jle: 두 번째 값이 첫 번째 값보다 작거나 같으면 점프
- jmp: 조건과 관계없이 무조건 점프

이 중 jmp는 비교 명령어가 필요 없는 무조건 점프 명령어이다. 전체 조건부 점프 명령어 목록은 부록 B에 정리되어 있다.

[3-20] 루프 종료 조건의 의미

이 프로그램에서는 %eax가 0일 경우 데이터 목록의 끝에 도달했음을 의미한다. 따라서 해당 조건이 만족되면 더 이상 비교할 데이터가 없으므로 루프를 종료하고 end_loop로 이동한다. 반대로 값이 0이 아니라면, 아직 처리해야 할 데이터가 남아 있으므로 다음 명령어로 진행하여 루프를 계속 수행한다.

[3-21] 다음 데이터 항목으로 이동

```
incl %edi  
movl data_items(,%edi,4), %eax
```

이전 논의에서 살펴본 것처럼 %edi 레지스터는 data_items에 저장된 숫자 목록의 인덱스를 나타낸다. incl 명령어는 %edi의 값을 1만큼 증가시키며, 이를 통해 다음 데이터 항목으로 이동한다. 그 다음 movl 명령어는 증가된 인덱스를 사용하여 목록에 다음 값을 읽어 %eax에 저장한다. 결과적으로 %eax에는 이제 비교 대상이 될 다음 숫자가 들어가게 된다.

[3-22] 현재 값과 최대값 비교

```
cmpl %ebx, %eax  
jle start_loop
```

이 명령어들은 현재 검사 중인 값과 지금까지 발견된 최대값을 비교한다. %eax에는 현재 값이, %ebx에는 지금까지의 최대값이 저장되어 있다. cmpl 명령어는 이 두 값을 비교하며, 그 결과는 상태 레지스터에 저장된다.

jle 명령어는 “jump if less or equal”의 약자로, 현재 값이 최대값보다 작거나 같을 경우 start_loop로 점프한다. 이는 현재 값이 최대값을 갱신할 필요가 없음을 의미하며, 프로그램은 다음 데이터 항목을 처리하기 위해 루프의 처음으로 되돌아간다.

[3-23] 최대값 갱신

```
movl %eax, %ebx  
jmp start_loop
```

만약 현재 값이 지금까지의 최대값보다 크다면, 조건부 점프가 실행되지 않고 이 코드가 수행된다. movl 명령어를 통해 현재 값을 %ebx 에 저장하여 최대값을 갱신한다. 이후 jmp 명령어를 사용해 조건 없이 루프의 시작으로 되돌아가 다음 항목을 계속 처리한다.

[3-24] 루프 종료와 프로그램 종료 처리

루프는 데이터 항목 중 값이 0 인 요소를 만날 때까지 계속 실행된다. 값이 0 일 경우, 이전에 설명한 종료 조건에 검사에 의해 loop_exit 로 점프하게 된다. 이 부분에서는 Linux 커널을 호출하여 프로그램을 종료한다.

프로그램을 종료하기 위해서는 시스템 호출을 사용해야 하며, 종료 시스템 호출 번호는 %eax 에 저장된다. 종료 호출의 번호는 1 이며, 종료 상태 코드는 %ebx 에 저장된다. 이 프로그램에서는 %ebx 가 이미 가장 큰 숫자를 보관하고 있으므로, 해당 값을 그대로 종료 상태로 사용할 수 있다.

```
movl $1, %eax  
int 0x80
```

위 명령어는 종료 시스템 호출 번호를 %eax 에 저장한 후, int 0x80 명령어를 통해 커널을 호출하여 프로그램을 종료한다.

[3-25] 정리 및 학습 방법 제안

비록 매우 작은 프로그램이지만, 이를 구현하고 이해하기 위해 많은 설명과 계획이 필요했다. 그러나 이 과정을 통해 루프, 조건부 점프, 레지스터 사용, 시스템 호출 등 어셈블리 프로그래밍의 핵심 개념을 학습할 수 있다.

이제 전체 프로그램을 다시 한 번 처음부터 끝까지 읽어보며, 주석을 중심으로 각 명령어가 어떤 역할을 하는지 확인해 보자. 이해되지 않는 부분이 있다면 해당 섹션으로 돌아가 다시 검토하는 것이 좋다. 또한 종이에 프로그램을 단계별로 적어가며 각 레지스터의 값 변화를 기록해 보면, 프로그램의 동작을 더욱 명확하게 이해할 수 있을 것이다.

[4] 주소 지정 모드

13 페이지에 있는 데이터 접근 방법 섹션에서는 어셈블리 언어에서 사용할 수 있는 다양한 주소 지정 모드에 대해 살펴보았다. 이 섹션에서는 그러한 주소 지정 모드들이 실제 어셈블리 언어 명령어 안에서 어떻게 표현되는지를 설명한다.

[4-1] 메모리 주소 참조의 일반 형식

어셈블리 언어에서는 메모리 주소를 참조하는 일반적인 형식은 다음과 같다.

ADDRESS_OR_OFFSET(%BASE_OR_OFFSET, %INDEX, MULTIPLIER)
--

이 형식에서 각 요소는 모두 선택 사항이며, 필요에 따라 생략할 수 있다.

[4-2] 주소 계산 방식

최종적으로 사용되는 메모리 주소는 다음과 같은 계산식을 통해 결정된다.

최종 주소 = 주소_또는_오프셋 + %기본_또는_오프셋 + (곱셈기 × %인덱스)

여기서 다음과 같은 규칙이 적용된다.

- 주소_또는_오프셋 과 곱셈기(MULTIPLIER)는 상수 값이어야 한다.
- %기본_또는_오프셋과 %인덱스는 레지스터여야 한다.
- 만약 특정 요소가 생략되면, 해당 값은 계산식에서 0 으로 간주된다.

이러한 규칙 덕분에 다양한 형태의 주소 지정이 하나의 통일된 방식으로 처리될 수 있다.

[4-3] 주소 지정 모드의 통합 표현

데이터 접근 방법 섹션에서 소개된 모든 주소 지정 모드는, 즉시 주소 지정 모드(immediate mode)를 제외하고, 위에서 설명한 형식으로 표현할 수 있다. 즉, 이 하나의 문법 구조만 이해하면 대부분의 메모리 접근 방식이 어떻게 동작하는지 해석할 수 있다.

[5] 주소 지정 모드의 종류

어셈블리 언어에서 메모리에 저장된 데이터나 레지스터에 접근하기 위해 여러 가지 주소 지정 모드(Addressing Mode)를 사용한다. 각 모드는 메모리 주소를 계산하는 방식에 따라 구분되며, 프로그램의 목적에 따라 적절한 방식을 선택해야 한다.

[5-1] 직접 주소 지정 모드(Direct Addressing Mode)

직접 주소 지정 모드는 ADDRESS_OR_OFFSET 부분만 사용하는 방식이다.

```
movl ADDRESS, %eax
```

이 명령어는 메모리 주소 ADDRESS 에 저장된 값을 %eax 레지스터로 로드한다. 주소가 코드에 직접 명시되므로 이해하기 쉽지만, 유연성은 떨어진다.

[5-2] 인덱스 주소 지정 모드(Indexed Addressing Mode)

인덱스 주소 지정 모드는 ADDRESS_OR_OFFSET 와 %INDEX 를 함께 사용하는 방식이다.

- 인덱스 레지스터로는 일반 목적 레지스터를 사용할 수 있다.
- 인덱스에 1, 2, 또는 4 의 상수 배수를 적용할 수 있어 바이트, 워드, 더블 워드 단위 접근이 가능하다.

예제

바이트 문자열이 string_start 에 저장되어 있고, 세 번째 바이트(인덱스 2)에 접근하고 싶다고 가정하자. 이때 %ecx 가 2 를 보유하고 있다면 다음과 같이 작성할 수 있다.

```
movl string_start(%ecx,1), %eax
```

이는 string_start 에서 시작하여 $1 \times \%ecx$ 만큼 주소를 이동한 위치의 값을 %eax 에 로드한다.

[5-3] 간접 주소 지정 모드(Indirect Addressing Mode)

간접 주소 지정 모드는 레지스터가 가리키는 주소에 저장된 값을 접근하는 방식이다.

예를 들어 %eax 에 메모리 주소가 저장되어 있다면, 그 주소에 있는 값을 %ebx 로 가져올 수 있다.

```
movl (%ebx), %ebx
```

즉, %ebx 는 값이 아니라 주소를 보유하고 있는 레지스터이다.

[5-4] 기준 포인터 주소 지정 모드(Base Pointer Addressing Mode)

기준 포인터 주소 지정 모드는 간접 주소 지정과 유사하지만, 레지스터에 저장된 주소에 상수 오프셋을 더한 위치에 접근한다.

예제

레코드의 시작 주소가 %eax 에 있고, 나이 값이 레코드의 시작으로부터 4 바이트 떨어진 위치에 있다면

```
movl 4(%eax), %ebx
```

이 명령어는 %eax + 4 위치의 값을 %ebx 로 로드한다.

[5-5] 즉시 주소 지정 모드(Immediate Mode)

즉시 주소 지정 모드는 메모리가 아닌 실제 값을 직접 로드하는 방식이다. 이 모드는 앞에서 설명한 일반적인 주소 형식을 따르지 않는다.

```
movl $12, %eax
```

여기서 \$기호는 즉시 값(immediate value)임을 나타낸다.

만약 \$를 생략하면, 숫자 12 는 값이 아니라 메모리 주소로 해석된다.

즉시 모드는 원본 피연산자(source operand)로만 사용할 수 있다.

[5-6] 레지스터 주소 지정 모드(Register Addressing Mode)

레지스터 주소 지정 모드는 레지스터 간에 데이터를 이동하는 방식이다.

```
movl %eax, %ebx
```

지금까지의 대부분의 예제에서 사용된 가장 기본적인 주소 지정 모드이며, 가장 빠르고 효율적인 방식이다.

[5-7] 데이터 크기와 레지스터 사용

지금까지 데이터를 한 번에 워드 단위(4 바이트)로 이동하기 위해 movl 명령어를 사용해 왔다. 그러나 경우에 따라 바이트 단위 이동이 필요할 수 있다.

이를 위해 다음과 같은 명령어가 사용된다.

- movb: 1 바이트 이동
- movw: 2 바이트 이동
- movl: 4 바이트 이동

[5-8] 부분 레지스터 사용

레지스터는 전체를 사용할 수도 있고, 일부나 사용할 수도 있다.

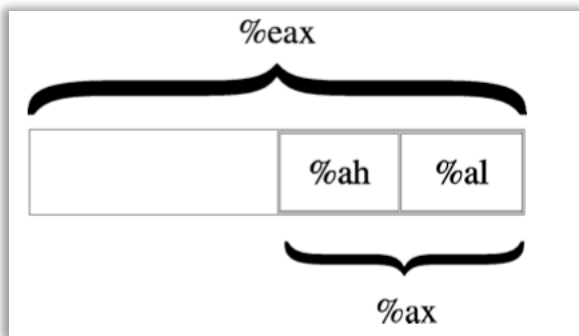
예를 들어 `%eax` 레지스터는 다음과 같이 나뉜다.

- `%eax`: 4 바이트
- `%ax`: 하위 2 바이트
- `%al`: 하위 1 바이트
- `%ah`: 상위 1 바이트

주의할 점

- `%eax` 에 값을 로드하면 `%ax`, `%al`, `%ah` 도 함께 변경된다.
- `%al` 이나 `%ah` 에 값을 로드하면 `%eax` 의 기존 값이 손상된다.

따라서 바이트와 워드를 동시에 섞어 사용하는 것은 피하는 것이 바람직하다.



문제

[1] 개념 이해하기

- {1} 프로그램에서 줄이 '#' 문자로 시작하면 무엇을 의미하는가
- {2} 어셈블리 언어 파일과 오브젝트 코드 파일의 차이는 무엇인가
- {3} 링커는 무엇을 하는가
- {4} `movl $1, %eax` 와 `movl 1, %eax` 의 차이는 무엇인가
- {5} 어떤 레지스터가 시스템 호출 번호를 저장하는가
- {6} 인덱스는 무엇에 사용되는가
- {7} 인덱스는 왜 보통 `eax` 에서 시작하는가
- {8} 명령어 `movl data_items(%edi4), %eax` 를 실행했을 때 `data_items` 의 주소가 3634 이고 `%edi` 가 13 의 값을 가지고 있다면, `%eax` 로 이동할 주소는 무엇인가
- {9} 일반 목적 레지스터를 나열하라
- {10} `movl` 과 `movb` 의 차이는 무엇인가
- {11} 흐름 제어란 무엇인가
- {12} 조건부 점프는 무엇을 하는가
- {13} 프로그램을 작성할 때 계획해야 할 사항은 무엇인가
- {14} 모든 명령어를 검토하고 각 피연산자에 대해 어떤 주소 지정 모드가 사용되고 있는지 나열하라

[2] 개념 사용하기

- {1} 첫 번째 프로그램을 수정하여 값 3 을 반환하도록 하라
- {2} 최대 프로그램을 수정하여 대신 최소값을 찾도록 하라
- {3} 최대 프로그램을 수정하여 목록을 끝내기 위해 숫자 0 대신 숫자 255 를 사용하도록 하라
- {4} 최대 프로그램을 수정하여 언제 멈출지를 알기 위해 숫자 0 대신 종료 주소를 사용하도록 하라
- {5} 최대 프로그램을 수정하여 언제 멈출지를 알기 위해 숫자 0 대신 길이 카운트를 사용하도록 하라
- {6} 이 명령어 `movl _start, %eax` 는 무엇을 하는가, 주소 지정 모드와 `_start` 의 의미에 대한 개념을 구체적으로 설명하시오. (`movl $_start, %eax` 와 무엇이 다른지)

[3] 심화

- {1} 첫 번째 프로그램에서 `int` 명령어 줄을 제거하도록 수정하라. 새 프로그램을 작성하고, 링크하고, 실행하라. 어떤 오류 메시지가 나타나는지 확인하고 그 이유를 설명하라
- {2} 지금까지 우리는 목록의 끝을 찾는 세 가지 접근 방식에 대해 논의했다. 특별한 숫자를 사용하고, 종료 주소를 사용하고, 길이 카운트를 사용하는 것이다. 어떤 접근 방식이 가장 좋다고 생각하는가, 그 이유를 작성하라. (추가: 목록이 정렬되어 있다는 것을 알고 있다면 어떤 접근 방식을 사용하겠는가, 그이유)

4장 함수에 대한 모든 것

[1] 복잡성 다루기

앞에서 작성한 프로그램들은 하나의 코드 섹션으로만 구성되어 있었다. 그러나 실제로 그런 방식으로 프로그램을 작성한다면 유지 보수가 거의 불가능해진다. 한 부분을 수정했을 때 다른 개발자가 작업 중인 코드에 부정적인 영향을 미칠 수 있기 때문에, 여러 사람이 동시에 프로젝트에 참여하기도 매우 어렵다.

이러한 문제를 해결하기 위해 프로그램은 서로 잘 정의된 인터페이스를 통해 통신하는 여러 개의 독립적인 조각으로 나뉘어야 한다. 이렇게 하면 각 부분을 독립적으로 개발하고 테스트할 수 있으며, 여러 프로그래머가 동시에 작업하더라도 충돌을 최소화할 수 있다.

[1-1] 함수란 무엇인가

프로그래머들은 프로그램을 함수(function)라는 단위로 나누어 복잡성을 관리한다. 함수는 특정 유형의 데이터에 대해 정의된 작업을 수행하는 코드의 단위이다.

예를 들어, 워드 프로세서 프로그램을 생각해보자. 사용자가 키를 입력할 때마다 실행되는 `handle_typed_character` 라는 함수가 있을 것이다. 이 함수는 사용자가 입력한 키와 현재 열려 있는 문서를 입력으로 받아, 그에 맞게 문서를 수정하는 역할을 한다. 이처럼 함수는 정해진 입력을 받아 정해진 동작을 수행하는 독립적인 코드 블록이다.

[1-2] 매개변수와 함수 인터페이스

함수가 처리하도록 전달받는 데이터 항목을 매개변수(parameter)라고 한다.

앞선 예제에서 사용자가 누른 키와 현재 문서는 `handle_typed_character` 함수의 매개변수이다.

함수의 매개변수 목록과, 각 매개변수가 어떻게 사용될 것으로 기대되는지를 묶어 함수 인터페이스(interface)라고 한다. 함수 인터페이스는 설계할 때 특히 신경써야 한다. 프로젝트의 여러 곳에서 해당 함수가 호출될 경우, 인터페이스를 변경하는 것은 매우 어려워진다. 잘 설계된 인터페이스는 함수 내부 구현을 몰라도 사용할 수 있게 해주며, 협업과 유지 보수를 훨씬 수월하게 만든다.

[1-3] 대규모 프로그램과 함수의 역할

전형적인 프로그램은 수백 또는 수천 개의 함수로 구성된다. 각 함수는 작고 명확하게 정의된 작업 하나만 수행하도록 설계되는 것이 이상적이다.

하지만 모든 함수가 동일한 수준에서 동작하는 것은 아니다. 시스템에서는 더 이상 쪼갤 수 없는 기본적인 기능을 제공하는 함수들이 존재한다. 이러한 함수들은 원시 함수(primitive functions)라고 부른다. 이 원시 함수들이 모든 기능의 토대가 된다.

[1-4] 원시 함수의 개념

그래픽 사용자 인터페이스(GUI)를 그리는 프로그램을 예로 들어보자.

메뉴를 생성하는 함수는 텍스트를 출력하고, 아이콘을 그리고, 배경을 칠하며, 마우스 위치를 계산하는 여러 함수들을 호출할 것이다.

그러나 이러한 함수들은 결국 선이나 점을 그리는 것과 같은 가장 기본적인 작업을 수행하는 함수에 도달하게 된다. 이 최하위 단계의 기능은 운영체제가 제공하는 원시 함수에 의해 수행된다.

[1-5] 함수 구조에 대한 두 가지 관점

프로그래밍은 다음 두 가지 관점으로 이해할 수 있다.

- | |
|---|
| <ol style="list-style-type: none">1. 큰 프로그램을 점점 더 작은 조각으로 나누어, 결국 원시 함수에 도달하는 과정2. 원시 함수 위에 점진적으로 기능을 쌓아 올려, 더 큰 기능을 만들어 가는 과정 |
|---|

이 두 관점은 서로 반대처럼 보이지만, 실제로는 같은 구조를 다른 방향에서 바라본 것이다.

[1-6] 어셈블리 언어에서의 함수

어셈블리 언어에서 원시 함수는 일반적으로 시스템 호출(system call)과 동일한 역할을 한다. 하지만 시스템 호출은 운영체제와 직접 상호작용하는 특별한 메커니즘이기 때문에, 이 장에서 다룰 진정한 의미의 함수와는 구분해서 생각해야 한다.

다음은 어셈블리 언어에서 함수가 어떻게 구현되고 호출되는지를 본격적으로 살펴보게 될 것이다.

[2] 함수의 작동원리

함수는 단순한 코드 덩어리가 아니라, 여러 가지 구성 요소들이 함께 작동하여 하나의 독립적인 작업 단위를 다룬다. 대부분의 프로그래밍 언어에서 함수는 비슷한 개념적 요소를 공유하며, 어셈블리 언어 역시 예외는 아니다. 이 섹션에서는 함수가 어떤 요소들로 구성되어 있으며, 각 요소가 어떤 역할을 하는지 살펴본다.

[2-1] 함수 이름

함수의 이름은 함수 코드가 시작되는 메모리 주소를 가리키는 기호(symbol)이다. 어셈블리 언어에서는 함수의 시작 부분에 레이블(label)을 붙여 함수 이름을 정의한다. 이는 이전에 점프(jump)를 위해 사용했던 레이블과 동일한 개념이다.

즉, 함수 이름은 사람이 이해하기 쉬운 이름일 뿐이며, 실제로는 코드 시작 주소에 대한 별칭이다. 함수 호출은 결국 이 주소로 실행 흐름을 이동시키는 작업이다.

[2-2] 함수 매개변수

함수 매개변수(parameter)는 함수가 작업을 수행하기 위해 외부로부터 전달받은 데이터이다. 예를 들어, 수학에서 사인(sin) 함수가 있다고 가정하자. 컴퓨터에게 "2의 사인을 구하라"고 요청한다면, sin은 함수의 이름이고 2는 매개변수가 된다.

함수에 따라 매개변수가 여러 개일 수도 있고, 하나도 없을 수도 있다. 매개변수는 함수가 재사용 가능하도록 만드는 핵심 요소이며, 같은 코드라고 다른 입력 값에 따라 다른 결과를 만들어낼 수 있게 해준다.

[2-3] 지역 변수(Local Variables)

지역 변수는 함수가 실행되는 동안에만 사용되는 임시 데이터 저장 공간이다. 함수가 종료되면 이 변수들은 더 이상 유효하지 않으며, 그 값은 버려진다. 이를 비유하자면, 함수는 호출될 때마다 새 메모지를 하나 받는 것과 같다. 함수는 이 메모지에 계산 중간 결과를 기록하고, 작업이 끝나면 메모지를 버린다. 이 메모지는 다른 함수에서는 볼 수도, 사용할 수도 없다.

지역 변수는 함수의 내부 구현을 외부로부터 숨기기 때문에, 프로그램의 안정성과 가독성을 높이는 데 매우 중요하다.

[2-4] 정적 변수(Static Variables)

정적 변수는 함수 내부에서 사용되지만, 함수가 종료된 후에도 값이 유지되는 변수이다. 함수가 다시 호출되면 이전에 저장된 값을 그대로 사용할 수 있다.

이 변수 역시 함수 외부에서는 접근할 수 없지만, 호출 간 상태가 유지된다는 점에서 지역 변수와 다르다. 이러한 특성 때문에 정적 변수는 때때로 유용하지만, 남용할 경우 프로그램의 흐름을 이해하기 어렵게 만들 수 있다. 그래서 정적 변수는 정말 필요한 경우가 아니라면 사용을 피하는 것이 일반적이다.

[2-5] 전역 변수(Global Variables)

전역 변수는 함수 외부에서 정의되며, 여러 함수가 함께 사용하는 데이터 저장소이다. 예를 들어, 간단한 텍스트 편집기에서는 현재 열려 있는 파일의 전체 내용을 전역 변수에 저장해 두고, 이를 여러 함수에서 참조하도록 만들 수 있다.

또한 프로그램 전체에서 공통으로 사용된 설정 값(configuration)들도 종종 전역 변수로 관리된다. 하지만 전역 변수는 너무 많아지면 프로그램의 의존성이 증가하고, 버그를 추적하기 어려워질 수 있으므로 신중하게 사용해야 한다.

[2-6] 반환 주소(Return Address)

반환 주소는 함수의 매개변수처럼 보이지 않지만, 실제로는 함수 호출 시 자동으로 전달되는 매우 중요한 정보이다. 반환 주소는 함수가 작업을 마친 후 어디로 돌아가야 하는지를 알려준다.

함수는 프로그램의 여러 위치에서 호출될 수 있기 때문에, 항상 동일한 위치로 돌아갈 수는 없다. 따라서 호출 시점마다 반환 주소가 달라진다.

대부분의 고급 언어에서는 이 과정이 자동으로 처리된다. 어셈블리 언어에서는 call 명령어가 반환 주소를 전달하고 ret 명령어는 그 주소를 사용하여 호출한 위치로 되돌아간다.

[2-7] 반환 값(Return Value)

반환 값은 함수가 계산한 결과를 호출한 쪽으로 돌려는 주요 수단이다.
대부분의 프로그래밍 언어에서는 함수가 하나의 반환 값만을 갖도록 제한한다.

여러 값을 반환해야 할 경우에는 구조체, 포인터, 전역 변수 등의 다른 방법을 사용한다.
어셈블리 언어에서는 반환 값이 특정 레지스터에 저장되는 방식으로 전달된다.

[2-8] 호출 규약(Calling Convention)

지금까지 살펴본 요소들은 대부분의 프로그래밍 언어에서 공통적으로 존재하지만, 변수의 저장 위치, 매개변수 전달 방식, 반환 값 전달 방식은 언어와 플랫폼마다 다르다.

이러한 약속을 호출 규약(calling convention)이라고 한다. 호출 규약은 함수가 호출될 때 어떤 데이터를 어디서 기대해야 하는지를 정의한다.

어셈블리 언어에서는 원하는 호출 규약을 직접 정의할 수 있다. 그러나 다른 언어(예: C 언어)로 작성된 함수와 함께 동작하려면, 반드시 해당 언어의 호출 규약을 따라야 한다.

이 책에서는 예제를 위해 C 언어의 호출 규약을 사용한다. 이는 가장 널리 사용되며, 리눅스 플랫폼의 사실상 표준이기 때문이다.

[3] C 호출 규약을 사용하는 어셈블리 언어 함수

어셈블리 언어로 함수를 작성하려면 스택(stack)의 동작 방식을 반드시 이해해야 한다. 모든 컴퓨터 프로그램은 함수가 올바르게 동작하도록 돕기 위해 스택이라는 특수한 메모리 영역을 사용한다.

[3-1] 스택의 개념과 동작 방식

스택은 책상 위에 쌓아 올린 종이 더미처럼 생각할 수 있다.

일반적으로 가장 최근 추가한 항목을 가장 먼저 꺼내는 구조(LIFO)를 가진다.

컴퓨터의 스택은 메모리의 가장 높은 주소에서 시작하여 아래 방향으로 성장한다.

즉, 우리가 말하는 “스택의 맨 위”는 실제 메모리 주소상으로는 더 낮은 주소에 위치한다.

이 점은 처음에는 혼란스럽지만 매우 중요하다.

낮은 주소	높은 주소
0x00000000	 0xBFFFFFFF

[3-2] pushl 과 popl 명령어

스택에 값을 추가할 때는 pushl 명령어를 사용한다.

- | |
|---|
| <ul style="list-style-type: none">- pushl: 레지스터 또는 메모리 값을 스택의 맨 위에 저장- popl: 스택의 맨 위 값을 제거하여 레지스터 또는 메모리로 이동 |
|---|

값을 스택에 푸시할 때마다 스택은 아래 방향으로 확장되며(스택에 값을 push 할수록 사용되는 메모리 주소의 숫자가 작아진다), 스택에서 값을 팝하면 다시 위쪽으로 줄어든다.

[3-3] 스택 포인터 레지스터(%esp)

현재 스택의 맨 위를 가리키는 레지스터는 %esp 이다.

이 레지스터는 항상 현재 스택 최상단의 주소를 보관한다.

- | |
|--|
| <ul style="list-style-type: none">- pushl 실행 시 -> %esp 는 4 감소- popl 실행 시 -> %esp 는 4 증가 |
|--|

(32 비트 환경에서는 한 워드는 4 바이트이기 때문이다.)

[3-4] 스택 값에 직접 접근하기

스택의 맨 위 값을 제거하지 않고 읽고 싶다면 간접 주소 지정 모드를 사용한다.

```
movl (%esp), %eax
```

이 명령은 스택의 맨 위 값을 `%eax` 로 복사한다.

반면 다음 코드는 전혀 다른 의미이다.

```
movl %esp, %eax
```

이 경우 `%eax` 에는 값이 아니라 주소(`%esp` 자체)가 저장된다.

[3-5] 스택 오프셋 접근

스택의 맨 위 바로 아래에 있는 값을 읽고 싶다면 오프셋을 사용한다.

```
movl 4(%esp), %eax
```

이는 기준 포인터 주소 지정 방식으로, `%esp+4` 가 가리키는 위치의 값을 가져온다.

```
ESP = 996
```

주소 ↑

1000 ← 4(%esp) (오래된 값)
996 ← (%esp) (맨 위, 최신 값)
992 ← -4(%esp) (다음 push 자리)

[3-6] C 호출 규약에서의 스택 역할

C 언어 호출 규약에서 스택은 다음을 저장하는 핵심 구조이다.

- 함수 매개변수
- 반환 주소
- 지역 변수

[3-7] 함수 호출 시 스택 상태

함수가 호출되기 전에 호출자는 매개변수를 역순으로 스택에 푸시한다.
그 다음 call 명령어를 실행한다.

call 명령어는 두 가지 작업을 수행한다.

1. 다음에 실행할 명령어 주소(반환 주소)를 스택에 푸시
2. 명령어 포인터 %eip 를 함수 시작 주소로 변경

이 시점의 스택 구조는 다음과 같다.

매개변수 #N

...

매개변수 2

매개변수 1

반환 주소 <--- (%esp)

[3-8] 베이스 포인터(%ebp)와 스택 프레임

함수가 시작되면 가장 먼저 다음 작업을 수행한다.

```
pushl %ebp  
movl %esp, %ebp
```

이 작업의 목적은 고정된 기준점을 만드는 것이다.

- %ebp 는 함수 시작 시점의 스택 위치를 기억(이 함수의 스택의 기준점)
- 이 영역을 스택 프레임(stack frame)이라고 부른다.

이후 스택에서 변화가 생겨도 %ebp 는 변하지 않으므로, 함수 매개변수와 지역 변수에 항상 일정한 오프셋으로 접근 가능하다.

[3-9] 매개변수 접근 방식

함수 내부에서 매개변수는 다음과 같이 접근한다.

매개변수 #1 -> 8(%ebp)

매개변수 #2 -> 12(%ebp)

...

매개변수 #N -> N*4 + 4(%ebp)

[3-10] 지역 변수 공간 확보

함수는 필요한 지역 변수 공간을 스택에 직접 확보한다.

```
subl $8, %esp
```

이 명령은 두 개의 지역 변수(2 워드)를 위한 공간을 만든다.

함수 진입 직후 스택

높은 주소

매개변수

반환 주소

이전 %ebp ← %ebp

낮은 주소 ← %esp

subl \$8, %esp 실행 후

높은 주소

매개변수

반환 주소

이전 %ebp ← %ebp

지역 변수 1 (-4(%ebp))

지역 변수 2 (-8(%ebp)) ← %esp

낮은 주소

지역 변수는 함수 실행 중에만 존재하며, 함수가 반환되면 사라진다.

pushl 명령어는 스택에 값을 저장하는 동작을 수행하면서 동시에 %esp 를 자동으로 감소시킨다. 즉, 스택에 실제로 의미 있는 데이터가 하나 들어가게 된다. 이 방식은 이미 저장해야 할 값이 존재할 때는 매우 적절하지만, 단순히 저장 공간만 필요한 경우에는 불필요한 값까지 함께 스택에 쌓게 되는 단점이 있다.

반면, 함수의 지역 변수는 많은 경우 함수가 실행되는 동안 임시로 사용할 메모리 공간만 필요할 뿐, 처음부터 특정 값으로 초기화될 필요는 없다. 이때 subl \$N, %esp 를 사용하면 %esp 를 아래로 이동시켜 필요한 크기만큼의 빈 공간을 한 번에 확보할 수 있다. 이 공간은 이후 -4(%ebp), -8(%ebp)와 같은 방식으로 접근하여 지역 변수로 사용된다. 따라서 지역 변수 공간을 만들 때는 실제 값을 스택에 쌓는 pushl 보다, 의미 없는 값을 넣지 않고 공간만 확보할 수 있는 subl 방식이 더 효율적이고 명확한 방법이 된다.

[3-11] 함수 종료 과정

함수가 종료될 때는 다음 순서를 따른다.

1. 반환 값을 %eax 에 저장
2. 스택 프레임 복원
3. 호출 위치로 복귀

```
movl %ebp, %esp  
popl %ebp  
ret
```

ret 명령은 스택에서 반환 주소를 팝하여 %eip 에 넣는다.

[3-12] 지역 변수의 생명 주기 주의사항

스택 프레임이 제거되면 해당 지역 변수 영역은 언제든지 덮어써질 수 있다.
따라서 지역 변수의 주소를 함수 밖에서 사용하면 심각한 오류가 발생한다.

[3-13] 호출자 측의 마무리 작업

함수가 반환되면 호출자는 다음을 수행해야 한다.

- %eax 에서 반환 값 확인
- 스택에 푸시한 매개변수 제거

```
addl $4 * 매개변수_개수, %esp
```

[3-14] 레지스터 파괴 규칙

함수를 호출할 때 모든 레지스터가 보존된다고 가정해서는 안된다.

- %ebp 만 호출 시점 값이 보장됨
- %eax 는 반드시 덮어써짐
- 나머지 레지스터도 안전하지 않음

보존이 필요한 레지스터는 함수 호출 전에 직접 스택에 저장해야 한다.

[3-15] 호출 규약의 중요성

호출 규약은 언어마다 다를 수 있다.

언어를 혼합하거나 외부 함수를 호출할 경우 호환 여부를 반드시 확인해야 한다.

C 호출 규약의 전체 사양(ABI)은 다음 문서에서 확인할 수 있다.

- Linux ABI Specification(LSB)

[4] 함수 예제: power 함수 분석

[4-1] 함수 예제 개요

실제 프로그램에서 함수 호출이 어떻게 작동하는지를 살펴본다.

이 예제에서 작성할 함수는 power 함수로, 두 개의 매개변수를 받아 숫자의 거듭제곱을 계산한다.

power 함수는 다음 두 가지 매개변수를 가진다.

첫 번째는 기준이 되는 숫자이며, 두 번째는 해당 숫자를 몇 제곱할 것인지를 나타내는 값이다. 예를 들어, 매개변수로 2와 3을 전달하면 결과는 2의 3 제곱인 8이 된다.

프로그램을 단순화하기 위해 거듭제곱 값은 1 이상의 정수만 허용한다.

이 프로그램은 두 개의 계산 결과인 2^3 과 5^2 를 구한 뒤, 그 결과를 더하여 반환한다.

[4-2] 메인 프로그램의 구조

메인 프로그램에서는 모든 값을 레지스터에 저장하므로 데이터 섹션에는 별도의 데이터가 없다. 메인 프로그램의 역할은 다음과 같다.

1. 함수에 전달할 매개변수를 스택에 푸시한다.
2. call 명령어를 사용해 함수를 호출한다.
3. 함수 호출이 끝난 뒤 스택 포인터를 원래 위치로 되돌린다.
4. 반환값은 %eax 레지스터에서 가져온다.

첫 번째 power 함수 호출에서는 2^3 을 계산한다.

반환값은 %eax에 저장되며, 이후 두 번째 함수를 호출을 위해 이 값을 스택에 임시로 저장한다.

두 번째 power 함수 호출에서는 5^2 를 계산한다. 이 결과 역시 %eax에 저장된다. 이후 스택에 저장해 두었던 첫 번째 결과를 꺼내 %ebx이 동 한뒤, 두 값을 더한다.

최종 결과는 %ebx에 저장되며, 프로그램은 이 값을 종료 상태 코드로 사용해 종료된다.

```

# 목적: 함수가 어떻게 작동하는지를 설명하는 프로그램
#  $2^3+5^2$ 
#
# 메인 프로그램의 모든 것은 레지스터에 저장되므로,
# 데이터 섹션에는 아무것도 없다.
.section .data
.section .text

.global _start
_start:
    pushl $3      # 두 번째 인수를 푸시
    pushl $2      # 첫 번째 인수를 푸시
    call power    # 스택 포인터를 되돌린다

    pushl %eax    # 첫 번째 결과를 저장한 후 다음 함수를 호출

    pushl $2      # 두 번째 인수를 푸시
    pushl $5      # 첫 번째 인수를 푸시
    call power    # 함수 포인터 되돌린다

    popl %ebx     # 두 번째 결과는 이미 %eax 에 있다.
                # 첫 번째 결과를 스택에 저장하였기에
                # 팝하여 %ebx 로 옮길 수 있다.

    addl %eax, %ebx    # 둘을 더하며 결과는 %ebx 에 있다
    movl $1, %eax      # 종료(%ebx 가 반환)
    int $0x80

.type power, @function
power:
    pushl %ebp        # 이전 베이스 포인터를 저장
    movl %esp, %ebp   # 스택 포인터를 베이스 포인터로 만들기

    movl 8(%ebp), %ebx # 첫 번째 인수를 %eax 에 넣기
    movl 12(%ebp), %ecx # 두 번째 인수를 %ecx 에 넣기

```



```
power_loop_start:
    cmpl $1, %ecx
    je end_power

    movl -4(%ebp), %eax
    imull %ebx, %eax

    movl %eax, -4(%ebp)

    decl %ecx
    jmp power_loop_start

end_power:
    movl -4(%ebp), %eax
    movl %ebp, %esp
    popl %ebp
    ret
```

[4-3] power 함수의 역할과 인터페이스

power 함수는 숫자의 거듭제곱을 계산하는 함수이다.

입력 값

- 첫 번째 인수: 기준 숫자
- 두 번째 인수: 거듭제곱 값

출력 값

- 계산된 결과를 반환값으로 제공한다. 반환값은 %eax 레지스터에 저장된다.

사용되는 주요 변수

- %ebx: 기준 숫자 저장
- %ecx: 거듭제곱 값 저장
- -4(%ebp): 현재 계산 결과를 저장하는 지역 변수
- %eax: 곱셈 및 임시 계산에 사용되는 레지스터

함수 앞부분에는 주석을 통해 함수의 목적, 입력, 출력, 제약 조건이 명확히 문서화되어 있다. 이러한 정보는 해당 함수를 사용하는 프로그래머에게 함수의 인터페이스를 제공하는 역할을 한다.

[4-4] 함수 선언과 진입 과정

.type power, @function 지시어는 링커에게 power 심볼이 함수임을 알려준다.

이 프로그램은 단일 파일로 구성되어 있으므로 필수는 아니지만, 올바른 관행으로 권장된다.

power: 레이블은 함수 코드의 시작 주소를 나타낸다.

call power 명령어는 이 레이블이 가리키는 주소로 제어를 이동시키며, 동시에 반환 주소를 스택에 저장한다.

[4-5] 함수 초기화와 스택 프레임 구성

함수가 시작되면 가장 먼저 다음 작업을 수행한다.

1. 이전 함수의 베이스 포인터 %ebp 를 스택에 저장한다.
2. 현재 스택 포인터 %esp 를 %ebp 로 복사한다.
3. 지역 변수 저장을 위해 스택 공간을 확보한다.

이 과정을 통해 함수 전용 스택 프레임이 생성된다. 스택 프레임은 함수 실행 동안 고정된 기준점 역할을 하며, 함수의 매개변수와 지역 변수에 일정한 오프셋으로 접근할 수 있게 해준다. 이 시점에서 스택에는 매개변수, 반환 주소, 이전 %ebp, 그리고 지역 변수가 정해진 구조로 배치된다.

[4-6] power 함수의 동작 원리

함수는 기준 숫자를 초기 결과값으로 설정한 뒤, 거듭제곱 값이 1 이 될 때까지 반복적으로 곱셈을 수행한다.

현재 결과는 지역 변수에 저장되며, 매 반복마다 기준 숫자를 곱한다.
곱셈이 수행될 때마다 거듭제곱 값은 1 씩 감소한다.

거듭제곱 값이 1 이 되면 루프를 종료하고, 최종 결과를 %eax 레지스터로 이동한다.

[4-7] 함수 종료 및 반환

함수가 종료될 때는 다음 과정을 거친다.

1. 반환값을 %eax 에 저장한다.
2. 스택 포인터를 %ebp 값으로 복원하여 현재 스택 프레임을 제거한다.
3. 이전 %ebp 값을 복원한다.
4. ret 명령어를 통해 호출한 위치로 제어를 반환한다.

이 과정을 통해 함수 실행 중 사용된 지역 변수 공간은 자동으로 폐기된다.

[4-8] 전체 프로그램에 대한 정리

메인 프로그램은 함수 호출과 결과 결합만을 담당하며, 실제 계산 로직은 power 함수에 캡슐화되어 있다. 이러한 구조는 코드의 가독성과 유지보수성을 높여준다.

두 번의 함수 호출 사이에서 결과를 스택에 저장하는 이유는, 함수 호출 시 대부분의 레지스터 값이 보존되지 않기 때문이다. %ebp 만이 호출 간에 안정적으로 유지되는 레지스터이므로, 필요한 값은 스택에 저장하는 방식이 사용된다.

프로그램이 제대로 동작하지 않을 경우, 종이에 스택 구조를 그려가며 %esp 와 %ebp 의 변화, 각 레지스터의 값을 단계별로 추적해보는 것이 도움이 된다.

[5] 스택을 사용하는 함수의 실제 동작 이해

[5-1] x86(32 비트) C 호출 규약(cdecl) 기반 factorial 함수 분석

이 예제는 어셈블리어에서 스택이 실제로 어떻게 사용되는지를 한 번에 정리해 주는 대표적인 사례이다. 특히 ESP, EBP, 인자 전달, 반환 주소, 재귀 호출 시 스택 변화까지 모두 포함되어 있어, 지금까지 헷갈렸던 개념들이 자연스럽게 정리되는 것을 목표로 한다.

```
# factorial 함수 정의(C에서 호출 가능)
.type factorial, @function
factorial:
    # -----
    # 함수 프로로그(function prologue)
    # -----

    pushl %ebp          # 이전 함수의 EBP 를 스택에 저장
                        # (함수 종료 시 원래 상태로 복구)

    movl %esp, %ebp     # 현재 ESP 를 EBP 에 복사
                        # 이 함수의 스택 기준점(frame pointer) 설정
                        # 이후 인자/지역변수 접근은 EBP 기준으로 함

    # -----
    # 함수 인자 읽기
    # -----
    movl 8(%ebp), %eax  # 첫 번째 인자 n 을 EAX 로 로드
                        # 4(%ebp) : return address
                        # 8(%ebp): 첫 번째 인자 n

    # -----
    # base case 검사: n == 1 ?
    # -----
    cmpl $1, %eax       # n 과 1 비교
    je end_factorial    # n == 1 이면 바로 종료
                        # (EAX 에 이미 1 이 들어 있으므로 반환값 준비 완료)

    # -----
    # 재귀 호출 준비: factorial(n-1)
    # -----
    decl %eax           # EAX = n - 1
    pushl %eax          # n-1 을 다음 factorial 호출의 인자로 push
```

```

call factorial      # factorial(n-1) 호출
                   # call 은 return address 를 자동으로 스택에 push
                   # 반환값은 EAX 에 저장

# -----
# 결과 계산: n * factorial(n-1)
# -----
movl 8(%ebp), %ebx #원래 인자 n 을 다시 EBX 로 로드
                   #(EAX 는 재귀 호출 결과로 이미 사용 중이므로)

imull %ebx, %eax    # EAX = EAX * EBX
                   # EAX = factorial(n-1) * n
                   # 반환값은 EAX 에 저장(cdecl 규칙)

# -----
# 함수 에필로그
# -----
end_factorial:
movl %ebp, %esp     # ESP 를 함수 시작 시점으로 복구
                   # (지역 변수, push 된 값들 정리)

popl %ebp           # 이전 함수의 EBP 복구

ret                # return
                   # 스택에서 return address 를 pop 하고
                   # 그 주소로 실행 흐름 복귀

```

[5-2] 함수가 수행하는 작업의 큰 그림

이 어셈블리 코드는 다음과 같은 C 함수와 동일한 동작을 수행한다.

```
int factorial(int n) {  
    if (n == 1) return 1;  
    return n * factorial(n-1);  
}
```

이를 x86 32 비트 환경에서 C 호출 규약(cdecl)에 맞게 구현한 것이다.

[5-3] 전제 조건(호출 규약 및 환경)

- 32 비트 x86 아키텍처
- C 호출 규약(cdecl)
- 함수 인자는 스택을 통해 전달
- 반환값은 EAX 레지스터에 저장
- 스택은 메모리의 높은 주소에서 낮은 주소 방향으로 성장

[5-4] 함수 호출 직후의 스택 상태

C 코드에서 다음과 같이 함수가 호출되었다고 가정한다.

```
factorial(5);
```

call factorial 실행 직후, 함수에 진입한 시점의 스택 구조는 다음과 같다.

높은 주소 ↑

```
8(%esp) : 인자 n = 5  
4(%esp) : return address  
(%esp)  : 아직 이전 EBP 는 저장되지 않음
```

이 시점에서 ESP 는 return address 를 가리키고 있다.

[5-5] 함수 프로로그: 이전 상태 저장 및 기준점 설정

```
pushl %ebp
```

이 명령은 이전 함수의 EBP 값을 스택에 저장한다.

함수 종료 시 원래 상태로 복구하기 위해 반드시 필요하다.

스택 상태는 다음과 같이 변한다.

높은 주소 ↑

12 : 인자 n

8 : return address

4 : 이전 EBP ← push 됨

ESP → 여기

```
movl %esp, %ebp
```

이 명령은 현재 ESP 값을 EBP 에 복사하여, 이 함수의 스택 프레임 기준점(frame pointer)을 설정한다.

이후부터는

- EBP 는 고정된 기준점 역할
- ESP 는 push/pop 에 따라 자유롭게 변할 수 있다.

[5-6] 함수 인자 접근

```
movl 8(%ebp), %eax
```

EBP 기준 스택 구조는 다음과 같다.

(%ebp) : 이전 EBP

4(%ebp) : return address

8(%ebp) : 첫 번째 인자 n

따라서 8(%ebp)는 항상 첫 번째 인자를 가리킨다.

이는 cdecl 호출 규칙에서의 고정된 규칙이다.

[5-7] base case 검사

```
cmpl $1, %eax  
je end_factorial
```

- 인자 n 이 1 인지 비교
- $n == 1$ 이면 재귀를 중단하고 함수 종료
- EAX 에 이미 1 이 들어 있으므로 반환값 준비 완료

[5-8] 재귀 호출 준비 및 실행

```
decl %eax  
pushl %eax  
call factorial
```

동작 과정은 다음과 같다.

1. decl %eax
 - $n - 1$ 계산
2. pushl %eax
 - 다음 factorial 호출의 인자로 스택에 저장
3. call factorial :
 - return address 를 자동으로 스택에 push
 - 함수 실행 후 결과는 EAX 에 저장

[5-9] 재귀 호출 이후의 처리

```
movl 8(%ebp), %ebx
```

재귀 호출이 끝난 시점에서

- EAX 에는 factorial($n-1$)의 결과가 들어 있음
- 원래 인자 n 값이 필요하므로 다시 스택에서 읽어옴

[5-10] 결과 계산

```
imull %ebx, %eax
```

- EAX = factorial($n-1$)
- EBX = n
- 곱셈 결과는 EAX 에 저장

이는 cdecl 규칙에 맞는 반환값 위치이다.

[5-11] 함수 에필로그: 스택 복구 및 반환

```
end_factorial:
```

```
    movl %ebp, %esp
```

```
    popl %ebp
```

```
    ret
```

1. `movl %ebp, %esp` : 함수 시작 시점의 스택 상태로 ESP 복구
2. `popl %ebp` : 이전 함수의 EBP 복원
3. `ret` : return address 를 pop 하고 해당 주소로 복귀

[5-12] 이 함수의 스택 구조

높은 주소

함수 인자 n ← 8(%ebp)

return address ← 4(%ebp)

이전 EBP ← (%ebp)

낮은 주소

문제

[1] 개념 이해하기

- {1} primitive 란 무엇인가
- {2} 호출 규약이란 무엇인가
- {3} 스택이란 무엇인가
- {4} pushl 과 popl 이 스택이 어떤 영향을 미치는지, 어떤 특수 목적 레지스터에 영향을 미치는지
- {5} 지역 변수란 무엇이며, 무엇에 사용되는지
- {6} 재귀 함수에서 지역 변수가 왜 그렇게 필요한지
- {7} %ebp 와 %esp 는 무엇에 사용되는지
- {8} 스택 프레임이란 무엇인지

[2] 개념 사용하기

- {1} 하나의 인수를 받아 그 인수의 제곱을 반환하는 square 라는 함수 작성하기
- {2} 작성한 square 함수를 테스트하는 프로그램을 작성하기
- {3} 3 장 '최대값 찾기' 섹션에 주어진 최대 프로그램을 변환하여 여러 값에 대한 포인터를 받아 최대값을 반환하는 함수를 만들기. 3 개의 서로 다른 리스트로 maximum 을 호출하고, 마지막 리스트의 결과를 프로그램의 종료 상태 코드로 반환하는 프로그램을 작성하기
- {4} 표준 호출 규약이 없을 경우 발생할 문제를 설명하기

[3] 심화

- {1} 더 큰 기본 연산 집합이 더 작은 연산 집합으로 표현될 수 있다고 가정할 때, 시스템은 많은 수의 기본 연산(primitives)을 갖는 것이 더 나을까, 아니면 적은 수의 기본 연산을 갖는 것이 더 좋은지, 그 이유는 무엇인지
- {2} 팩토리얼 함수는 재귀를 사용하지 않고도 작성할 수 있다. 재귀를 사용하지 않는 방식으로 팩토리얼 함수를 작성하기
- {3} 정기적으로 사용하는 컴퓨터의 응용 프로그램 하나를 떠올려 본다. 그 프로그램에서 특정 기능 하나를 선택한 뒤, 그 기능을 여러 개의 함수로 나누어 보기, 또한 해당 기능과 프로그램의 나머지 부분 사이에 필요한 함수 인터페이스를 정의하기
- {4} 자신만의 호출 규약을 하나 설계하기, 더 나아가 이 장에서 나오는 프로그램들을 그 호출 규약을 사용하도록 다시 작성하기

예시

- 매개변수 스택이 아니라 레지스터로 전달
- 매개변수 전달 순서를 변경하기
- 반환 값을 다른 레지스터나 메모리 위치에 저장하도록 하는 방식
- {5} 스택을 사용하지 않고 호출 규약을 만들 수 있는지, 가능하다면 호출 규약은 어떤 제약 사항이나 한계점을 가지는지

5 장 파일 처리

많은 컴퓨터 프로그래밍은 파일과 밀접한 관계가 있다. 컴퓨터를 재부팅하면 이전 세션에서 남아 있는 것은 결국 디스크에 저장된 데이터 뿐이다. 이처럼 파일에 저장된 데이터는 프로그램이 실행되지 않을 때에도 디스크에 계속 남아 있기 때문에 지속적인 데이터(persistend data)라고 불린다.

[1] 유닉스 파일 개념

각 운영체제는 파일을 처리하는 고유한 방식을 가지고 있다. 그 중에서도 리눅스에서 사용하는 UNIX 방식은 가장 단순하고 보편적인 방법이다. UNIX 에서 파일은 어떤 프로그램이 생성했는지와 관계없이 모두 바이트의 순차적 스트림으로 취급된다.

파일에 접근하기 위해서는 먼저 파일 이름을 이용해 파일을 연다. 운영체제는 해당 파일을 참조하기 위해 사용할 수 있는 파일 설명자(file descriptor)라는 숫자를 프로그램에 제공한다. 이후 프로그램은 이 파일 설명자를 사용하여 파일을 읽거나 쓸 수 있다. 읽기와 쓰기 작업이 끝나면 파일을 닫게 되며, 이때 해당 파일 설명자는 더 이상 유효하지 않게 된다.

[2] 파일 처리 절차

우리의 프로그램에서는 다음과 같은 절차로 파일을 처리한다.

[2-1] 파일 열기(open 시스템 호출)

먼저 리눅스에 열고자 하는 파일의 이름과 파일을 어떤 모드로 열 것인지를 알려준다. 이는 open 시스템 호출을 통해 수행되며, 파일 이름, 모드를 나타내는 숫자, 그리고 권한 집합을 매개변수로 사용한다.

이때 `%eax`에는 시스템 호출 번호가 저장되며, open의 시스템 호출 번호는 5이다. 파일 이름의 첫 번째 문자의 주소는 `%ebx`에 저장되어야 한다. 파일의 읽기/쓰기 의도는 숫자로 표현되며 `%ecx`에 저장된다. 현재 기준으로 읽기 전용 파일에는 0을 사용하고, 쓰기용 파일에는 03101을 사용한다. (선행 0 포함)

마지막으로 파일의 권한 집합은 `%edx`에 숫자로 저장된다. UNIX 권한에 익숙하지 않다면 0666을 사용하면 된다. 이때도 선행 0을 반드시 포함해야 한다.

[2-2] 파일 설명자 반환

open 시스템 호출이 성공하면, 리눅스는 `%eax` 레지스터에 파일 설명자를 반환한다. 이 값은 프로그램 전체에서 해당 파일을 참조하는 데 사용되는 중요한 식별자이므로 반드시 기억해야 한다.

[2-3] 파일 읽기 및 쓰기(read/write 시스템 호출)

다음 단계에서는 파일에 대해 읽기 및 쓰기 작업을 수행한다. 이때마다 리눅스에 파일 설명자를 제공해야 한다.

`read`는 시스템 호출 번호 3이며, 호출 시 `%ebx`에는 파일 설명자, `%ecx`에는 읽은 데이터를 저장할 버퍼의 주소, `%edx`에는 버퍼의 크기가 저장되어야 한다. 버퍼는 이후에 설명될 `.bss` 섹션에 위치한다. `read`는 읽어들이는 바이트 수 또는 오류 코드를 반환하는데, 오류 코드는 항상 음수이므로 이를 통해 구별할 수 있다.

`write`는 시스템 호출 번호 4이며, `read`와 동일한 매개변수를 요구한다. 단, 이 경우 버퍼에는 이미 기록할 데이터가 채워져 있어야 한다. `write` 역시 `%ebx`에 기록된 바이트 수 또는 오류 코드를 반환한다.

[2-4] 파일 닫기(close 시스템 호출)

파일 작업이 모두 끝나면, 리눅스에 파일을 닫도록 지시해야 한다. 이는 `close` 시스템 호출(번호 6)을 통해 수행된다. `close`의 유일한 매개변수는 `%ebx`에 저장된 파일 설명자이다. 파일을 닫은 이후에는 해당 파일 설명자를 더 이상 사용할 수 없다.

[3] 버퍼와 .bss 섹션

이전 섹션에는 버퍼에 대해 언급했지만, 그것이 무엇인지에 대한 설명은 하지 않았다. 버퍼(buffer)란 대량의 데이터를 전송하기 위해 사용되는 연속적인 바이트 블록이다. 파일을 읽을 때 운영체제는 읽어온 데이터를 저장할 장소가 필요하며, 이 저장 공간이 바로 버퍼이다.

일반적으로 버퍼는 데이터를 임시로 저장하는 용도로 사용된다. 이후 프로그램은 버퍼에 저장된 데이터를 읽어, 처리하기 쉬운 형태로 변환한다. 그러나 우리의 프로그램은 그렇게 복잡하지 않기 때문에 반드시 이런 변환 과정을 거칠 필요는 없다.

예를 들어, 파일에서 한 줄의 텍스트를 읽고 싶지만 그 줄의 길이를 모른다고 가정해 보자. 이 경우, 일정한 수의 바이트(또는 문자)를 버퍼로 읽어 들인 후 줄 끝 문자를 찾는다. 줄 끝 문자를 발견하면, 그 지점까지의 모든 문자를 다른 위치로 복사한다. 만약 줄 끝 문자를 찾지 못했다면, 또 다른 버퍼를 할당하여 계속해서 파일에서 데이터를 읽는다. 이 과정에서 이전 버퍼에 일부 문자가 남아 있을 수 있으며, 이후 파일에서 데이터를 다시 읽을 때 이 남은 데이터를 시작점으로 사용할 수 있다.

또 한가지 주목해야 할 점은, 버퍼의 크기는 프로그래머가 미리 정해 놓은 고정된 크기라는 것이다. 예를 들어, 한 번에 500 바이트씩 데이터를 읽고 싶다면, 사용되지 않는 500 바이트 크기의 메모리 위치를 read 시스템 호출에 전달하고, 동시에 크기 값으로 숫자 500 을 전달해야 한다. 버퍼의 크기는 애플리케이션의 필요에 따라 더 작게 또는 더 크게 조정할 수 있다.

버퍼를 생성하려면 정적 저장소 또는 동적 저장소를 예약해야 한다. 정적 저장소는 지금까지 다뤄온 방식으로, 선언된 저장 위치를 사용하는 것이다. 이는 .long 이나 .byte 지시어를 통해 이루어진다. 동적 저장소는 이후 다룰 “더 많은 메모리 얻기” 섹션에서 설명될 것이다.

하지만 .byte 지시어를 사용해 버퍼를 선언하는 데에는 몇 가지 문제가 있다. 첫째, 입력하기가 번거롭다. 예를 들어, 500 바이트 크기의 버퍼를 만들기 위해서는 .byte 선언 뒤에 500 개의 숫자를 입력해야 한다. 이 숫자들은 단지 공간을 차지할 뿐, 실제로는 아무런 의미도 없다. 둘째, 이렇게 선언된 데이터는 실행 파일 자체에서 공간을 차지하게 된다. 지금까지의 예제에서는 큰 문제가 되지 않지만, 프로그램이 커질수록 이는 심각한 문제가 될 수 있다. 500 바이트가 필요할 때마다 실행 파일에 500 바이트를 그대로 낭비하게 되기 때문이다.

이 두 가지 문제를 해결하기 위한 방법이 있다. 지금까지 우리는 .text 섹션과 .data 섹션, 이렇게 두 개의 프로그램 섹션에 대해서만 이야기해 왔다. 하지만 여기에 .bss 섹션이라는 또 다른 섹션이 존재한다. .bss 섹션은 .data 섹션과 유사하지만, 실행 파일에서 실제 공간을 차지하지 않는다는 중요한 차이점이 있다.

.bss 섹션에서는 저장 공간을 예약만 할 수 있고 초기화는 할 수 없다. 반면, .data 섹션에서는 저장 공간을 예약함과 동시에 초기 값을 설정할 수 있다. .bss 섹션은 버퍼를 선언하는 데 매우 적합한데, 그 이유는 버퍼는 어차피 초기화할 필요가 없고 단지 저장 공간만 확보하면 되기 때문이다.

이 작업을 수행하기 위해, 우리는 다음과 같은 명령을 사용한다.

[3-1] .bss 섹션과 .lcomm 지시어

```
.section .bss  
.lcomm my_buffer, 500
```

이 지시어인 .lcomm 은 버퍼로 사용할 수 있는 500 바이트 크기의 저장 공간을 예약하고, 그 저장 공간을 참조할 수 있는 기호(symbol) my_buffer 를 생성한다. 이 공간은 .bss 섹션에 위치하므로, 실행 파일 크기를 증가시키지 않으며 초기화되지 않은 상태로 프로그램 실행 시 메모리가 할당된다.

이제 파일을 읽기 위해 파일을 열고, 그 결과로 얻은 파일 설명자(file descriptor)가 %ebx 레지스터에 저장되어 있다고 가정해 보자. 이 경우, 다음과 같은 코드로 파일에서 데이터를 읽을 수 있다.

```
movl $my_buffer, %ecx  
movl $500, %edx  
movl $3, %eax  
int $0x80
```

이 코드는 최대 500 바이트를 파일에서 읽어 my_buffer 가 가리키는 버퍼에 저장한다.

[3-2] read 시스템 호출에서 각 레지스터의 의미

이 read 시스템 호출에서 각 레지스터는 다음과 같은 역할을 한다.

- %eax: 시스템 호출 번호(read = 3)
- %ebx: 파일 설명자
- %ecx: 데이터를 저장할 버퍼의 시작 주소
- %edx: 읽을 최대 바이트 수(여기서는 500)

[3-3] 왜 my_buffer 앞에 \$를 붙이는가

이 예제에서 중요한 점은 my_buffer 앞에 달러 기호 \$를 붙였다는 것이다.

```
movl $my_buffer, %ecx
```

달러 기호 \$는 즉시 주소 지정 모드(immediate addressing mode)를 의미한다.

- \$my_buffer -> my_buffer 가 가리키는 주소 값 자체
- my_buffer -> my_buffer 라는 메모리 위치에 저장된 값

우리가 read 시스템 호출에 전달해야 하는 것은 버퍼에 저장된 값이 아니라, 버퍼의 시작 주소이다. 따라서 달러 기호를 사용하여 my_buffer 가 나타내는 숫자(주소)를 %ecx 에 로드해야 한다.

[4] 표준 및 특수 파일

프로그램은 기본적으로 아무 파일도 열리지 않은 상태에서 시작한다고 생각하기 쉽지만 그렇지 않다. 리눅스 프로그램은 실행이 시작될 때 최소한 세 개의 파일 설명자(file descriptor)를 이미 열어 둔 상태로 시작한다. 이 세 개는 표준 입출력을 위해 운영체제가 제공하는 것이다.

[4-1] 표준 입력(STDIN)

STDIN 은 표준 입력(Standard Input)이다.

이 파일은 읽기 전용이며, 일반적으로 키보드 입력을 나타낸다.

STDIN 은 항상 파일 디스크립터 0 번을 사용한다.

즉, 프로그램이 파일 디스크립터 0 에서 read 를 수행하면, 기본적으로 키보드로부터 입력을 읽게 된다.

[4-2] 표준 출력(STDOUT)

STDOUT 은 표준 출력(Standard Output)이다.

이 파일은 쓰기 전용이며, 일반적으로 화면 출력(터미널)을 나타낸다.

STDOPUT 은 항상 파일 디스크립터 1 번을 사용한다.

프로그램이 파일 디스크립터 1 에 write 를 수행하면, 그 결과는 화면에 출력된다.

[4-3] 표준 오류(STDERR)

STDERR 는 표준 오류(Standard Error)이다.

이 파일 역시 쓰기 전용이며, 일반적으로 화면에 출력된다.

STDERR 는 항상 파일 디스크립터 2 번을 사용한다.

일반적인 처리 결과는 STDOUT 으로 출력되지만, 오류 메시지나 경고 메시지는 STDERR 로 출력된다. 이렇게 구분함으로써, 출력 결과와 오류 메시지를 서로 다른 위치로 분리할 수 있다.

[4-4] 표준 파일의 리디렉션

이 세가지 "파일"은 실제로 키보드나 화면일 필요는 없다. UNIX 및 리눅스 환경에서는 이 표준 파일들이 실제 파일로부터 입력을 받거나, 또는 실제 파일로 출력되도록 리디렉션 될 수 있다.

이러한 기능은 UNIX 명령줄 환경에서 제공되며, 프로그램은 이러한 간접성을 전혀 인식할 필요가 없다. 프로그램은 평소처럼 표준 파일 디스크립터(0, 1, 2)를 사용하기만 하면 된다.

[4-4] UNIX 에서의 파일 개념

UNIX 기반 운영 체제에서는 모든 입출력 대상이 파일로 취급된다는 점이 매우 중요하다. 즉, 우리가 다루는 많은 "파일"은 실제 디스크에 존재하는 파일이 아닐 수 있다.

예를 들어 다음과 같은 것들도 모두 파일처럼 취급한다.

- 네트워크 연결
- 직렬 포트
- 오디오 장치
- 프로세스 간 통신을 위한 파이프

이러한 파일 중 일부는 일반 파일과는 다른 방식으로 생성하거나 열어야 할 수도 있지만, 일단 열리고 나면 모두 동일하게 read 와 write 시스템 호출을 사용하여 읽고 쓸 수 있다.

[5] 프로그램에서 파일 사용하기

앞서 설명한 파일 처리 개념을 실제로 적용하기 위해 간단한 예제 프로그램을 작성한다. 이 프로그램의 기능은 다음과 같다. 두 개의 파일을 입력으로 받아 하나의 파일에서 데이터를 읽고, 읽은 데이터 중 모든 소문자를 대문자로 변환한 후, 그 결과를 다른 파일에 기록한다.

본 격적으로 코드를 작성하기 전에, 이 작업을 완료하기 위해 어떤 단계들이 필요한지 먼저 생각하자.

[5-1] 프로그램에 필요한 기능들

먼저, 메모리 블록에 저장된 문자를 대문자로 변환하는 기능이 필요하다.

이 함수는 변환할 데이터가 저장된 메모리 블록의 시작 주소와, 변환할 데이터의 크기를 매개변수로 받아야 한다.

다음으로, 파일로부터 데이터를 버퍼에 반복적으로 읽어들이는 코드가 필요하다. 이 코드는 버퍼에 데이터를 읽은 후, 앞에서 정의한 대문자 변환 함수를 호출하고, 변환이 끝난 데이터를 다시 다른 파일에 써야 한다.

마지막으로, 프로그램이 시작될 때 필요한 파일들을 열고, 모든 작업이 끝난 후에는 파일을 닫는 처리가 필요하다.

[5-2] 문제를 설계하는 순서

여기서 주목할 점은, 작업의 실제 수행 순서와 반대되는 순서로 문제를 정의했다는 것이다. 즉, 파일을 여는 것부터 생각한 것이 아니라, 가장 핵심적인 작업인 “문자 블록을 대문자로 변환하는 것”부터 정의했다.

이러한 접근 방식은 복잡한 프로그램을 작성할 때 매우 유용한 요령이다. 먼저 프로그램이 수행해야 할 본질적인 작업을 결정하고, 그 다음에 그 작업을 가능하게 하기 위해 어떤 준비와 처리가 필요한지를 차례로 생각하는 것이다.

이 경우에는 문자 블록을 변환하는 작업이 핵심이며, 이를 위해 파일을 열고, 데이터를 반복적으로 읽고, 다시 디스크에 기록해야 한다는 점을 도출할 수 있다.

[5-3] 문제 분해의 중요성

프로그램의 핵심 중 하나는 문제를 점점 더 작은 조각으로 나누는 것이다. 문제를 충분히 작게 나누면, 각 조각은 비교적 쉽게 해결할 수 있다. 그런 다음, 이 작은 조각들을 다시 조합하여 전체가 정상적으로 동작하는 프로그램을 만들 수 있다.

이러한 방식은 어셈블리 언어처럼 저수준 언어를 다룰 때 특히 중요하다.

[5-4] .equ 지시어 소개

아마도 지금쯤 시스템 호출 번호나 인터럽트 번호처럼 많은 숫자 상수들을 모두 외우는 것은 불가능하다고 느꼈을 것이다. 이를 돕기 위해 이 프로그램에서는 새로운 어셈블리 지시어인 .equ 를 소개한다.

.equ 지시어는 숫자에 이름을 붙일 수 있게 해준다. 예를 들어 다음과 같이 정의할 수 있다.

```
.equ LINUX_SYSCALL, 0x80
```

이후 코드에는 LINUX_SYSCALL 을 사용하면, 어셈블러는 이를 자동으로 0x80 으로 치환한다. 따라서 다음과 같이 쓸 수 있다.

```
int $LINUX_SYSCALL
```

이 방식은 코드의 가독성을 높이고, 숫자를 외울 필요를 줄여 주어 프로그래밍을 훨씬 수월하게 만든다. 어셈블리 코딩은 본질적으로 복잡하지만, 이러한 방법을 사용하면 충분히 관리 가능한 수준으로 만들 수 있다.

[5-5] 프로그램 구조에 대한 안내

이제 전체 프로그램 코드가 제시된다. 코드에는 실제로 점프에 사용되는 레이블보다 더 많은 레이블이 존재한다는 점에 주의하자. 이들 중 일부는 단순히 코드의 흐름을 명확하게 하기 위해 존재한다.

프로그램을 한 줄씩 따라가면서, 각 상황에서 어떤 일이 발생하는지 살펴보는 것이 중요하다. 코드 뒤에는 프로그램의 각 부분이 어떤 역할을 수행하는지에 대한 자세한 설명이 이어질 것이다.

```

# -----
# 목적:
#       입력 파일을 읽어 모든 소문자를 대문자로 변환한 뒤
#       출력 파일에 기록하는 프로그램
#
# 사용법:
#       program input_file output_file
#
# -----

# -----
#DATA SECTION
# -----
.section .data

# 시스템 콜 번호
.equ SYS_EXIT, 1
.equ SYS_READ, 3
.equ SYS_WRITE, 4
.equ SYS_OPEN, 5
.equ SYS_CLOSE, 6

# 파일 열기 옵션
.equ O_RDONLY, 0
.equ O_CREAT_WRONLY_TRUNC, 03101

# 표준 파일 디스크립터
.equ STDIN, 0
.equ STDOUT, 1
.equ STDERR, 2

# 기타 상수
.equ LINUX_SYSCALL, 0x80
.equ END_OF_FILE, 0

```

```

# -----
# BSS SECTION
# -----
.section .bss

# 버퍼 크기
.equ BUFFER_SIZE, 500
.lcomm BUFFER_DATA, BUFFER_SIZE


# -----
# TEXT SECTION
# -----
.section .text

# 스택 프레임 오프셋
.equ ST_SIZE_RESERVE, 8
.equ ST_FD_IN, -4
.equ ST_FD_OUT, -8


# 프로그램 인자
.equ ST_ARGC, 0
.equ ST_ARGV_0, 4    # 프로그램 이름
.equ ST_ARGV_1, 8    # 입력 파일
.equ ST_ARGV_2, 12   # 출력 파일


.global _start

# -----
# 프로그램 시작
# -----
_start:
    # 스택 기준점 설정
    movl %esp, %ebp

    # 파일 디스크립터 저장 공간 확보
    subl $ST_SIZE_RESERVE, %esp

```

```

# -----
# 입력 파일 열기
# -----
    movl $SYS_OPEN, %eax
    movl ST_ARGV_1(%ebp), %ebx
    movl $O_RDONLY, %ecx
    movl $0666, %edx
    int $LINUX_SYSCALL

    # 입력 파일 디스크립터 저장
    movl %eax, ST_FD_IN(%ebp)

# -----
# 출력 파일 열기
# -----
    movl $SYS_OPEN, %eax
    movl ST_ARGV_2(%ebp), %ebx
    movl $O_CREAT_WRONLY_TRUNC, %ecx
    movl $0666, %edx
    int $LINUX_SYSCALL

    # 출력 파일 디스크립터 저장
    movl %eax, ST_FD_OUT(%ebp)

# -----
# 메인 읽기/쓰기 루프
# -----
read_loop:
    # 파일 읽기
    movl $SYS_READ, %eax
    movl ST_FD_IN(%ebp), %ebx
    movl $BUFFER_DATA, %ecx
    movl $BUFFER_SIZE, %edx
    int $LINUX_SYSCALL

    # EOF 또는 오류 검사
    cmpl $END_OF_FILE, %eax
    jle end_program

    # 대문자 변환 함수 호출
    pushl $BUFFER_DATA    # 버퍼 주소

```

```

    pushl %eax          # 읽은 바이트 수
    call convert_to_upper
    addl $8, %esp       # 인자 정리

    # 출력 파일에 쓰기
    movl %eax, %edx
    movl $SYS_WRITE, %eax
    movl ST_FD_OUT(%ebp), %ebx
    movl $BUFFER_DATA, %ecx
    int $LINUX_SYSCALL

    jmp read_loop

# -----
# 프로그램 종료 처리
# -----
end_program:
    # 출력 파일 닫기
    movl $SYS_CLOSE, %eax
    movl ST_FD_OUT(%ebp), %ebx
    int $LINUX_SYSCALL

    # 입력 파일 닫기
    movl $SYS_CLOSE, %eax
    movl ST_FD_IN(%ebp), %ebx
    int $LINUX_SYSCALL

    # 종료
    movl $SYS_EXIT, %eax
    movl $0, %ebx
    int $LINUX_SYSCALL

# -----
# 함수: convert_to_upper
#
# 입력:
#   [ebp+8] : 버퍼 길이
#   [ebp+12] : 버퍼 주소
#
# 동작:
#   버퍼 내 소문자(a-z)를 대문자로 변환

```

```

# -----

# 문자 상수
.equ LOWERCASE_A, 'a'
.equ LOWERCASE_Z, 'z'
.equ UPPER_CONVERSION, 'A' - 'a'

# 스택 오프셋
.equ ST_BUFFER_LEN, 8
.equ ST_BUFFER, 12

convert_to_upper:
    pushl %ebp
    movl %esp, %ebp

    # 매개변수 로드
    movl ST_BUFFER(%ebp), %eax # 버퍼 주소
    movl ST_BUFFER_LEN(%ebp), %ebx # 길이
    movl $0, %edi             # 인덱스

    # 길이가 0 이면 종료
    cmpl $0, %ebx
    je convert_end

convert_loop:
    movb (%eax,%edi,1), %cl

    cmpb $LOWERCASE_A, %cl
    jl skip_char
    cmpb $LOWERCASE_Z, %cl
    jg skip_char

    addb $UPPER_CONVERSION, %cl
    movb %cl, (%eax,%edi,1)

skip_char:
    incl %edi
    cmpl %edi, %ebx
    jne convert_loop

convert_end:

```

```
movl %ebp, %esp  
popl %ebp  
ret
```

[5-6] 컴파일 및 실행 방법

```
as toupper.s -o toupper.o  
ld toupper.o -o toupper  
./toupper toupper.s toupper.uppercase
```

위 명령을 실행하면 toupper.s 파일의 모든 소문자가 대문자로 변환되어 toupper.uppercase 파일에 저장된다.

[6] 전체 동작 흐름

1. 커널이 argc/argv 를 스택에 올린 상태로 프로그램 시작
2. _start 에서 스택 프레임 생성
3. 입력 파일 open
4. 출력 파일 open
5. read -> 대문자 변환 -> write 반복
6. EOF 도달 시 파일 close 후 exit

[6-1] DATA SECTION(.data)

시스템 콜 번호

- SYS_EXIT = 1
- SYS_READ = 3
- SYS_WRITE = 4
- SYS_OPEN = 5
- SYS_CLOSE = 6

리눅스 32 비트에서는 int 0x80 인터럽트를 통해 시스템 콜을 수행한다.

파일 열기 옵션

- O_RDONLY = 0
- O_CREAT_WRONLY_TRUNC = 03101(8 진수)
 - O_WRONLY | O_CREAT | O_TRUNC

표준 파일 디스크립터

- STDIN = 0
- STDOUT = 1
- STDERR = 2

[6-2] BSS SECTION(.bss)

프로그램을 파일을 한 글자씩 처리하지 않고, 고정 크기 버퍼 단위로 읽는다.

```
.equ BUFFER_SIZE, 500
.lcomm BUFFER_DATA, BUFFER_SIZE
```

- BUFFER_DATA: 파일 내용을 임시로 저장할 메모리 공간
- BUFFER_SIZE: 버퍼 크기(500 바이트)

[6-3] 프로그램 시작 시 스택 구조

리눅스 커널은 프로그램 실행 전에 스택에 다음 정보를 적재한다.

높은 주소

argv[2] ← 출력 파일 이름, "output.txt"

argv[1] ← 입력 파일 이름, "input.txt"

argv[0] ← 프로그램 이름, "program"

argc

낮은 주소(ESP)

[6-4] _start 에서의 스택 프레임 구성

기준점 설정

```
movl %esp, %ebp
```

EBP 를 기준 레지스터로 사용한다.

로컬 변수 공간 확보

```
subl $8, %esp
```

입력/출력 파일 디스크립터 저장 공간을 확보한다.

스택 구조

높은 주소

$\text{argv}[2] \leftarrow \text{ebp} + 12$

$\text{argv}[1] \leftarrow \text{ebp} + 8$

$\text{argv}[0] \leftarrow \text{ebp} + 4$

$\text{argc} \leftarrow \text{ebp} + 0$

$\text{FD_IN} \leftarrow \text{ebp} - 4$

$\text{FD_OUT} \leftarrow \text{ebp} - 8$

낮은 주소(ESP)

[6-5] 입력 파일 열기(open)

```
movl $SYS_OPEN, %eax
movl ST_ARGV_1(%ebp), %ebx
movl $O_RDONLY, %ecx
movl $0666, %edx
int $0x80
```

- 반환값(eax): 입력 파일 디스크립터
- 해당 값을 ST_FD_IN 에 저장

[6-6] 출력 파일 열기(open)

```
movl $SYS_OPEN, %eax
movl ST_ARGV_2(%ebp), %ebx
movl $O_CREAT_WRONLY_TRUNC, %ecx
movl $0666, %edx
int $0x80
```

- 파일이 없으면 생성
- 있으면 기존 내용 삭제 후 새로 작성

[6-7] 메인 read/write 루프

파일 읽기

```
movl $SYS_READ, %eax
movl ST_FD_IN(%ebp), %ebx
movl $BUFFER_DATA, %ecx
movl $BUFFER_SIZE, %edx
int $0x80
```

- eax = 실제 읽은 바이트 수
- eax ≤ 0 이면 EOF 또는 오류

대문자 변환 함수 호출 시 스택

```
pushl $BUFFER_DATA
pushl %eax
call convert_to_upper
```

높은 주소

argv[2]

argv[1]

argv[0]

argc

FD_IN

FD_OUT

buffer address

buffer length

return address $\leftarrow$ ESP

낮은 주소

[6-10] convert_to_upper 함수 스택 구조

함수 프로로그

```
pushl %ebp  
movl %esp, %ebp
```

함수 내부 스택 구조

높은 주소

buffer address $\leftarrow$ ebp + 12

buffer length $\leftarrow$ ebp + 8

return address $\leftarrow$ ebp + 4

old ebp $\leftarrow$ ebp

낮은 주소

변환 로직

- 각 바이트를 순회
- 'a' $\leq$ 문자 $\leq$ 'z' 인 경우
- ASCII 차이('A'-'a')를 더해 대문자로 변환

[6-11] 함수 종료 및 스택 복원

```
movl %ebp, %esp  
popl %ebp  
ret
```

호출자는 다음 명령으로 인자 제거

```
addl $8, %esp
```

[6-12] 출력 파일 쓰기

```
movl %eax, %edx  
movl $SYS_WRITE, %eax  
movl ST_FD_OUT(%ebp), %ebx  
movl $BUFFER_DATA, %ecx  
int $0x80
```

[6-13] 프로그램 종료 처리

1. 출력 파일 close
2. 입력 파일 close
3. SYS_EXIT 호출

[6-14] 전체 스택 흐름 요약

초기 argc/argv
↓
_start 스택 프레임
↓
파일 디스크립터 저장
↓
(read loop)
↓
convert_to_upper 호출
↓
함수 프레임 생성/해제
↓
exit

문제

[1] 개념 이해하기

- {1} 파일 설명자의 생애 주기를 설명하기
- {2} 표준 파일 설명자는 무엇이며, 무엇에 사용되는지
- {3} 버퍼가 무엇인지
- {4} .data 섹션과 .bss 섹션의 차이점은 무엇인지
- {5} 파일을 읽고 쓰는 것과 관련된 시스템 호출은 무엇인지

[2] 개념 사용하기

- {1} toupper 프로그램을 수정하여 STDIN 에 읽고 STDOUT 에 쓰도록 하기. (명령줄의 파일을 사용하지 말고)
- {2} 버퍼의 크기를 변경하기
- {3} 프로그램을 다시 작성하여 파일 설명자를 저장하기 위해 스택 대신 .bss 섹션의 저장소를 사용하기
- {4} heynew.txt 라는 파일을 생성하고 그 안에 "Hey diddle diddle!"라는 단어를 작성하는 프로그램을 작성하기

[3] 심화

- {1} 버퍼의 크기가 어떤 차이를 만드는가
- {2} 이 시스템 호출 각각에서 반환될 수 있는 오류 결과는 무엇인가
- {3} ARGV(명령줄 인자의 개수)에 따라 파일 인자를 사용하거나 표준 입력/출력을 사용하도록 프로그램을 구현하기
- {4} 프로그램을 수정하여 각 시스템 호출의 결과를 확인하고, 오류가 발생할 경우 STDOUT 에 오류 메시지를 출력하도록 하기

6장 간단한 레코드 읽기 및 쓰기

5장에서 언급했듯이, 많은 애플리케이션은 지속적인(persistent) 데이터를 다룬다. 이는 데이터가 디스크의 파일에 저장되어 프로그램보다 더 오래 살아남는다는 의미이다. 프로그램을 종료한 뒤 다시 실행하더라도, 이전에 작업하던 상태로 돌아올 수 있다.

이러한 지속적인 데이터에는 두 가지 기본적인 유형이 있다. 바로 구조화된 데이터(structured data)와 비구조화된 데이터(unstructured data)이다.

비구조화된 데이터는 우리가 toupper 프로그램에서 다루었던 것과 같은 형태이다. 이 프로그램은 사람이 입력한 텍스트 파일을 처리했는데, 이 파일의 내용은 프로그램이 의미를 해석할 수 없는 임의의 텍스트였다. 즉, 프로그램은 사용자가 자유 형식의 텍스트로 무엇을 말하려는지 이해할 수 없다.

반면에 구조화된 데이터는 컴퓨터가 처리하는데 매우 뛰어난 형태의 데이터이다. 구조화된 데이터는 필드(field)와 레코드(record)로 나뉘어진 데이터이며, 대부분의 경우 필드와 레코드는 고정된 길이를 가진다. 데이터가 고정 길이 레코드와 고정 형식의 필드로 나뉘어 있기 때문에, 컴퓨터는 해당 데이터를 정확하게 해석할 수 있다.

구조화된 데이터가 가변 길이 필드를 포함할 수도 있지만, 그 정도의 복잡성이 필요하다면 일반적으로는 데이터베이스를 사용하는 것이 더 적합하다.

데이터베이스는 지속적인 구조화 데이터를 관리해 주는 프로그램으로, 파일 입출력이나 검색, 기본적인 처리 작업을 직접 작성할 필요가 없게 해 준다. 다만 약간의 오버헤드와 복잡성을 추가하지만, 복잡한 데이터 처리 작업에서는 매우 유용하다.

[1-1] 고정 길이 레코드

이 장에서 간단한 고정 길이 레코드를 읽고 쓰는 방법을 다룬다.

예를 들어, 우리가 아는 사람들에 대한 기본 정보를 저장하고 싶다고 가정한다. 이 경우 다음과 같은 고정 길이 레코드를 상상할 수 있다.

- | |
|---|
| <ul style="list-style-type: none">- Firstname(이름): 40 바이트- Lastname(성): 40 바이트- Address(주소): 240 바이트- Age(나이): 4 바이트 |
|---|

이 레코드에서 나이를 제외한 모든 항목은 문자 데이터이다. 나이는 단순한 숫자 필드이며, 표준 4 바이트 워드를 사용한다. 나이를 1 바이트로 표현할 수도 있지만, 워드 단위를 사용하는 편이 처리하기 더 쉽다.

[1-2] 공통 정의 파일 분리

프로그래밍을 하다 보면, 프로그램 안에서 반복적으로 사용하거나 여러 프로그램에서 공통으로 사용해야 하는 정의들이 자주 등장한다. 이런 정의들은 필요할 때마다 어셈블리 파일에 포함(include)할 수 있도록 별도의 파일로 분리하는 것이 좋다.

앞으로 작성할 프로그램에서는 위에서 정의한 레코드의 각 부분에 접근해야 한다. 이를 위해서는 레코드의 시작 주소로부터 각 필드까지의 오프셋(offset)을 알아야 하며, 이 오프셋을 이용해 베이스 포인터 주소 지정 방식으로 접근한다.

다음은 위의 레코드 구조에 대한 오프셋을 정의한 상수들이다. 이 내용은 record-def.라는 파일에 저장한다.

```
.equ RECORD_FIRSTNAME, 0
.equ RECORD_LASTNAME, 40
.equ RECORD_ADDRESS, 80
.equ RECORD_AGE, 320
.equ RECORD_SIZE, 324
```

[1-3] 리눅스 공통 상수 정의

또한, 여러 프로그램에서 반복적으로 사용되는 리눅스 관련 상수들이 있다. 이 상수들 역시 별도의 파일로 분리해 두는 것이 좋다. 다음 내용은 linux.s 파일에 저장한다.

```
# 공통 리눅스 정의
# 시스템 호출 번호
.equ SYS_EXIT, 1
.equ SYS_READ, 3
.equ SYS_WRITE, 4
.equ SYS_OPEN, 5
.equ SYS_CLOSE, 6
.equ SYS_BRK, 45

# 시스템 호출 인터럽트 번호
.equ LINUX_SYSCALL, 0x80

# 표준 파일 설명자
.equ STDIN, 0
.equ STDOUT, 1
.equ STDERR, 2

# 공통 상태 코드
.equ END_OF_FILE, 0
```

[2-1] 이 장에서 작성할 프로그램 개요

이 장에서 record-def.s 에 정의된 구조를 사용하여 세 개의 프로그램을 작성한다.

1. 여러 개의 레코드를 포함하는 파일을 생성하는 프로그램
2. 파일에 저장된 레코드를 화면에 출력하는 프로그램
3. 모든 레코드의 나이를 1 살씩 증가시키는 프로그램

이 프로그램에서는 공통 상수뿐만 아니라, 여러 프로그램에서 공통으로 사용되는 두 개의 함수도 사용한다. 하나는 레코드를 읽는 함수, 다른 하나는 레코드를 쓰는 함수이다.

이 함수들이 동작하기 위해 필요한 인자는 다음 두 가지이다.

- 레코드를 읽거나 쓸 버퍼의 위치
- 읽거나 쓸 대상인 파일 디스크립터

[2-2] 레코드 읽기 함수(read-record.s)

다음은 레코드를 읽는 함수이다.

```
.include "record-def.s"
.include "linux.s"
```

이 함수는 주어진 파일 디스크립터로부터 레코드 크기만큼의 데이터를 읽어, 지정된 버퍼에 저장하고 그 결과를 반환한다. 함수의 반환값은 읽은 바이트 수이며, 이는 호출한 프로그램에서 사용된다.

```

.include "record-def.s"
.include "linux.s"
# 목적: 이 함수는 파일 디스크립터로부터 하나의 레코드를 읽는다
#
# 입력: 파일 디스크립터 번호와 데이터를 저장할 버퍼 주소
#
# 출력: 버퍼에 데이터를 채우고, 결과를 상태 코드(%eax)로 반환한다
#
# [스택 로컬 변수]
.equ ST_READ_BUFFER, 8      # 첫 번째 인자: 버퍼 주소(ebp + 8)
.equ ST_FILEDES, 12         # 두 번째 인자: 파일 디스크립터(ebp + 12)

.section .text
.globl read_record
.type read_record, @function

read_record:
    # [함수 시작 - 프로로그]
    pushl %ebp               # 이전 함수의 기준점(EBP)을 스택에 보관
    movl %esp, %ebp         # 현재 스택 포인터(ESP)를 새로운 기준점(EBP)으로 설정

    # [레지스터 보존]
    pushl %ebx               # %ebx 는 호출된 함수에서 보호해야 할 레지스터이므로 보관
                                # 또한 리눅스 시스템 콜에서 파일 디스크립터를 넣는 용도로 사용

    # [시스템 콜 준비: SYS_READ]
    # 리눅스 read 시스템 콜:
    # %eax=3, %ebx=파일디스크립터, %ecx=버퍼주소, %edx=읽을크기

    movl ST_FILEDES(%ebp), %ebx
    # 스택 상위(ebp+12)에서 파일 디스크립터를 가져와 %ebx 에 복사
    movl ST_READ_BUFFER(%ebp), %ecx
    # 스택 상위(ebp+8)에서 버퍼 주소를 가져와 %ecx 에 복사
    movl $RECORD_SIZE, %edx
    # 읽어올 크기를 %edx 에 설정(record-def.s 에 정의됨)
    movl $SYS_READ, %eax
    # 시스템 콜 번호 3(읽기)을 %eax 에 설정

    # [커널 호출]
    int $LINUX_SYSCALL # 리눅스 커널에 명령 실행 요청
                        # 결과값(읽은 바이트 수)은 자동으로 %eax 에 저장

```

```

# [함수 종료 - 에필로그]
# 참고: %eax 의 결과값은 건드리지않고 그대로 호출한 프로그램으로 넘겨줌

popl %ebx          # 보관했던 %ebx 값을 복원
movl %ebp, %esp     # 스택 포인터를 기준점으로 되돌림(지역 변수가 있다면 정리됨)
popl %ebp           # 이전 함수의 기준점(EBP)을 복원
ret                 # 스택의 리턴 주소를 확인하여 호출한 곳으로 돌아감

```

popl %ebx: 빌린 레지스터 되돌려주기

이 명령은 %ebx 레지스터에 들어 있던 파일 디스크립터 값을 "없애는" 동작이 아니다. 함수 시작시 pushl %ebx 로 스택에 저장해 두었던 호출자(caller)가 원래 사용하던 %ebx 값을 다시 복원하는 과정이다. read_record 함수는 리눅스 시스템 콜을 위해 %ebx 를 파일 디스크립터 용도로 사용했지만, %ebx 는 호출 규약상 callee-saved register 이므로 함수가 끝날 때 반드시 원래 값으로 되돌려주어야 한다. 이를 지키지 않으면, 함수를 호출한 쪽에서 %ebx 에 저장해 두었던 중요한 값이 망가져 프로그램의 동작이 꼬일 수 있다.

movl %ebp, %esp: 스택 작업 공간 정리

이 명령은 스택 포인터 %esp 를 현재 함수의 기준점인 %ebp 위치로 되돌리는 동작이다. 함수 내부에서 여러 번 push 를 하거나 지역 변수를 위해 스택을 확장했을 수 있는데, 이때 하나하나 pop 으로 정리하지 않고 기준점 %ebp 로 %esp 를 한 번에 끌어올려서 그 아래에 쌓였던 임시 데이터들을 통째로 무시한다. 즉, 현재 함수가 사용하던 스택 작업 공간을 일괄 정리하는 역할을 한다.

popl %ebp: 이전 함수의 기준점 복원

이 명령은 스택에 저장되어 있던 이전 함수의 기준점(EBP)을 꺼내 다시 %ebp 레지스터에 넣는 과정이다. 함수가 시작될 때 pushl %ebp 로 이전 기준점을 저장해 두었기 때문에, 이제 함수가 끝나면서 그 값을 다시 복구해야 한다. 이 동작을 통해 스택 기준점이 read_record 함수가 아닌, 이 함수를 호출한 상위 함수의 스택 구조로 정확히 돌아가게 된다.

ret: 호출 지점으로 복귀

ret 명령은 스택 맨 위에 남아 있는 리턴주소(return address)를 꺼내 해당 주소로 CPU 실행 흐름을 이동시킨다. 이 주소는 함수가 호출될 때 자동으로 스택에 저장된 값으로, ret 이 실행되는 순간 프로그램은 read_record 를 호출했던 코드의 바로 다음 줄로 돌아가 실행을 계속한다. 이로써 함수의 모든 실행과 정리 작업이 완전히 끝난다.

[2-4] 레코드 쓰기 함수(write-record.s)

레코드를 쓰는 함수는 읽기 함수와 거의 동일한 구조를 가지며, 지정된 버퍼의 내용을 파일 디스크립터로 출력한다. 역시 반환값으로 상태 코드를 제공한다.

```
.include "linux.s"
.include "record-def.s"

.equ ST_WRITE_BUFFER, 8    # 첫 번째 인자: 쓸 데이터가 있는 버퍼 주소
.equ ST_FILEDES, 12        # 두 번째 인자: 쓸 파일의 디스크립터

.section .text
.globl write_record
.type write_record, @function

write_record:
    pushl %ebp
    movl %esp, %ebp
    pushl %ebx

    movl $SYS_WRITE, %eax    # 쓰기 시스템 콜 번호
    movl ST_FILEDES(%ebp), %ebx    # 파일 디스크립터 로드
    movl ST_WRITE_BUFFER(%ebp), %ecx    # 버퍼 주소 로드
    movl $RECORD_SIZE, %edx    # 쓸 크기(324 바이트)
    int $LINUX_SYSCALL        # 커널 호출

    popl %ebx
    movl %ebp, %esp
    popl %ebp
    ret
```

[3-1] 레코드 파일 생성 프로그램

이 프로그램은 하드코딩된 레코드 데이터를 디스크에 기록한다. 전체 동작 과정은 다음과 같다.

1. 파일을 연다
2. 세 개의 레코드를 기록한다.
3. 파일을 닫는다.

각 문자열 필드는 .rept 지시어를 사용하여 정의된 길이가 될 때까지 널 문자로 채워진다. 이렇게 하면 모든 레코드가 정확히 동일한 크기를 가지게 된다.

[3-2] .include 와 .rept 지시어

.include 지시어는 지정된 파일의 내용을 해당 위치에 그대로 삽입하는 역할을 한다. 함수는 링커가 결합해주기 때문에 .include 가 필요 없지만, 상수 정의는 반드시 .include 를 통해 포함해야 한다.

.rept 지시어는 .rept 와 .endr 사이의 내용을 지정된 횟수만큼 반복한다. 이 장에서는 주로 문자열 필드를 널 문자로 패딩하여 고정 길이를 맞추는 데 사용된다.

[3-3] 메인 프로그램 1: 데이터 파일 생성(write-records.s)

```
include "linux.s"
.include "record-def.s"

.section .data
# 저장할 레코드 데이터들을 정의
# .rept 지시어를 사용하여 남는 공간을 0으로 채워 고정 길이를 맞춤

record1:
    .ascii "FredrickW0"
    .rept 31          # 40 바이트를 맞추기 위한 패딩(글자 8+NULL1 = 9, 40-9=31)
    .byte 0
    .endr

    .ascii "BartlettW0"
    .rept 31
    .byte 0
    .endr

    .ascii "4242 S Prairie Tulsa, OK 55555W0"
    .rept 209        # 240 바이트 맞추기 위한 패딩
```

```

        .byte 0
        .endr

        .long 45          # 나이 (4 바이트)

record2:
        .ascii "MarilynW0"
        .rept 32
        .byte 0
        .endr

        .ascii "TaylorW0"
        .rept 33
        .byte 0
        .endr

        .ascii "2224 S Johannan St Chicago, IL 12345W0"
        .rept 203
        .byte 0
        .endr

        .long 29

record3:
        .ascii "DerrickW0"
        .rept 32
        .byte 0
        .endr

        .ascii "McIntireW0"
        .rept 31
        .byte 0
        .endr

        .ascii "500 W Oakland San Diego, CA 54321W0"
        .rept 206
        .byte 0
        .endr

        .long 36

```



```

file_name:
    .ascii "test.dat\0"        # 생성할 파일 이름

.equ ST_FILE_DESCRIPTOR, -4    # 지역 변수: 파일 디스크립터 저장용

.globl _start
_start:
    movl %esp, %ebp
    subl $4, %esp              # 파일 디스크립터를 위한 공간 확보

    # 파일 열기(생성 또는 쓰기 전용)
    movl $SYS_OPEN, %eax
    movl $file_name, %ebx
    movl $0101, %ecx           # O_CREATE | O_WRONLY 설정
    movl $0666, %edx           # 파일 권한
    int $LINUX_SYSCALL

    movl %eax, ST_FILE_DESCRIPTOR(%ebp) # 생성된 FD 저장

    # 레코드 1 쓰기
    pushl ST_FILE_DESCRIPTOR(%ebp)
    pushl %record1
    call write_record
    addl $8, %esp               #스택 정리

    # 레코드 2 쓰기
    pushl ST_FILE_DESCRIPTOR(%ebp)
    pushl %record2
    call write_record
    addl $8, %esp               #스택 정리

    # 레코드 3 쓰기
    pushl ST_FILE_DESCRIPTOR(%ebp)
    pushl %record3
    call write_record
    addl $8, %esp               #스택 정리

    # 레코드 4 쓰기
    pushl ST_FILE_DESCRIPTOR(%ebp)
    pushl %record14

```

```

call write_record
addl $8, %esp           #스택 정리

# 파일 닫기 및 종료
movl $SYS_CLOSE, %eax
movl ST_FILE_DESCRIPTOR(%ebp), %ebx
int $LINUYX_SYSCALL

movl $SYS_EXIT, %eax
movl $0, %ebx
int $LINUX_SYSCALL

```

[3-4] 메인 프로그램 2: 데이터 읽기 및 표시(read-record.s)

이 함수는 문자열의 시작 주소를 받아, 널 문자가 나올 때까지 한 바이트씩 검사하며 문자 개수를 센 후 그 결과를 반환한다.

```

.type count_chars, @function
.globl count_chars
.equ ST_STRING_START_ADDRESS, 8

count_chars:
    pushl %ebp
    movl %esp, %ebp
    movl $0, %ecx        # 카운터 초기화
    movl ST_STRING_START_ADDRESS(%ebp), %edx

count_loop_begin:
    movb (%edx), %al      # 한 바이트 읽기
    cmpb $0, %al         # NULL 문자인가?
    je count_loop_end    # 맞으면 종료
    incl %ecx            # 카운트 + 1
    incl %edx            # 다음 주소
    jmp count_loop_begin

count_loop_end:
    movl %ecx, %eax      # 결과값을 %eax 에 담아 반환
    popl %ebp
    ret

```

[3-5] 메인 로직 : read-records.s

레코드를 읽는 프로그램은 파일을 열고, 레코드를 하나씩 읽어 각 레코드의 이름(first name)을 화면에 출력한다.

이때 이름의 길이는 사람마다 다르기 때문에, 문자열 끝을 나타내는 널 문자가 나올 때까지 문자를 세는 함수가 필요하다. 이는 C 언어의 strlen 함수와 동일한 동작을 한다.

```
.include "linux.s"
.include "record-def.s"

.section .data
file_name:
    .ascii "test.dat\0"

.section .bss
.lcomm record_buffer, RECORD_SIZE    # 레코드를 읽어올 빈 버퍼 공간

.section .text
.globl _start

_start:
    # -----
    # 지역 변수 오프셋 정의
    # -----
    .equ ST_INPUT_DESCRIPTOR, -4      # 입력 파일 디스크립터
    .equ ST_OUTPUT_DESCRIPTOR, -8     # 출력 파일 디스크립터

    # -----
    # 스택 프레임 수동 생성
    # -----
    movl %esp, %ebp                  # 현재 스택 위치를 기준으로 설정
    subl $8, %esp                    # 지역 변수 2개(8 바이트) 공간 확보

    # -----
    # 파일 열기 (읽기 전용)
    # -----
    movl $SYS_OPEN, %eax
    movl $file_name, %ebx
    movl $0, %ecx                     # Read-Only
    movl $0666, %edx
    int $LINUX_SYSCALL
```

```

# open()의 반환값(fd)을 지역 변수에 저장
movl %eax, ST_INPUT_DESCRIPTOR(%ebp)
movl $STDOUT, ST_OUTPUT_DESCRIPTOR(%ebp)

record_read_loop:
# -----
# read_record(fd, record_buffer)
# -----
pushl ST_INPUT_DESCRIPTOR(%ebp) # 두 번째 인자: fd
pushl $record_buffer           # 첫 번째 인자: 버퍼 주소
call  read_record
addl  $8, %esp                  # push 한 인자 2 개 정리

# -----
# EOF 검사
# -----
cmpl $RECORD_SIZE, %eax
jne  finished_reading

# -----
# 이름 필드 길이 계산
# -----
pushl $RECORD_FIRSTNAME + record_buffer
call  count_chars
addl  $4, %esp

# -----
# 이름 출력
# -----
movl %eax, %edx                # 글자 수
movl ST_OUTPUT_DESCRIPTOR(%ebp), %ebx
movl $SYS_WRITE, %eax
movl $RECORD_FIRSTNAME + record_buffer, %ecx
int  $LINUX_SYSCALL

# -----
# 줄바꿈 출력
# -----
pushl ST_OUTPUT_DESCRIPTOR(%ebp)
call  write_newline

```

```

    addl $4, %esp

    jmp record_read_loop

finished_reading:
    # -----
    # 프로그램 종료
    # -----
    movl $SYS_EXIT, %eax
    movl $0, %ebx
    int $LINUX_SYSCALL

```

[3-6] 메인 프로그램 3: 데이터 수정(add-year.s)

다음 작업을 수행한다.

1. 입력 파일과 출력 파일을 연다.
2. 입력 파일에서 레코드를 하나씩 읽는다.
3. 각 레코드의 나이를 1 증가시킨다.
4. 수정된 레코드를 출력 파일에 기록한다.

이 프로그램 역시 레코드를 버퍼에 읽어 들인 후, 해당 필드를 수정하고 다시 기록하는 방식으로 동작한다.

```

#include "linux.s"
#include "record-def.s"

.section .data
input_file_name:
    .ascii "test.dat\0"

output_file_name:
    .ascii "testout.dat\0"

.section .bss
.lcomm record_buffer, RECORD_SIZE

.section .text
.globl _start

```

```

_start:
.equ ST_INPUT_DESCRIPTOR, -4
.equ ST_OUTPUT_DESCRIPTOR, -8

movl %esp, %ebp
subl $8, %esp

# 입력 파일 열기
movl $SYS_OPEN, %eax
movl $input_file_name, %ebx
movl $0, %ecx
int $LINUX_SYSCALL
movl %eax, ST_INPUT_DESCRIPTOR(%ebp)

# 출력 파일 열기
movl $SYS_OPEN, %eax
movl $output_file_name, %ebx
movl $0101, %ecx
movl $0666, %edx
int $LINUX_SYSCALL
movl %eax, ST_OUTPUT_DESCRIPTOR(%ebp)

loop_begin:
# 레코드 읽기
pushl ST_INPUT_DESCRIPTOR(%ebp)
pushl $record_buffer
call read_record
addl $8, %esp

# 파일 끝인지 확인
cmpl $RECORD_SIZE, %eax
jne loop_end

# 나이 증가
incl record_buffer + RECORD_AGE

# 수정된 레코드 쓰기
pushl ST_OUTPUT_DESCRIPTOR(%ebp)
pushl $record_buffer

```

```
call write_record
addl $8, %esp

jmp loop_begin

loop_end:
    movl $SYS_EXIT, %eax
    movl $0, %ebx
    int $LINUX_SYSCALL
```

[4] 마무리

고정 길이 레코드를 사용하는 방식은 매우 단순하다. 레코드를 일정 크기의 블록으로 읽어 버퍼에 저장하고, 필요한 처리를 한 뒤 다시 파일에 기록하면 된다.

다만 이 장에서는 숫자 출력 방법을 아직 다루지 않았기 때문에, 나이 증가 결과를 화면에서 직접 확인할 수 없다. 숫자 출력은 이후 장에서 다루며, 해당 내용을 학습한 후 이 프로그램을 다시 수정해 보는 것도 좋은 연습이 될 것이다.

문제

[1] 개념 이해하기

- {1} 레코드란 무엇인가
- {2} 가변 길이 레코드에 비해 고정 길이 레코드의 장점은 무엇인가
- {3} 여러 개의 어셈블리 소스 파일에 상수를 공통으로 사용하려면 어떻게 해야 하는가?
- {4} 왜 하나의 프로젝트를 여러 개의 소스파일로 나누고 싶을까
- {5} `incl record_buffer + RECORD_AGE` 명령어는 무엇을 하는가
 - 이 명령어는 어떤 주소 지정 방식을 사용하는가
 - 이 경우 `incl` 명령어는 몇 개의 피연산자를 가지는가
 - 이 명령어에서 어떤 부분은 어셈블러가 처리하고, 어떤 부분은 프로그램이 실행될 때 처리하는가?

[2] 개념 사용하기

- {1} 이 장에서 정의한 `person` 구조체에 새로운 데이터 멤버를 하나 추가하고, 그에 맞게 레코드를 읽고 쓰는 함수들과 프로그램을 수정하라. 프로그램을 실행하기 전에 반드시 다시 어셈블하고 링크해야 한다.
- {2} 반복문을 사용하여 동일한 레코드 30 개를 파일에 쓰는 프로그램을 작성하라.
- {3} 파일에서 가장 큰 나이를 찾아 그 값을 프로그램의 종료 상태 코드로 반환하는 프로그램을 작성하라.
- {4} 파일에서 가장 작은 나이를 찾아 그 값을 프로그램의 종료 상태 코드로 반환하는 프로그램을 작성하라.

[3] 심화

- {1} 이 장의 프로그램들을 명령줄 인자를 사용하여 파일 이름을 지정하도록 수정하라
- {2} `lseek` 시스템 콜을 조사하라. 그리고 `add-year` 프로그램을 수정하여, 소스 파일을 읽기와 쓰기 모두 가능한 모드(`$2`, `read/write` 모드)로 열고, 읽은 레코드를 같은 파일에 다시 덮어써서 수정된 내용을 저장하도록 하라.
- {3} 이 프로그램들에서 사용된 시스템 콜들이 반환할 수 있는 다양한 오류 코드들을 조사하라. 그 중 하나를 선택하여, `%eax` 값을 검사하여 오류가 발생했는지 확인하고, 오류가 있다면 `STDERR` 로 오류 메시지를 출력한 뒤 종료하는 코드를 추가하라.
- {4} 키보드로부터 데이터를 입력받아 파일에 레코드 하나를 추가하는 프로그램을 작성하라. 이때 입력 데이터에는 반드시 최소 하나의 널 문자(`W0`)가 포함되도록 해야 하며, 사용자가 입력을 끝냈다는 것을 알릴 수 있는 방법도 필요하다. 아직 문자 → 숫자 변환을 배우지 않았으므로, 나이는 키보드로 입력받을 수 없고 기본 나이 값을 사용해야 한다.
- {5} 최대 5 글자까지 두 문자열을 비교하는 `compare-strings` 함수를 작성하라. 그런 다음, 사용자가 5 글자를 입력하면 이름(`first name`)이 5 글자로 시작하는 모든 레코드를 출력하는 프로그램을 작성하라.

7장 견고한 프로그램 개발

이 장에서는 견고한(robust) 프로그램을 개발하는 방법을 다룬다. 견고한 프로그램이란 오류 상황을 우아하게 처리할 수 있는 프로그램이다. 즉, 사용자가 무엇을 하더라도 절대 크래시가 나지 않는 프로그램을 말한다. 견고한 프로그램을 만드는 것은 프로그래밍 실무에서 필수적이다. 하지만 견고한 프로그램을 작성하는 데에는 규율과 노력이 필요하며, 보통 발생할 수 있는 모든 문제를 찾아내고, 각 문제에 대해 프로그램이 취할 행동 계획을 세우는 과정을 포함한다.

[1] 시간은 어디로 가는가

프로그래머들은 일정 관리를 잘하지 못한다. 거의 모든 프로그래밍 프로젝트에서, 프로그래머들은 프로그램이나 함수 하나를 개발하는 데 처음 예상했던 시간의 두 배, 네 배, 심지어 여덟 배까지 걸리곤 한다. 이 문제에는 여러 가지 이유가 있다.

- 프로그래머들은 회의나 기타 코딩 외 활동에 필요한 시간을 항상 일정에 포함시키지 않는다.
- 프로젝트에서 변경 요청과 승인 등을 주고받는 데 걸리는 피드백 시간을 과소평가하는 경우가 많다.
- 자신들이 만들고 있는 것의 전체 범위를 완전히 이해하지 못하는 경우가 많다.
- 익숙하지 않은 유형의 프로젝트에 대해 일정을 추가해야 하는 경우, 정확한 일정 산정이 어렵다.
- 프로그램을 완전히 견고하게 만드는 데 걸리는 시간을 과소평가하는 경우가 많다.

마지막 항목이 여기서 우리가 관심을 두는 부분이다. 견고한 프로그램을 개발하는 데에는 사람들이 일반적으로 생각하는 것보다 훨씬 더 많은 시간과 노력이 필요하다. 이는 숙련된 프로그래머에게도 해당된다. 프로그래머들은 당장 눈앞의 문제를 해결하는 데에만 집중한 나머지, 발생할 수 있는 부분적인 문제들을 살펴보지 못하는 경우가 많다.

toupper 프로그램을 보면, 사용자가 선택한 파일이 존재하지 않을 경우에 대한 대응 방법이 전혀 없다. 프로그램은 그대로 작업을 시도해 버린다. 오류 메시지도 출력하지 않기 때문에, 사용자는 자신이 파일 이름을 잘못 입력했는지도 알 수 없다. 만약 대상 파일이 네트워크 드라이브에 있고, 네트워크가 일시적으로 실패했다고 가정해 보자. 운영체제는 상태 코드를 %eax 레지스터에 반환하고 있지만, 우리는 이를 확인하지 않고 있다. 따라서 오류가 발생하더라도 사용자는 전혀 인지할 수 없다. 이 프로그램은 분명히 견고하지 않다. 보듯이, 아주 간단한 프로그램에서도 프로그래머가 대응해야 할 수 많은 문제가 발생할 수 있다.

대규모 프로그램에서는 상황이 훨씬 더 복잡해진다. 일반적으로 성공적인 경우보다 오류 조건이 훨씬 더 많다. 따라서 상태 코드를 확인하고, 오류 처리기를 작성하며, 프로그램을 견고하게 만들기 위한 유사한 작업에 대부분의 시간을 쓰게 될 것이라고 항상 예상한다. 프로그램을 개발하는 데 2 주가 걸렸다면, 그것을 견고하게 만드는 데에는 최소한 2 주가 더

걸릴 가능성이 크다. 화면에 나타나는 모든 오류 메시지는 누군가가 직접 코드로 작성한 결과라는 점을 기억하라.

[2] 견고한 프로그램 개발을 위한 몇 가지 팁

[2-1] 사용자 테스트

테스트는 프로그래머가 수행하는 가장 필수적인 작업 중 하나이다. 무언가를 테스트하지 않았다면, 그것은 동작하지 않는다고 가정해야 한다. 그러나 테스트는 단순히 프로그램이 잘 동작하는지를 확인하는 것만이 아니라, 프로그램이 망가지지 않는지를 확인하는 것이다.

예를 들어, 양의 정수만 처리해야 하는 프로그램이 있다면 사용자가 음수를 입력했을 때 어떻게 되는지를 테스트해야 한다. 문자나 0 을 입력했을 때도 마찬가지다. 숫자 앞이나 뒤에 공백을 넣었을 때, 혹은 기타 사소한 가능성들도 모두 테스트해야 한다. 사용자의 데이터를 사용자 입장에서 합리적으로 처리하고, 그 데이터를 프로그램의 나머지 부분에 전달할 때도 일관성 있게 처리해야 한다.

프로그램이 의미 없는 입력을 발견했을 때는 적절한 동작을 수행해야 한다. 프로그램의 성격에 따라 이는 프로그램 종료, 사용자에게 값 재입력 요청, 중앙 오류 로그에 기록, 작업 롤백, 혹은 무시하고 계속 진행하는 것일 수 있다.

자신만 테스트하는 것으로는 충분하지 않다. 다른 프로그래머나 실제 사용자들에게도 테스트를 맡겨야 한다. 사용자에게 문제가 되는 부분이 있다면, 개발자 입장에서는 괜찮아 보이더라도 반드시 수정해야 한다. 사용자가 프로그램을 올바르게 사용하는 방법을 모른다면, 그것 여섯 수정해야 할 버그로 취급해야 한다.

사용자들은 개발자가 발견하지 못한 훨씬 더 많은 버그를 찾아낸다. 그 이유는 사용자는 컴퓨터가 무엇을 기대하는지 모르기 때문이다. 개발자는 컴퓨터가 기대하는 입력을 알고 있으므로, 컴퓨터 입장에서 의미있는 데이터를 입력할 가능성이 훨씬 높다. 반면 사용자는 자신에게 의미 있는 데이터를 입력한다. 비프로그래머에게 테스트를 맡기면 프로그램이 실제 얼마나 견고한지를 훨씬 정확하게 파악할 수 있다.

[2-2] 데이터 테스트

프로그램을 설계할 때 각 함수는 어떤 타입과 범위의 데이터를 받아들이는지, 혹은 받아들이지 않는지를 매우 명확하게 정의해야 한다. 그리고 해당 함수가 올바른 데이터를 받았을 때 명세대로 동작하는 지 테스트해야 한다. 가장 중요한 것은 코너 케이스(edge case)를 테스트하는 것이다. 코너 케이스란 문제를 일으키거나 예상치 못한 동작을 할 가능성이 가장 높은 입력이다.

숫자 데이터를 테스트할 때는 반드시 확인해야 할 코너 케이스는 다음과 같다.

- 숫자 0
- 숫자 1
- 예상 범위 내의 숫자
- 예상 범위를 벗어난 수자
- 예상 범위의 첫 번째 숫자
- 예상 범위의 마지막 숫자
- 예상 범위보다 바로 작은 숫자
- 예상 범위보다 바로 큰 숫자

예를 들어, 5 에서 200 사이의 값을 받아야 하는 프로그램이라면 최소한 0, 1, 4, 5, 153, 200, 2001, 255 를 테스트해야 한다. (153 과 255 는 범위 안과 밖에서 임의로 선택한 값이다.) 데이터 목록도 마찬가지로, 항목이 0 개일 때, 1 개일 때, 매우 많은 개수일 때 모두 테스트해야 한다.

또한 분기점도 테스트해야 한다. 예를 들어 30 세 미만과 30 세 이상을 다르게 처리하는 코드가 있다면, 29 세, 30 세, 31 세에 대해 테스트 해야 한다.

일부 내부 함수는 이전 단계에서 이미 오류 검사를 했기 때문에 올바른 데이터를 받는다고 가정할 수도 있다. 그러나 개발 중에는 다른 코드에 오류가 있을 수 있으므로 여전히 검증이 필요하다. 이를 위해 대부분의 언어는 데이터의 정합성을 쉽게 검사할 수 있는 기능을 제공한다. C 언어에서는 assert 매크로가 있다. 코드에 `assert(a > b);` 를 넣으면 조건이 참이 아닐 때 해당 코드에 도달하면 오류를 발생시킨다. 또한 코드가 안정화된 이후에는 불필요한 검사이므로, 컴파일 시 assert 를 비활성화할 수도 있다. 이를 통해 공개용 코드의 성능 저하 없이 데이터의 올바름을 검증할 수 있다.

[2-3] 모듈 테스트

프로그램 전체뿐 아니라 개별 구성 요소도 테스트해야 한다. 개발 과정에서 각 함수를 직접 만든 데이터를 넣어 테스트하여 올바르게 반응하는지 확인해야 한다.

이를 효과적으로 수행하려면 테스트 전용 함수를 만들어야 한다. 이러한 함수들을 드라이버(driver)라고 부른다(하드웨어 드라이버와는 다르다). 드라이버는 테스트할 함수를 호출하고, 데이터를 제공하며, 결과를 검사하는 역할만 한다. 이는 아직 완성되지 않은 프로그램의 일부를 테스트할 때 특히 유용하다.

테스트 중인 코드가 아직 구현되지 않은 함수를 호출하는 경우가 많다. 이를 해결하기 위해 스텝(stub)이라는 간단한 함수를 작성할 수 있다. 스텝은 필요한 값을 단순히 반환해주는 함수이다. 예를 들어, 전자상거래 애플리케이션에서 `is_ready_to_checkout` 라는 함수가 있었다면, 실제 구현 전에 항상 `true` 를 반환하도록 만들어, 해당 함수에 의존하는 다른 코드들을 먼저 테스트 할 수 있다.

[3] 효과적인 오류 처리

테스트 방법뿐만 아니라, 오류가 감지되었을 때 무엇을 해야 하는지도 중요하다.

[3-1] 모든 상황에 대한 오류 코드

진정으로 견고한 소프트웨어는 가능한 모든 상황에 대해 고유한 오류 코드를 가진다. 오류 코드만 보아도 코드의 어느 위치에서 오류가 발생했는지 알 수 있어야 한다.

이는 사용자가 오류 코드를 보고할 때 보통 오류 코드만을 제공할 수 있기 때문에 중요하다. 따라서 오류 코드는 최대한 유용해야 한다. 오류 코드는 설명적인 오류 메시지와 함께 제공되어야 하지만, 오류 원인을 추측하려고 해서는 안된다. 오류가 무엇이 발생했는지를 설명하는 데에만 집중해야 한다.

1995 년에 필자가 근무했던 인터넷 서비스 제공업체의 사례를 보면, 당시 지원하던 웹 브라우저 중 하나는 네트워크 오류의 원인을 항상 추측해서 사용자에게 알려주었다. 인터넷에 연결되지 않은 상태에서 웹 사이트에 접속하면, ISP 에 문제가 있다거나 서버가 다운되었다며 ISP 에 문의하라고 안내했다. 실제로는 인터넷에 연결만 하면 되는 경우였는데, 이 메시지 때문에 전체 문의의 약 4분의 1이 발생했다. 이처럼 원인을 추측하는 것은 문제를 해결하기 보다 더 많은 문제를 만든다. 오류 코드는 단순히 보고하고, 문제 해결은 별도의 문서(문제 해결 가이드)가 담당하는 것이 바람직하다.

[3-2] 복구 지점

오류 처리를 단순화하기 위해 프로그램을 여러 개의 독립적인 단위로 나누고, 각 단위 전체를 하나의 실패 및 복구 단위로 취급하는 것이 유용하다. 예를 들어, 설정 파일 읽기를 하나의 단위로 나눌 수 있다. 설정 파일 열기, 읽기, 해석 중 어디에서든 오류가 발생하면, 단순히 "설정 파일 오류"로 처리하고 해당 복구 지점으로 이동한다. 이렇게 하면 오류 처리 메커니즘의 수를 크게 줄일 수 있다.

단, 오류 메시지는 여전히 구체적이어야 한다. 복구 지점은 오류 복구 단위일 뿐, 오류 탐지를 단순화하는 것은 아니다. 오류 탐지는 여전히 정확해야 하며, 오류 보고 역시 정확한 코드와 메시지를 포함해야 한다.

복구 지점을 사용할 경우, 다양한 상황을 처리하기 위한 정리(clean) 코드가 필요하다. 예를 들어, 설정 파일 오류의 경우, 파일이 열린 채로 남아 있는지 확인해야 할 수 있다. 오류 발생 시점에 따라 파일이 열려 있을 수도 있기 때문이다. 복구 함수는 이러한 상태를 점검하고 시스템을 일관된 상태로 되돌려야 한다.

가장 단순한 복구 방법은 프로그램 전체를 하나의 복구 지점으로 감싸는 것이다. 오류 코드와 메시지를 받아 출력한 후 프로그램을 종료하는 오류 보고 함수를 두는 방식이다. 이는 실제 환경에서는 최선의 방법은 아니지만, 최후의 수단으로는 유용하다.

[4] 프로그램을 더 견고하게 만들기

6장에서 만든 `add-year.s` 프로그램을 조금 더 견고하게 만드는 과정을 살펴본다.

이 프로그램은 비교적 단순하기 때문에, 전체 프로그램을 하나로 감싸는 단일 복구 지점만 사용하겠다. 복구 시에 우리가 할 일은 오류를 출력하고 프로그램을 종료하는 것뿐이다. 이를 위한 코드는 매우 간단하다.

```
.include "linux.s"
.equ ST_ERROR_CODE, 8
.equ ST_ERROR_MSG, 12
.globl error_exit
.type error_exit, @function
error_exit:
    pushl %ebp
    movl %esp, %ebp
```

오류 코드를 출력한다.

```
#Write out error code
movl ST_ERROR_CODE(%ebp), %ecx
pushl %ecx
call count_chars
popl %ecx
movl %eax, %edx
movl $STDERR, %ebx
movl $SYS_WRITE, %eax
int $LINUX_SYSCALL
```

오류 메시지를 출력한다.

```
#Write out error message
movl ST_ERROR_MSG(%ebp), %ecx
pushl %ecx
call count_chars
popl %ecx
movl %eax, %edx
movl $STDERR, %ebx
movl $SYS_WRITE, %eax
int $LINUX_SYSCALL
pushl $STDERR
call write_newline
```

상대 코드 1로 프로그램을 종료한다

```
#Exit with status 1
movl $SYS_EXIT, %eax
movl $1, %ebx
int $LINUX_SYSCALL
```

이 코드를 `error-exit.s` 라는 파일에 입력하라. 이 함수를 호출하려면, 오류 메시지의 주소를 스택에 먼저 push 하고, 그 다음 오류 코드를 push 한 후 이 함수를 호출하면 된다.

이제 `add-year` 프로그램에서 잠재적인 오류 지점을 찾아보자. 먼저, 우리는 `open` 시스템 콜이 실제로 제대로 수행되었는지를 확인하지 않고 있다. 리눅스는 상태 코드를 `%eax` 에 반환하므로, 오류가 있는지 확인해야 한다.

```
#Open file for reading
movl $SYS_OPEN, %eax
movl $input_file_name, %ebx
movl $0, %ecx
movl $0666, %edx
int $LINUX_SYSCALL
movl %eax, INPUT_DESCRIPTOR(%ebp)
```

아래 코드는 %eax 가 음수인지 테스트한다. 음수가 아니면 continue_processing 으로 점프한다. 음수라면, 그 음수가 의미하는 오류 조건을 처리한다.

```
#This will test and see if %eax is
#negative. If it is not negative, it
#will jump to continue_processing.
#Otherwise it will handle the error
#condition that the negative number
#represents.
cmpl $0, %eax
jl continue_processing
```

오류를 전달한다.

```
#Send the error
.section .data
no_open_file_code:
.ascii "0001: W0"
no_open_file_msg:
.ascii "Can't Open Input FileW0"
.section .text
pushl $no_open_file_msg
pushl $no_open_file_code
call error_exit
continue_processing:
```

이처럼 시스템 콜을 호출한 후, 반환값이 0 보다 작은지 검사하여 오류 여부를 확인한다. 만약 그렇다면, 오류 보고 및 종료 루틴을 호출한다.

시스템 콜, 함수 호출, 또는 오류가 발생할 수 있는 모든 명령어 뒤에는 반드시 오류 검사 및 처리 코드를 추가해야 한다.

파일을 어셈블하고 링크하려면 다음과 같이 실행한다.

```
as add-year.s -o add-year.o
as error-exit.s -o error-exit.o
ld add-year.o write-newline.o error-exit.o read-record.o write-record.o count-chars.o -o
add-year
```

이제 필요한 파일 없이 프로그램을 실행해 보자. 프로그램은 이제 깔끔하고 우아하게 종료된다.

문제

[1] 개념 이해하기

- {1} 프로그래머들이 일정에 어려움을 겪는 이유는 무엇인가
- {2} 가장 좋아하는 프로그램을 하나를 골라서, 완전히 잘못된 방식으로 사용해 보자. 잘못된 형식의 파일을 열어보고, 유효하지 않은 옵션을 선택하고, 열려 있어야 할 창을 닫는 등 여러 행동을 해보자. 그 프로그램이 고려해야 했던 서로 다른 오류 상황이 몇 가지나 되는지 세어보자.
- {3} 코너 케이스란 무엇인가, 숫자 데이터에서의 코너 케이스를 예를 들 수 있는가
- {4} 사용자 테스트가 왜 그렇게 중요한가
- {5} 스텝(stub)과 드라이버(driver)가 무엇을 위해 사용되는가, 이 둘의 차이점은 무엇인가
- {6} 복구 지점은 무엇을 위해 사용되는가
- {7} 프로그램에는 서로 다른 오류 코드가 몇 개나 있어야 하는가

[2] 개념 사용하기

- {1} add-year.s 프로그램을 살펴보고, 모든 시스템 콜 뒤에 오류 검사 코드를 추가하라
- {2} 지금까지 작성한 다른 프로그램 하나를 선택하여, 그 프로그램에도 오류 검사를 추가하라
- {3} add-year.s 에 복구 메커니즘을 추가하여, 표준 파일을 열 수 없을 경우 STDIN 에서 읽도록 만들어라.

[3] 심화

- {1} 만약 오류 보고 함수(error-reporting function) 자체가 실패한다면, 무엇을 해야 하는가 혹은 아무것도 하지 않아야 하는가 그 이유는 무엇인가
- {2} 적어도 하나의 오픈 소스 프로그램에서 버그를 찾아보고, 그에 대한 버그 리포트를 제출하라
- {3} 이전 과제에서 찾은 버그를 직접 수정하라.

8장 코드 라이브러리를 이용한 함수 공유

이제쯤이면 컴퓨터가 아주 단순한 작업을 수행하는 데에도 많은 일을 해야 한다는 사실을 깨달았을 것이다. 그 때문에 컴퓨터가 단순한 작업조차 수행하도록 하기 위해서도 우리는 많은 코드를 작성해야 한다. 게다가 프로그래밍 작업은 보통 단순하지 않다. 따라서 우리는 이 과정을 좀 더 쉽게 만들 방법이 필요하다.

이를 위한 여러 방법에는 여러 가지가 있으며, 그 중 대표적인 것은 다음과 같다.

- 어셈블리 언어 대신 고급 언어로 코드를 작성한다.
- 미리 작성된 코드들을 많이 확보해 두고, 이를 잘라 붙여서 자신의 프로그램에 사용한다.
- 시스템에 존재하는 함수 집합을 여러 프로그램이 공유하여 사용할 수 있도록 한다.

이 세 가지 방법은 보통 하나의 프로젝트에서 모두 어느 정도씩 사용된다. 첫 번째 방법은 11장에서 더 자세히 다룰 것이다. 두 번째 방법은 유용하지만 다음과 같은 단점이 있다.

- 복사된 코드는 주변 코드에 맞게 크게 수정해야 하는 경우가 많다.
- 복사된 코드가 포함된 모든 프로그램은 동일한 코드를 각각 포함하게 되어 공간을 낭비한다.
- 복사된 코드에서 버그가 발견되면, 그 코드를 사용한 모든 프로그램을 각각 수정해야 한다.

따라서 두 번째 방법은 보통 제한적으로만 사용된다. 특정 유형의 작업을 위한 뼈대 코드(skeleton code)를 복사한 뒤, 프로그램에 맞는 세부 사항을 추가하는 경우에만 주로 사용된다.

세 번째 방법이 가장 많이 사용된다. 이 방법은 공유 코드의 중앙 저장소를 두는 방식이다. 각 프로그램이 동일한 함수를 각각 저장하느라 공간을 낭비하는 대신, 필요한 함수가 들어 있는 공유 라이브러리를 가리키기만 하면 된다. 만약 이 함수 중 하나에서 버그가 발견되면, 해당 라이브러리 파일 하나만 수정하면 되고, 이를 사용하는 모든 프로그램은 자동으로 수정된 버전을 사용하게 된다.

이 방식의 주요 단점은 의존성 문제를 만든다는 점이다. 예를 들어, 다음과 같다.

- 여러 애플리케이션이 하나의 공유 파일을 사용하고 있을 때, 언제 그 파일을 삭제해도 안전한지 어떻게 알 수 있는가? 예를 들어, 세 개의 애플리케이션이 하나의 함수 파일을 공유하고 있는데, 그중 두 개가 삭제되었다면, 시스템은 아직 남아 있는 하나의 프로그램이 그 코드를 사요하고 있다는 사실을 어떻게 알 수 있을까?
- 일부 프로그램은 공유 함수에 존재하던 버그에 의존하고 있을 수도 있다. 따라서 공유 라이브러리를 업그레이드하면서 해당 버그를 수정하면, 그 버그에 의존하던 프로그램이 오히려 동작하지 않게 될 수 있다.

이러한 문제들을 흔히 "DLL 지옥(DLL hell)"이라고 불리는 현상의 원인이 된다. 그럼에도 불구하고 일반적으로는 장점이 단점보다 크다고 여겨진다.

프로그래밍에서 이러한 공유 코드 파일들을 공유 라이브러리(shared libraries), 공유 객체(shared objects), 동적 링크 라이브러리(dynamic-link libraries), DLL, 또는 .so 파일이라고 불린다. 이 책에서는 이를 공유 라이브러리라고 부르겠다.

[1-1] 공유 라이브러리 사용하기

여기서 살펴볼 프로그램은 매우 단순하다. 화면에 hello world 라는 문자를 출력하고 종료하는 프로그램이다. 라이브러리를 사용하지 않은 일반적인 프로그램 helloworld-nolib.s 는 다음과 같다.

```
# 목적: "hello world" 메시지를 화면에 출력하고 프로그램을 안전하게 종료합니다.
#
# 사용되는 시스템 콜:
# - SYS_WRITE (4): 데이터를 파일(또는 화면)에 씀
# - SYS_EXIT (1): 프로그램을 종료함

.include "linux.s"          # SYS_WRITE, SYS_EXIT, STDOUT 등의 상수가 정의된 파일 포함

.section .data
helloworld:
    .ascii "hello world\n" # 출력할 문자열 정의 (끝에 줄바꿈 \n 포함)

helloworld_end:
    # [길이 계산 기술]
    # helloworld_end 는 문자열이 끝난 직후의 주소를 가리킵니다.
    # (끝 주소 - 시작 주소)를 계산하면 문자열의 바이트 수가 자동으로 계산됩니다.
    # 이렇게 하면 문자열을 수정해도 길이를 일일이 다시 셀 필요가 없습니다.
    .equ helloworld_len, helloworld_end - helloworld

.section .text
.globl _start

_start:
    ### 1. 화면에 메시지 출력 (SYS_WRITE) ###
    movl $STDOUT, %ebx      # 첫 번째 인자: 어디에 쓸 것인가? (표준 출력 = 1)
    movl $helloworld, %ecx   # 두 번째 인자: 쓸 데이터의 주소는 어디인가?
```

```

movl $helloworld_len, %edx # 세 번째 인자: 몇 바이트를 쓸 것인가?
movl $SYS_WRITE, %eax      # 시스템 콜 번호 넣기 (write = 4)
int $LINUX_SYSCALL         # 리눅스 커널 호출 (실행!)

### 2. 프로그램 종료 (SYS_EXIT) ###
movl $0, %ebx              # 첫 번째 인자: 종료 상태 코드 (0 = 정상 종료)
movl $SYS_EXIT, %eax       # 시스템 콜 번호 넣기 (exit = 1)
int $LINUX_SYSCALL         # 리눅스 커널 호출 (종료!)

```

이 프로그램은 길지 않다. 하지만 라이브러리를 사용하는 helloworld-lib 프로그램이 얼마나 짧은지 살펴보자.

```

# 목적: C 표준 라이브러리(printf, exit)를 호출하여 "hello world"를 출력하고 종료합니다.
#
# 이 방식의 특징:
# - 시스템 콜 번호를 외울 필요가 없습니다.
# - printf 같은 고수준 함수를 사용할 수 있습니다.
# - 컴파일 시 C 라이브러리와 연결(Link)해야 합니다.

.section .data
helloworld:
    # printf는 문자열의 끝을 NULL(0) 문자로 인식합니다.
    # 따라서 끝에 \0을 반드시 붙여줘야 합니다.
    .ascii "hello world\0"

.section .text
.globl _start

_start:
    # [printf 호출 과정]
    # C 함수 호출 규약: 인자를 스택에 '역순'으로 넣습니다.
    # printf(helloworld)를 호출하려는 것이므로 helloworld의 주소를 푸시합니다.
    pushl $helloworld      # 문자열의 주소를 스택에 푸시
    call printf             # printf 함수 호출

    # [exit 호출 과정]
    # exit(0)을 호출하기 위해 인자 0을 스택에 푸시합니다.
    pushl $0               # 종료 상태 코드 0을 스택에 푸시
    call exit               # exit 함수 호출 (프로그램 종료)

```

훨씬 짧다! 하지만 공유 라이브러리를 사용하는 프로그램을 빌드하는 과정은 일반적인 경우와 조금 다르다.

첫 번째 프로그램은 다음과 같이 평소처럼 빌드할 수 있다.

```
as helloworld-nolib.s -o helloworld-nolib.o
ld helloworld-nolib.o -o helloworld-nolib
```

하지만 두 번째 프로그램을 빌드하려면 다음과 같이 해야 한다 .

```
as helloworld-lib.s -o helloworld-lib.o
ld -dynamic-linker /lib/ld-linux.so.2 W
-o helloworld-lib helloworld-lib.o -lc
```

첫 번째 줄의 백슬래쉬(W)는 명령이 다음 줄로 이어진다는 의미이다.

-dynamic-linker /lib/ld-linux.so.2 옵션은 프로그램이 라이브러리에 연결될 수 있도록 한다. 이 옵션은 실행 전에 운영체제가 /lib/ld-linux.so.2 프로그램을 먼저 로드하여 외부 라이브러리를 불러오고 프로그램과 연결하도록 만든다. 이 프로그램을 동적 링커(dynamic linker)라고 한다.

-lc 옵션은 GNU/Linux 시스템에서 libc.so 라고 불리는 C 라이브러리에 연결하겠다는 의미이다. 링커는 라이브러리 이름 c 앞에 lib 를 붙이고 뒤에 .so 를 붙여서 실제 파일 이름을 만든다.

이 라이브러리에는 다양한 작업을 자동화하는 많은 함수들이 있다. 우리가 사용하는 함수는 문자열을 출력하는 printf 와 프로그램을 종료하는 exit 이다.

[1-2] 공유 라이브러리는 어떻게 동작하는가

우리가 처음 작성한 프로그램들에서는 모든 코드가 하나의 소스 파일 안에 들어 있었다. 이러한 프로그램들을 정적 링크 실행 파일(statically-linked executables)이라고 부르는데, 이는 커널에서 처리되지 않는 프로그램 실행에 필요한 모든 기능이 프로그램 내부에 포함되어 있기 때문이다.

6 장에서 작성한 프로그램들에서는 메인 프로그램 파일뿐만 아니라 여러 프로그램에서 공통으로 사용하는 루틴들이 들어 있는 파일들도 사용했다. 이 경우에도 링크 시점(link-time)에 링커를 사용해 모든 코드를 하나로 결합했기 때문에, 여전히 정적 링크 방식이었다.

하지만 helloworld-lib 프로그램에서는 공유 라이브러리를 사용하기 시작했다.

공유 라이브러리를 사용하면 프로그램은 동적(dynamically-linked)가 된다. 이는 프로그램을 실행하는 데 필요한 모든 코드가 프로그램 파일 자체에 들어 있는 것이 아니라, 외부 라이브러리에 들어 있다는 뜻이다.

helloworld 프로그램을 링크할 때 명령에 `-lc` 옵션을 추가하면, 이는 링커에게 helloworld.o 안에 정의되지 않은 심볼(symbol)을 C 라이브러리(libc.so)에서 찾으라고 지시하는 것이다. 그러나 실제로는 어떤 코드도 프로그램 안에 추가하지 않고, 단지 프로그램이 어디를 참조해야 하는지만 기록한다.

helloworld 프로그램이 시작되면, 먼저 `/lib/ld-linux.so.2` 파일이 로드된다. 이것이 동적 링커(dynamic linker)이다.

동적 링커는 helloworld 프로그램을 살펴보고, 실행을 위해 C 라이브러리가 필요하다는 것을 확인한다. 그 다음 표준 위치들(`/etc/ld.so.conf`에 나열된 경로들과 `LD_LIBRARY_PATH` 환경변수에 지정된 경로들)에서 libc.so 라는 파일을 찾는다.

그 후 필요한 모든 심볼(이 경우에는 `printf` 와 `exit`)을 라이브러리 안에서 찾아 프로그램의 가상 메모리에 로드한다. 마지막으로, 프로그램 안에 있는 모든 `printf` 호출을 라이브러리 안에 있는 실제 `printf` 함수의 주소로 교체한다.

다음 명령을 실행해 보자.

```
ldd ./helloworld-nolib
```

그러면 동적 실행 파일이 아니다(not a dynamic executable)라고 출력될 것이다. 이는 앞에서 말한 것처럼 helloworld-nolib 이 정적 링크 실행 파일이기 때문이다.

하지만 다음을 실행해보자.

```
ldd ./helloworld-lib
```

그러면 다음과 비슷한 결과가 출력될 것이다.

```
libc.so.6 => /lib/libc.so.6 (0x4001d000)
/lib/ld-linux.so.2 => /lib/ld-linux.so.2 (0x40000000)
```

괄호 안의 숫자는 시스템마다 다를 수 있다. 이 출력은 helloworld 프로그램이 libc.so.6(여기서 .6 은 버전 번호이다)에 링크되어 있으며, 이 파일이 /lib/libc.so.6 에 있고, /lib/ld-linux.so.2 는 /lib/ld-linux.so.2 에 있다는 뜻이다.

이 라이브러리들은 프로그램이 실행되기 전에 반드시 로드되어야 한다. 관심이 있다면, 자신의 리눅스 배포판에 포함된 여러 프로그램에 대해 ldd 명령을 실행해보고, 어떤 라이브러리들에 의존하고 있는지 확인해 보자.

[2-1] 라이브러리에 대한 정보 찾기

이제 라이브러리에 대해 알게 되었으니, 다음 질문은 시스템에 어떤 라이브러리들이 있고 그것들이 무엇을 하는지 어떻게 알아내느냐는 것이다.

하지만 잠시 이 질문을 미뤄두고 다른 질문을 하겠다. 프로그래머들은 문서에서 서로에게 함수들을 어떻게 설명할까?

printf 함수를 살펴본다. 이 함수의 호출 인터페이스(보통 프로토타입이라고 부름)는 다음과 같다.

```
int printf(char *string, ...);
```

리눅스에서 함수들은 C 프로그래밍 언어로 설명된다. 실제로 대부분의 리눅스 프로그램은 C 로 작성된다. 그렇기 때문에 대부분의 문서와 바이너리 호환성은 C 언어를 기준으로 정의된다. 위에 나온 printf 함수의 인터페이스 역시 C 언어로 기술되어 있다.

이 정의는 printf 라는 함수가 존재한다는 것을 의미한다. 괄호 안에 있는 것들은 함수의 매개변수(인자)들이다. 첫 번째 매개변수는 char *string 이다. 이는 string 이라는 이름의 매개변수가 있으며(이름 자체는 설명을 위한 것일 뿐 중요하지 않다), 타입이 char * 라는 뜻이다. char 는 1 바이트 문자 하나를 의미한다. 뒤에 붙은 *는 실제 문자가 아니라 문자 하나 또는 문자들의 연속이 저장된 주소를 인자로 원한다는 뜻이다.

앞에서 본 helloworld 프로그램을 떠올려 보면, 함수 호출은 다음과 같았다.

```
pushl $hello  
call printf
```

즉, 실제 문자열이 아니라 hello 문자열의 주소를 스택에 넣은 것이다.

문자열의 길이를 푸시하지 않았다는 점도 눈치챘을 것이다. printf 가 문자열의 끝을 알 수 있었던 이유는 문자열의 끝에 널 문자(W0)를 붙였기 때문이다. 많은 함수들, 특히 C 언어 함수들이 이런 방식으로 동작한다.

함수 정의 앞에 있는 int 는 함수가 반환할 값의 타입을 의미하며, 반환 시 그 값은 %eax 레지스터에 들어간다. printf 는 실행이 끝나면 int 값을 반환한다. '

이제 char *string 뒤에 점 세 개(...)가 있다. 이는 문자열 뒤에 개수 제한 없이 추가 인자들을 받을 수 있다는 의미다. 대부분의 함수는 정해진 개수의 인자만 받을 수 있지만, printf 는 여러 개를 받을 수 있다.

printf 는 문자열 매개변수를 살펴보고, %s 가 나오면 스택에서 문자열 하나를 가져와 삽입하고, %d 가 나오면 숫자를 하나 가져와 삽입한다. 이는 다음 예제를 보면 가장 잘 이해할 수 있다.

```
# PURPOSE: 이 프로그램은 printf 호출 방법을 보여주기 위한 것이다
#
.section .data
# 이 문자열은 포맷 문자열이라 불린다. 첫 번째 매개변수이며,
# printf 는 이를 이용해 몇 개의 매개변수가 전달되었는지,
# 그리고 그 타입이 무엇인지 알아낸다.
firststring:
.ascii "Hello! %s is a %s who loves the number %d\n\n"
name:
.ascii "Jonathan\n"
personstring:
.ascii "person\n"
# 이것은 .equ 로도 만들 수 있었지만,
# 그냥 실제 메모리 위치를 주기로 했다
numberloved:
.long 3

.section .text
.globl _start
_start:
# 매개변수는 함수 프로토타입에 나열된 순서의
# 역순으로 전달된다는 점에 주의하라
pushl numberloved      # 이것이 %d
pushl $personstring    # 이것이 두 번째 %s
pushl $name            # 이것이 첫 번째 %s
pushl $firststring     # 이것이 프로토타입에 나온
                      # 포맷 문자열
call printf

pushl $0
call exit
```

이 코드를 printf_example.s 라는 파일 이름으로 입력한 뒤, 다음 명령을 실행하라.


```
as printf-example.s -o printf-example.o
ld printf-example.o -o printf-example -lc W
-dynamic-linker /lib/ld-linux.so.2
```

그 다음 ./printf-example 를 실행하면 다음과 같은 출력이 나올 것이다.

```
Hello! Jonathan is a person who loves the number 3
```

코드를 보면, 포맷 문자열이 프로토타입에서 첫 번째 인자임에도 불구하고 실제로는 마지막에 push 된 것을 볼 수 있다. 함수의 매개변수는 항상 역순으로 스택에 전달한다.

printf 가 매개변수 개수를 어떻게 아는지 궁금할 수도 있다. printf 는 문자열을 훑어보면서 %d 와 %s 가 몇 개 있는지 세고, 그 개수만큼의 인자를 스택에서 꺼낸다. %d 에 해당하면 숫자로 처리하고, %s 에 해당하면 널 종료 문자열을 가리키는 포인터로 처리한다.

printf 에는 이외에도 많은 기능이 있지만, 여기서는 가장 자주 사용되는 것들만 다뤘다. 보듯이 printf 는 출력 작업을 매우 쉽게 해 주지만, 문자열 길이를 계산하고, 제어 문자를 찾고, 스택에서 인자를 꺼내고, 숫자를 문자열로 변환하는 등 많은 작업을 수행해야 하므로 오버헤드도 크다.

우리는 C 언어의 함수 프로토타입을 사용해 라이브러리 함수를 호출하는 방법을 살펴보았다. 이를 제대로 활용하려면 사용되는 데이터 타입들도 알아야 한다. 주요 타입들은 다음과 같다.

int: 정수형 숫자(x86 프로세서에서 4 바이트)

long: 정수형 숫자(x86 프로세서에서 4 바이트)

long long: long 보다 큰 정수형 숫자(x86 프로세서에서 8 바이트)

short: int 보다 작은 정수형 숫자(x86 프로세서에서 2 바이트)

char: 1 바이트 정수형 숫자, 주로 문자 데이터를 저장하는데 사용되며, ASCII 문자열은 보통 문자 하나당 1 바이트를 사용한다.

float: 부동소수점 숫자(x86 프로세서에서 4 바이트), 부동소수점 수는 10 장에서 더 자세히 설명된다.

double: float 보다 큰 부동소수점 숫자(x86 프로세서에서 8 바이트)

unsigned: 위 타입들 앞에 붙어 부호 없는 값으로 사용되도록 하는 수식자이다. signed 와 unsigned 의 차이는 10 장에서 다룬다.

* : 별포(*)는 실제 값이 아니라 해당 값을 저장하고 있는 주소(포인터)임을 나타낸다. 예를 들어, my_location 이라는 메모리 위치에 값 20 이 저장되어 있다고 하자. 프로토타입에서 int 를 요구한다면 pushl my_location 을 사용한다. 하지만 int *를 요구한다면 pushl \$my_location 을 사용해 주소를 넘긴다.

포인터는 단일 값의 주소뿐만 아니라, 연속된 메모리 공간의 시작 주소를 전달하는 데도 사용된다. 이를 배열(array)이라고 한다.

struct: struct 는 여러 데이터 항목을 하나의 이름 아래 묶은 것이다. 예를 들어 다음과 같이 선언할 수 있다.

```
struct teststruct {  
    int a;  
    char *b;  
};
```

struct teststruct 를 보면, 이는 연속된 두 개의 워드로 이루어져 있으며, 첫 번째는 정수이고, 두 번째는 문자(또는 문자열)를 가리키는 포인터임을 알 수 있다.

struct 자체가 함수 인자로 전달되는 경우는 거의 없으며, 대신 struct 에 대한 포인터가 전달된다. 이는 struct 가 차지하는 메모리 공간이 커질 수 있어 전달이 복잡하기 때문이다.

typedef: typedef는 타입에 새로운 이름을 붙이는 기능이다. 예를 들어, C 프로그램에서 다음과 같이 쓸 수 있다.

```
typedef int myowntype;
```

이후 myowntype를 사용하면 int와 동일하다. 이 때문에 함수 프로토타입을 볼 때 typedef나 struct 가 실제로 무엇을 의미하는지 찾아봐야 해서 번거로울 수 있다. 하지만 typedef 는 타입에 더 의미 있고 설명적인 이름을 붙이는 데 유용하다.

호환성 참고 사항: 여기에 나열된 크기들은 인텔 호환(x86) 머신 기준이다. 다른 머신에서는 크기가 다를 수 있다. 또한 워드보다 짧은 매개변수라도 함수 호출 시에는 스택에 long 크기로 전달된다.

이것이 함수 문서를 읽는 방법이다. 이제 다시 라이브러리에 대한 정보를 어떻게 찾는지로 돌아가자. 대부분의 시스템 라이브러리는 /usr/lib 또는 /lib 에 있다. 라이브러리가 정의하는 심볼만 보고 싶다면 다음 명령을 실행하면 된다.

`objdump -R FILENAME`

여기서 FILENAME 은 라이브러리의 전체 경로이다.

하지만 이 출력은 필요한 인터페이스를 찾기에는 그다지 도움이 되지 않는다. 보통은 처음부터 어떤 라이브러리를 사용할지 알고 있어야 하며, 문서를 읽는 방식이 일반적이다.

대부분의 라이브러리는 메뉴얼이나 man 페이지를 제공한다. 라이브러리 문서를 찾는데 가장 좋은 곳은 웹이다. GNU 프로젝트의 라이브러리들이나 man 페이지보다 더 자세한 info 페이지를 제공하는 경우도 많다.

[3-1] 유용한 함수들

C 라이브러리에는 알아두면 좋은 여러 유용한 함수들이 있다.

`size_t strlen(const char *s)`

- 널(null)로 종료되는 문자열의 길이를 계산한다.

`int strcmp(const char *s1, const char *s2)`

- 두 문자열을 알파벳 순서로 비교한다.

`char *strdup(const char *s)`

- 문자열을 가리키는 포인터를 받으며, 새로운 메모리 위치에 문자열의 복사본을 만드록 그 새로운 위치를 반환한다.

`FILE *fopen(const char *filename, const char *opentype)`

- 관련되고 버퍼링되는 파일을 연다.(파일 디스크립터를 직접 사용하는 것보다 읽기와 쓰기가 더 쉽다)

`int fclose(FILE *stream)`

- fopen 으로 열었던 파일을 닫는다.

`char *fgets(char *s, int count, FILE *stream)`

- 파일에서 한 줄의 문자를 읽어 문자열 `s`에 저장한다.

`char fputs(const char *s, FILE *stream)`

- 열린 파일에 문자열을 쓴다.

`int fprintf(FILE *stream, const char *template, ...)`

- `printf`와 거의 동일하지만, 표준 출력이 아니라 열린 파일에 출력한다.

이 라이브러리에 대한 전체 메뉴얼은 다음 주소에서 확인할 수 있다.

<http://www.gnu.org/software/libc/manual/>

[3-2] 공유 라이브러리 만들기

이제 6장에서 사용했던 모든 공용 코드를 하나의 공유 라이브러리로 만들어 프로그램에서 사용하고 싶다고 가정해 보자.

먼저, 평소처럼 각각을 어셈블한다.

```
as write-record.s -o write-record.o
as read-record.s -o read-record.o
```

이제 이 오브젝트 파일들을 하나의 프로그램으로 링크하는 대신, 공유 라이브러리로 링크한다. 그러면 링크 명령은 다음과 같이 바뀐다.

```
ld -shared write-record.o read-record.o -o librecord.so
```

이 명령은 두 파일을 하나로 묶어 `librecord.so`라는 공유 라이브러리를 만든다.

이 파일은 여러 프로그램에서 사용할 수 있다. 만약 내부의 함수를 수정해야 한다면, 이 파일 하나만 수정하면 되며, 어떤 프로그램들이 이를 사용하는지 일일이 신경 쓸 필요가 없다.

이제 이 라이브러리를 사용하는 방법을 살펴보자. `write-records` 프로그램을 링크하려면 다음과 같이 한다.

```
as write-records.s -o write-records.o
ld -L. -dynamic-linker /lib/ld-linux.so.2 \
-o write-records -lrecord write-records.o
```

여기서 `-L.` 옵션은 링커에게 현재 디렉터리에서 라이브러리를 찾으라고 지시한다.(기본저공로 `/lib`, `/usr/lib` 등의 디렉터리만 검색한다.)

앞서 본 것처럼 `-dynamic-linker /lib/ld-linux.so.2` 는 동적 링커를 지정한다.
`-lrecord` 옵션은 `librecord.so` 파일에서 함수들을 찾도록 링커에게 지시한다.

이제 `write-records` 프로그램은 빌드되었지만, 그대로 실행하면 동작하지 않는다. 실행을 시도하면 다음과 같은 오류가 발생한다.

```
./write-records: error while loading shared libraries:
librecord.so: cannot open shared object file: No such file or directory
```

이는 기본적으로 동적 링커가 `/lib`, `/usr/lib`, 그리고 `/etc/ld.so.conf` 에 등록된 디렉터리만 라이브러리 검색 대상으로 삼기 때문이다.

이 문제를 해결하려면 라이브러리를 해당 디렉터리 중 하나로 옮기거나, 다음 명령을 실행해야 한다.

```
LD_LIBRARY_PATH=.
export LD_LIBRARY_PATH
```

만약 이 방법이 오류를 낸다면, 대신 다음을 사용한다.

```
setenv LD_LIBRARY_PATH .
```

이제 `./write-recors` 를 입력하면 프로그램이 정상적으로 실행된다.

`LD_LIBRARY_PATH` 를 설정하면, 동적 링커가 공유 라이브러리를 검색할 경로에 사용자가 지정한 경로를 추가하게 된다.

[4] 추가 참고 자료

동적 링크에 대해 더 알고 싶다면 다음 자료들을 참고하라.

ld.so 의 man 페이지에는 리눅스 동적 링커의 동작 방식에 대한 많은 정보가 담겨 있다.

<http://www.benyossef.com/presentations/dlink/>

- 리눅스 동적 링크에 대한 훌륭한 발표 자료

<http://www.linuxjournal.com/article.php?sid=1059>

<http://www.linuxjournal.com/article.php?sid=1060>

- ELF 파일 형식에 대한 좋은 입문 자료이며, 더 자세한 내용은

<http://www.cs.ucdavis.edu/~haungs/paper/node10.html>

- 에서 확인할 수 있다.

<http://www.iecc.com/linker/linker10.html>

- ELF 파일에서 동적 링크가 어떻게 동작하는지를 매우 잘 설명한 자료

문제

[1] 개념 이해하기

- {1} 공유 라이브러리의 장점과 단점은 무엇인가
- {2} 라이브러리의 이름이 foo 라면, 실제 라이브러리 파일 이름은 무엇인가
- {3} ldd 명령어는 무엇을 하는가
- {4} foo.o 와 bar.o 파일이 있고, 이 둘을 함께 링크하면서 kramer 라는 라이브러리를 동적으로 링크하고 싶다면, 최종 실행 파일을 만들기 위한 링크 명령은 무엇인가
- {5} typedef 는 무엇을 위해 사용되는가
- {6} struct 는 무엇을 위해 사용되는가
- {7} int 타입의 데이터 요소와 int * 타입의 데이터 요소의 차이는 무엇인가, 또한 프로그램에서 이 둘은 어떻게 다르게 접근해야 하는가
- {8} foo.o 라는 오브젝트 파일이 있을 때, bar 라는 이름의 공유 라이브러리를 생성하는 명령은 무엇인가
- {9} LD_LIBRARY_PATH 의 목적은 무엇인가

[2] 개념 사용하기

- {1} 이전 장에서 작성한 프로그램 중 하나 이상을 수정하여, 종료 상태 코드로 결과를 반환하는 대신 printf 를 사용해 화면에 결과를 출력하라. 또한 종료 상태 코드는 0 으로 설정하라.
- {2} 4 장 ‘재귀함수’ 절에서 작성한 factorial 함수를 공유 라이브러리로 만들어라. 그런 다음, 메인 프로그램을 다시 작성하여 해당 라이브러리를 동적으로 링크하도록 하라.
- {3} 위 프로그램을 다시 수정하여 c 라이브러리에도 링크하도록 하라. 그리고 c 라이브러리의 printf 함수를 사용해 팩토리얼 계산 결과를 출력하라.
- {4} toupper 프로그램을 다시 작성하여, 시스템 콜 대신 c 라이브러리의 파일 관련 함수들을 사용하도록 하라.

[3] 심화

- {1} GNU/Linux 동적 링커가 사용하는 모든 환경 변수들의 목록을 만들어라
- {2} 현재 사용되고 있거나 과거에 사용되었던 다양한 실행 파일 형식들을 조사하라. 각 형식의 장점과 단점을 설명하라
- {3} 그래픽, 데이터베이스, 과학 계산 등 자신이 관심 있는 프로그래밍 분야를 하나 선택하라. 그 분야에서 사용되는 라이브러리를 찾아, 해당 라이브러리를 기본적으로 사용하는 프로그램을 작성하라.
- {4} LD_PRELOAD 의 사용 목적을 조사하라
exit 함수를 포함하는 공유 라이브러리를 만들어, 프로그램이 종료되기 전에 STDERR 에 메시지를 출력하도록 하라. 그런 다음 LD_PRELOAD 를 사용해 여러 프로그램을 실행하라. 그 결과는 무엇인가

9 장 중급 메모리 주제

[1] 컴퓨터가 메모리를 바라보는 방식

컴퓨터 내부의 메모리가 어떻게 동작하는지 다시 한 번 살펴본다. 2 장도 다시 읽어보면 도움이 될 것이다.

컴퓨터는 메모리를 번호가 매겨진 아주 긴 저장 공간의 연속으로 본다. 수백만 개에도 달하는 번호가 붙은 저장 공간의 나열이다. 모든 것은 이 저장 공간에 저장된다. 프로그램도, 데이터도, 모든 것이 여기에 들어 있다.

각 저장 공간은 서로 완전히 동일하게 보인다. 프로그램을 담고 있는 공간과 데이터를 담고 있는 공간은 컴퓨터 입장에서 차이가 없다. 실제로 실행 파일이 “어디서부터 실행을 시작하라”고 알려주지 않는 한, 컴퓨터는 어떤 것이 프로그램이고 어떤 것이 데이터인지 알지 못한다.

이 저장 공간 하나하나를 바이트(byte)라고 한다. 컴퓨터는 이 바이트 최대 네 개를 묶어 하나의 워드(word)로 사용할 수 있다. 일반적으로 숫자 데이터는 워드 단위로 처리된다.

앞서 언급했듯이, 명령어 또한 같은 메모리 공간에 저장된다. 각 명령어는 길이가 서로 다르다. 대부분의 명령어는 명령 자체를 위해 한두 개의 저장 공간을 사용하고, 그 뒤에 명령어 인자(arguments)를 위한 저장 공간을 사용한다.

예를 들어, 다음 명령어는:

```
movl data_items(%edi,4), %ebx
```

총 7 개의 저장 공간을 차지한다.

- 처음 두 개는 명령어 자체
- 세 번째는 어떤 레지스터를 사용할지에 대한 정보
- 나머지 네 개는 data_items 의 메모리 주소를 담고 있다.

메모리 안에서 명령어는 다른 숫자들과 완전히 동일하게 보인다. 명령어 자체도 숫자이기 때문에, 일반 숫자처럼 레지스터로 옮기거나 다시 메모리로 옮길 수 있다.

이 장은 컴퓨터 메모리의 세부적인 동작에 초점을 맞춘다. 먼저 이 장에서 사용할 기본 용어들을 정리해 보자.

[2] 기본 용어

Byte(바이트): 저장 공간 하나의 크기이다. x86 프로세서에서 바이트는 0 부터 255 까지의 숫자를 저장할 수 있다.

Word(워드): 일반적인 레지스터의 크기이다. x86 프로세서에서 워드는 4 바이트이다. 대부분의 컴퓨터 연산은 워드 단위로 처리된다.

Address(주소): 메모리에서 하나의 바이트를 가리키는 숫자이다. 예를 들어, 첫 번째 바이트의 주소는 0, 두 번째는 1 이다. 레지스터에 있지 않은 모든 데이터는 주소를 가진다. 여러 바이트에 걸쳐 데이터의 주소는 첫 번째 바이트의 주소를 사용한다.

보통 우리는 실제 숫자 주소를 직접 입력하지 않고, 어셈블러에게 맡긴다. 코드에서 사용하는 레이블(label)은 그 위치의 주소와 동일한 의미를 가진다. 어셈블러는 프로그램에서 해당 심볼을 사용할 때마다 실제 주소로 치환한다.

예를 들어, 다음 코드가 있다고 가정한다.

```
.section .data
my_data:
.long 2, 3, 4
```

프로그램에서 my_data 를 사용할 때마다, 이는 .long 지시어의 첫 번째 주 값의 주소로 대체된다.

Pointer(포인터): 포인터는 주소 값을 담고 있는 레지스터나 메모리 워드이다. 프로그램에서 %ebp 는 현재 스택 프레임을 가리키는 포인터로 사용된다. 모든 베이스 포인터 주소 지정은 포인터를 기반으로 한다. 포인터는 매우 중요한 개념이다.

[3] 리눅스 프로그램의 메모리 구조

프로그램이 메모리에 로드되면 각 .section 은 각각 독립된 메모리 영역에 로드된다. 소스 코드에서 떨어져 있더라도, 같은 섹션에 속한 코드와 데이터는 한 곳에 모인다.

- 실제 명령어(.text 섹션)는 0x08048000 주소에 로드된다.
- .data 섹션은 그 뒤에
- .bss 섹션은 그 다음에 위치한다.

리눅스에서 접근 가능한 마지막 주소는 0xbfffffff 이다. 스택은 이 위치에서 시작하여 아래쪽(낮은 주소)으로 성장한다. 그 사이에는 매우 큰 빈공간이 존재한다.

[3-1] 프로그램 시작 시 스택 구조

스택의 맨 아래(메모리 상 가장 높은 주소)에는 값이 0 인 워드가 있다.

그 다음에는 널 종료(null-terminated)된 프로그램 이름이 ASCII 문자열로 저장된다.

그 뒤에는 환경 변수들이 오고 그 다음에 명령줄 인자(command-line arguments)가 위치한다.

예를 들어 as sourcefile.s -o objectfile.o 를 실행하면, as, sourcefile.s, -o, objectfile.o 가 인자로 들어간다.

그 다음에는 인자의 개수가 저장된다.

프로그램이 시작될 때 %esp 는 이 위치를 가리킨다.

스택에 push 가 발생할수록 %esp 는 더 낮은 주소로 이동한다.

예를 들어,

```
pushl %eax
```

은 다음과 동일하다.

```
movl %eax, (%esp)
subl $4, %esp
```

마찬가지로,

```
pop %eax
```

은 다음과 같다.

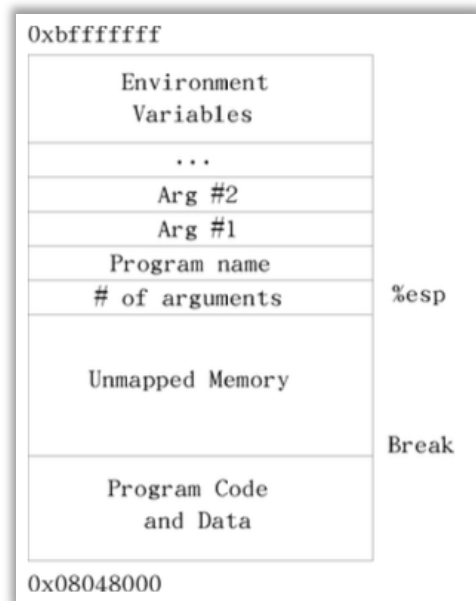
```
movl (%esp), %eax
addl $4, %esp
```

[4] 접근 불가능한 메모리 영역

프로그램의 데이터 영역은 메모리의 아래쪽에서 위로 확장되고, 스택은 위에서 아래로 확장된다.

이 둘 사이의 중간 영역은 접근 불가능한 메모리이다. 커널에 요청하지 않는 한 접근할 수 없으며, 접근 시 보통 segmentation fault 오류가 발생한다.

프로그램이 접근할 수 있는 마지막 주소를 system break(break)라고 한다.



[5] 모든 메모리 주소는 거짓이다.

[5-1] 모든 메모리 주소는 왜 ‘거짓’인가

프로그램이 사용하는 메모리 주소는 실제 RAM 의 위치를 직접 가리키지 않는다. 우리가 코드에서 접근하는 모든 주소는 가상 주소(virtual address)이며, 실제 하드웨어 메모리인 물리 주소(physical address)와는 분리되어 있다. 이 구조 덕분에 프로그램은 자신이 메모리를 독점적으로 사용하는 것처럼 동작할 수 있고, 다른 프로그램의 메모리를 침범하지 않는다. 즉, 프로그램이 믿는 메모리 주소는 운영체제가 만들어낸 일종의 환상이며, 그래서 “모든 메모리 주소는 거짓”이라고 표현할 수 있다.

[5-2] 물리 메모리와 가상 메모리의 차이

물리 메모리는 컴퓨터 내부에 실제로 존재하는 RAM 칩을 의미하며, 용량이 한정되어 있다. 물리 주소는 이 RAM 칩 내의 정확한 위치를 나타낸다. 반면 가상 메모리는 프로그램 관점에서 보이는 메모리 공간으로, 실제 위치와는 무관하게 연속적인 주소 공간처럼 보이도록 설계되어 있다. 프로그램은 물리 메모리에 직접 접근할 수 없고, 오직 가상 메모리만 대상으로 동작한다. 운영체제는 가상 주소와 물리 주소를 연결하는 중간 관리자 역할을 수행한다.

물리 메모리(physical memory): 실제 RAM 칩
- 가상 메모리(virtual memory): 프로그램이 인식하는 메모리

[5-3] 모든 프로그램이 같은 주소에서 시작해도 괜찮은 이유

모든 사용자 프로그램은 자신이 항상 동일한 주소에서 실행된다고 믿는다. 예를 들어, 코드 영역은 보통 0x08048000 에서 시작하고, 스택은 0xbfffffff 근처에서 시작한다고 인식한다. 하지만 실제로는 서로 다른 프로그램이 전혀 다른 물리 메모리 영역을 사용한다. 리눅스는 프로그램을 실행할 때 비어 있는 물리 메모리를 하나 할당하고, 그 물리 메모리를 해당 프로그램에게는 특정 가상 주소로 보이도록 CPU 에 설정한다. 이 덕분에 여러 프로그램이 동시에 실행되더라도 서로의 메모리를 덮어쓰지 않는다. 이 가상 주소와 물리 주소를 연결하는 과정을 매핑(mapping)이라 한다.

[5-4] break 영역이 접근 불가능한 이유

프로그램의 메모리 구조에서 .bss 영역과 스택 사이에는 접근할 수 없는 빈 공간이 존재한다. 이 공간이 바로 break 이후의 영역이다. 이 영역은 실제로 비어 있어서 접근할 수 없는 것이 아니라, 아직 물리 메모리에 매핑되지 않은 가상 주소 영역이기 때문에 접근이 불가능하다. 모든 가상 주소를 미리 물리 메모리에 매핑하면 메모리가 낭비되므로, 운영체제는 필요한 부분만 선택적으로 매핑한다. 따라서 프로그램이 매핑되지 않은 break 영역을 접근하면 운영체제는 이를 잘못된 접근으로 판단하고 Segmentation Fault 를 발생시킨다.

- 프로그램이 인식하는 주소 -> 가상 주소
- 실제 RAM 주소 -> 물리 주소
- 이 둘을 연결하는 과정 -> 매핑(mapping)

[5-5] 스택 메모리가 예외적으로 동작하는 이유

스택은 break 영역과 달리 접근 시 자동으로 메모리가 확장된다. 이는 리눅스가 스택에 대해서는 특별한 정책을 적용하기 때문이다. 프로그램이 스택에 데이터를 푸시하면서 아직 매핑되지 않은 영역을 접근하면, 운영체제는 이를 감지하고 즉시 새로운 물리 메모리를 할당하여 해당 가상 주소에 매핑한다. 이 방식 덕분에 스택은 필요에 따라 자연스럽게 성장할 수 있으며, 프로그래머는 이를 직접 관리할 필요가 없다.

[5-6] 가상 메모리와 디스크(Swap)의 관계

가상 메모리는 반드시 물리 RAM에만 매핑되는 것은 아니다. 리눅스는 스왑(Swap)이라는 개념을 통해 가상 메모리를 디스크 공간에도 매핑할 수 있다. 예를 들어, 물리 메모리가 부족한 상황에서도 더 큰 프로그램을 실행할 수 있는 이유가 여기에 있다. 메모리가 부족해지면, 운영체제는 당장 사용되지 않는 메모리 페이지를 디스크에 옮기고, 그 공간을 다른 프로그램을 위해 사용한다. 프로그램은 자신이 사용하는 메모리가 RAM인지 디스크인지 전혀 알지 못한다.

[5-7] 디스크에 있는 메모리에 접근할 때 발생하는 과정

x86 프로세서는 디스크에 직접 접근할 수 없기 때문에, 디스크에 매핑된 메모리에 접근하면 CPU는 이를 처리하지 못하고 운영체제에 제어를 넘긴다. 운영체제는 해당 메모리가 디스크에 있음을 확인한 후, 현재 메모리에 있는 다른 디스크로 옮겨 공간을 확보하고, 필요한 데이터를 디스크에서 RAM으로 다시 불러온다. 이후 가상 주소와 물리 주소의 매핑 정보를 갱신한 뒤, 프로그램을 중단된 명령어부터 다시 실행한다. 이 과정을 통해 프로그램은 아무 일도 없었던 것처럼 계속 실행한다.

예를 들어:

- RAM: 16MB
- 사용중: 8MB
- 필요한 프로그램 메모리: 20MB

-> 스왑이 있다면 실행 가능

필요한 메모리가 디스크에 있다면

- CPU 가 접근을 시도
- 접근 불가 -> 리눅스 호출
- 리눅스가 디스크에서 메모리를 RAM 으로 가져옴
- 주소 매핑 테이블 수정
- 프로그램 실행 재개

이 모든 과정은 프로그래머에게 투명하게 처리 된다.

[5-8] 페이지 단위 메모리 관리의 의미

리눅스에서 메모리는 페이지(page)단위로 관리되며, x86 시스템에서는 한 페이지가 4096 바이트이다. 메모리 할당, 매핑, 스와핑은 모두 페이지 단위로 이루어진다. 이 구조는 관리 효율을 높여주지만, 동시에 프로그래머에게 중요한 의미를 가진다. 메모리 접근이 넓은 범위에 흩어져 있으면 페이지 교체가 자주 발생하고, 이로 인해 성능이 급격히 저하될 수 있다. 따라서 메모리 접근의 지역성(locality)을 유지하는 것이 중요하다.

[5-9] 스왑 데스(Swap Death) 현상

시스템이 너무 많은 프로그램이 실행되면, 운영체제는 메모리 내용을 계속해서 RAM 과 디스크 사이에서 교환하게 된다. 이 상태에서는 CPU 가 실제 연산을 수행하기보다 스와핑 작업에 대부분의 시간을 소비하게 되며, 시스템이 거의 멈춘 것처럼 느껴진다. 이를 스왑 데스(swap death)라고 부른다. 일반적으로 메모리를 과도하게 사용하는 프로그램을 종료하면 회복할 수 있지만, 사용자 경험은 크게 저하된다.

[5-10] RSS(Resident Set Size)의 의미

RSS(Resident Set Size)는 현재 프로그램이 실제 물리 메모리(RAM)에 차지하고 있는 메모리 양을 의미한다. 리눅스에서 top 명령어를 통해 확인할 수 있으며, 가상 메모리 사용량과는 구분된다. RSS 값이 크다는 것은 프로그램이 현재 많은 데이터를 RAM 에 보유하고 있다는 뜻이며, 메모리 부족 상황에서는 중요한 성능 지표가 된다.

[6-1] 더 많은 메모리 얻기

우리는 리눅스가 모든 가상 메모리를 물리 메모리나 스왑 공간에 매핑한다는 것을 알고 있다. 아직 매핑되지 않은 가상 메모리에 접근하려고 하면 세그멘테이션 폴트(segmentation fault)라는 오류가 발생하며, 이는 프로그램을 종료시킨다.

앞서 배운 프로그램 브레이크 포인트(program break point)는 프로그램이 사용할 수 있는 마지막 유효 주소이다. 만약 프로그램이 필요로 하는 저장 공간의 크기를 미리 알고 있다면, 필요한 모든 메모리를 .data 나 .bss 섹션에 추가해 두면 된다. 그러면 프로그램 시작 시 그 메모리는 모두 준비되어 있다.

하지만 필요한 메모리의 크기를 미리 알 수 없는 경우도 많다. 예를 들어, 텍스트 에디터를 생각해 보자. 사용자가 편집할 파일의 길이는 실행 전에 알 수 없다. 최대 파일 크기를 정해두고 그 이상은 사용할 수 없다고 제한할 수도 있겠지만, 파일이 작을 경우 이는 메모리 낭비가 된다.

이러한 문제를 해결하기 위해 리눅스는 애플리케이션의 메모리 요구에 맞게 브레이크 포인트를 이동시킬 수 있는 기능을 제공한다.

더 많은 메모리가 필요할 경우, 리눅스에 새로운 브레이크 포인트의 위치를 지정하면 된다. 그러면 리눅스는 현재 브레이크 포인트와 새 브레이크 포인트 사이의 모든 메모리를 매핑하고, 브레이크 포인트를 지정한 위치로 옮긴다. 이렇게 확보된 메모리는 프로그램에서 사용할 수 있게 된다.

브레이크 포인트를 이동시키는 방법은 brk 시스템 콜을 사용하는 것이다.

- brk 시스템 콜의 번호는 45 번이며 %eax 에 넣는다.
- %ebx 에는 요청하는 새로운 브레이크 포인트 주소를 넣는다.
- 이후 int \$0x80 을 호출하여 리눅스에 요청한다.

메모리를 매핑한 후, 리눅스는 새로운 브레이크 포인트를 %eax 에 반환한다. 이 값은 요청한 값보다 클수도 있는데, 이는 리눅스가 페이지 단위로 올림(round up)하기 때문이다.

만약 요청을 충족할 만큼의 물리 메모리나 스왑 공간이 없다면, 리눅스는 %eax 에 0 을 반환한다. 또한, %ebx 에 0 을 넣고 brk 를 호출하면, 단순히 현재 사용 가능한 마지막 메모리 주소를 반환한다.

[6-2] 이 방식의 문제점

이 방법의 문제는 요청한 메모리를 직접 관리해야 한다는 점이다.

예를 들어,

- 파일 하나를 읽기 위해 브레이크 포인트를 늘리고
- 또 다른 파일을 위해 다시 브레이크 포인트를 늘린 뒤
- 첫 번째 파일을 더 이상 사용하지 않게 되었다고 하자

이 경우, 사용하지 않는 메모리 공간이 중간에 생기지만, 여전히 매핑된 상태로 남아 있게 된다. 이런 방식으로 계속 브레이크 포인트를 늘려가다 보면, 실제로는 사용하지 않는 공간이 많음에도 불구하고 메모리가 부족해질 수 있다.

이 문제를 해결하기 위해 필요한 것이 바로 메모리 관리자(memory manager)이다 .

[6-3] 메모리 관리자(Memory Manager)

메모리 관리자는 프로그램을 대신해 메모리 할당과 해제를 처리해주는 루틴들의 집합이다. 대부분의 메모리 관리자는 두 가지 기본 기능을 제공한다.

- allocate: 필요한 메모리 크기를 전달하면, 사용할 수 있는 메모리의 주소를 반환
- deallocate: 사용이 끝난 메모리를 반환하여 다시 사용할 수 있게 함

이러한 방식의 메모리 관리를 동적 메모리 할당(dynamic memory allocation)이라고 한다. 이 방식은 메모리 중간에 생기는 “구멍(hole)”을 최소화하여, 메모리를 최대한 효율적으로 사용할 수 있게 해준다. 메모리 관리자가 사용하는 메모리 풀(pool)은 보통 힙(heap)이라고 불린다.

[6-4] 메모리 관리자의 동작 방식

메모리 관리자는 다음과 같은 정보를 추적한다.

- 현재 시스템 브레이크 위치
- 이미 할당된 메모리 영역들

힙에 있는 각 메모리 블록을 사용 중인지, 사용하지 않는지 표시한다. 메모리를 요청하면, 적절한 크기의 미사용 블록이 있는지 확인한다.

- 있다면 -> 그 메모리를 반환
- 없다면 -> brk 시스템 콜을 호출하여 더 많은 메모리 요청

메모리를 해제하면, 해당 블록을 미사용 상태로 표시하여 이후 요청에서 재사용할 수 있도록 한다.

[7] 간단한 메모리 관리자

여기에서는 간단한 메모리 관리자를 하나 보여준다. 매우 원시적인 형태이지만, 원리를 이해하는 데에는 충분하다. 항상 그렇듯이, 먼저 프로그램 전체를 제시하고, 그 다음에 자세한 설명을 덧붙일 것이다. 코드가 길어 보이지만 대부분은 주석이다.

```
#목적(PURPOSE): 메모리 사용을 관리하는 프로그램
# 요청에 따라 메모리를 할당하고 해제한다
#
#비고(NOTES): 이 루틴을 사용하는 프로그램은
# 특정 크기의 메모리를 요청한다.
# 우리는 실제로 그보다 더 많은 공간을 사용하지만,
# 그 여분의 공간은 우리가 돌려주는 포인터 앞부분에 둔다.
# 즉, 크기 필드와 AVAILABLE / UNAVAILABLE 표시를
# 앞에 추가한다. 메모리 구조는 다음과 같다.
#
# #####
# #사용 가능 표시#메모리 크기#실제 메모리 공간#
# #####
# ^-- 반환되는 포인터는
# 여기 를 가리킨다
#
# 우리가 반환하는 포인터는 실제로 요청된 메모리 위치만
# 가리킨다. 이렇게 하면 호출하는 프로그램이 더 쉽게
# 사용할 수 있고, 내부 구조를 바꾸더라도 호출 측 코드는
# 변경할 필요가 없어진다.
```

```
.section .data
##### 전역 변수 #####
# 우리가 관리하는 메모리의 시작 위치
heap_begin:
.long 0

# 우리가 관리하는 메모리의 끝 바로 다음 위치
current_break:
.long 0
```

```
##### 구조 정보 #####
# 메모리 영역 헤더의 크기
.equ HEADER_SIZE, 8

# 헤더에서 "사용 가능" 플래그의 위치
.equ HDR_AVAIL_OFFSET, 0

# 헤더에서 크기 필드의 위치
.equ HDR_SIZE_OFFSET, 4
```

```
##### 상수 #####
.equ UNAVAILABLE, 0   # 이미 할당된 메모리를 표시
.equ AVAILABLE, 1     # 반환되어 다시 사용할 수 있는 메모리 표시
.equ SYS_BRK, 45      # brk 시스템 콜 번호
.equ LINUX_SYSCALL, 0x80
```

```
.section .text
```

```
##### 함수 #####
## allocate_init ##
# 목적:
# 이 함수는 메모리 관리 루틴을 초기화한다.
# (heap_begin 과 current_break 를 설정한다)
# 매개변수 없음, 반환값 없음

.globl allocate_init
.type allocate_init,@function
allocate_init:
    pushl %ebp
    movl %esp, %ebp

    # brk 시스템 콜을 %ebx=0 으로 호출하면
    # 현재 사용 가능한 마지막 주소를 반환한다
    movl $SYS_BRK, %eax
    movl $0, %ebx
    int $LINUX_SYSCALL

    movl %ebp, %esp
    popl %ebp
```

```

## allocate ##
# 목적:
# 메모리를 할당하는 함수
# 사용 가능한 블록이 있으면 그것을 사용하고,
# 없으면 Linux 에 새 메모리를 요청한다
#
# 매개변수:
# 요청할 메모리 크기 (1 개)
#
# 반환값:
# 할당된 메모리 주소 (%eax)
# 실패 시 %eax = 0

# 사용 레지스터:
# %ecx : 요청된 메모리 크기
# %eax : 현재 검사 중인 메모리 영역
# %ebx : 현재 break 위치
# %edx : 현재 메모리 영역 크기

.globl allocate
.type allocate,@function
.equ ST_MEM_SIZE, 8

allocate:
    pushl %ebp
    movl %esp, %ebp
    movl ST_MEM_SIZE(%ebp), %ecx

    movl heap_begin, %eax
    movl current_break, %ebx

alloc_loop_begin:
    # 모든 메모리 영역을 순회
    cmpl %ebx, %eax
    je move_break

    movl HDR_SIZE_OFFSET(%eax), %edx

    cmpl $UNAVAILABLE, HDR_AVAIL_OFFSET(%eax)

```

```
je next_location
```

```
cmpl %edx, %ecx
```

```
jle allocate_here
```

```
next_location:
```

```
addl $HEADER_SIZE, %eax
```

```
addl %edx, %eax
```

```
jmp alloc_loop_begin
```

```
allocate_here:
```

```
movl $UNAVAILABLE, HDR_AVAIL_OFFSET(%eax)
```

```
addl $HEADER_SIZE, %eax
```

```
movl %ebp, %esp
```

```
popl %ebp
```

```
ret
```

```
move_break:
```

```
addl $HEADER_SIZE, %ebx
```

```
addl %ecx, %ebx
```

```
pushl %eax
```

```
pushl %ecx
```

```
pushl %ebx
```

```
movl $SYS_BRK, %eax
```

```
int $LINUX_SYSCALL
```

```
cmpl $0, %eax
```

```
je error
```

```
popl %ebx
```

```
popl %ecx
```

```
popl %eax
```

```
movl $UNAVAILABLE, HDR_AVAIL_OFFSET(%eax)
```

```
movl %ecx, HDR_SIZE_OFFSET(%eax)
```

```
addl $HEADER_SIZE, %eax
```

```
movl %ebx, current_break
```

```

movl $UNAVAILABLE, HDR_AVAIL_OFFSET(%eax)
movl %ecx, HDR_SIZE_OFFSET(%eax)
addl $HEADER_SIZE, %eax
movl %ebx, current_break

movl %ebp, %esp
popl %ebp
ret

```

error:

```

movl $0, %eax
movl %ebp, %esp
popl %ebp
ret

```

deallocate

목적:

사용이 끝난 메모리를 다시 풀(pool)에 반환

#

매개변수:

반환할 메모리 주소

#

반환값:

없음

.globl deallocate

.type deallocate,@function

.equ ST_MEMORY_SEG, 4

deallocate:

```

movl ST_MEMORY_SEG(%esp), %eax
subl $HEADER_SIZE, %eax
movl $AVAILABLE, HDR_AVAIL_OFFSET(%eax)
ret

```

가장 먼저 눈에 띄는 점은 `_start` 심볼이 없다는 것이다. 이유는 이 코드는 하나의 완전한 프로그램이 아니라, 함수들의 집합이기 때문이다. 메모리 관리자는 단독으로 실행되는 프로그램이 아니라, 다른 프로그램에서 사용되는 유틸리티이다. 어셈블은 다음과 같이 수행한다.

```
as alloc.s -o alloc.o
```

[7-1] 변수와 상수(Variables and Constans)

프로그램의 시작 부분에는 두 개의 위치가 설정되어 있다.

```
heap_begin:
.long 0
current_break:
.long 0
```

기억하자. 우리가 관리하는 메모리 영역은 일반적으로 힙(heap)이라고 불린다. 프로그램을 어셈블할 때는 힙의 시작 위치가 어디인지 모르고, 현재 break 가 어디인지 모른다. 따라서 우리는 이 주소들을 저장할 공간만 확보해두고, 일단 0 으로 채워둔다.

다음으로 힙의 구조를 정의하기 위한 상수 집합이 있다. 이 메모리 관리자는 각 메모리 영역 앞에 해당 메모리를 설명하는 짧은 기록(record)을 둔다. 이 기록에는 사용 가능 여부를 나타내는 워드 하나와 영역의 크기를 나타내는 워드 하나가 포함된다. 실제로 할당된 메모리는 이 기록 바로 뒤에 위치한다.

사용 가능 플래그는 이 메모리 영역이 할당인지, 아니면 현재 사용 중인지 표시하는 데 사용된다. 크기 필드는 이 영역이 할당 요청을 처리할 수 있을 만큼 충분히 큰지 여부를 알려줄 뿐만 아니라, 다음 메모리 영역의 위치를 계산하는 데에도 사용된다. 다음 상수들은 이 기록을 설명한다.

```
.equ HEADER_SIZE, 8
.equ HDR_AVAIL_OFFSET, 0
.equ HDR_SIZE_OFFSET, 4
```

이는 헤더 전체가 8 바이트이며, 사용 가능 플래그는 시작 위치에서 0 바이트 오프셋에 있고, 크기 필드는 시작 위치에서 4 바이트 오프셋에 있다는 뜻이다. 우리가 항상 이 상수들을 사용하도록 주의한다면, 나중에 헤더에 더 많은 정보를 추가하더라도 많은 수정 작업을 하지 않아도 된다.

사용 가능 필드에 사용할 값은 사용 불가 일 때 0, 사용 가능일 때 1 이다. 읽기 쉽게 하기 위해 다음과 같이 정의한다.

```
.equ UNAVAILABLE, 0
.equ AVAILABLE, 1
```

마지막으로 Linux 시스템 콜 정의가 있다.

```
.equ BRK, 45
.equ LINUX_SYSCALL, 0x80
```

[7-2] allocate_init 함수

이 함수는 매우 단순하다. 앞에서 설명한 heap_begin 과 current_break 변수를 설정하는 역할만 한다. 앞에서 설명했듯이, 현재 break 는 brk 시스템 콜을 사용해 얻을 수 있다. 따라서 함수는 다음과 같이 시작한다.

```
pushl %ebp
movl %esp, %ebp
movl $SYS_BRK, %eax
movl $0, %ebx
int $LINUX_SYSCALL
```

int \$LINUX_SYSCALL 이후에 %eax 에는 마지막 유효 주소가 들어 있다. 하지만 우리는 마지막 유효 주소가 아니라 첫 번째 유효하지 않은 주소가 필요하므로 %eax 를 1 증가시킨다. 그런 다음 그 값을 heap_begin 과 current_break 에 저장하고 함수에서 빠져나온다. 코드는 다음과 같다.

```
incl %eax
movl %eax, current_break
movl %eax, heap_begin
movl %ebp, %esp
popl %ebp
ret
```

힙은 heap_begin 과 current_break 사이의 메모리 영역이므로, 이는 처음에는 힙의 크기가 0 바이트임을 의미한다. 이후 allocate 함수가 호출될 때 필요에 따라 힙을 확장하게 된다.

[7-3] allocate 함수

이 함수가 핵심이다. 먼저 함수의 전체 흐름을 살펴본다.

1. 힙의 시작 위치에서 시작한다.
2. 힙의 끝에 도달했는지 확인한다.
3. 힙의 끝이라면 Linux 로부터 필요한 메모리를 요청하고, 이를 “사용 불가”로 표시한 뒤 반환한다. Linux 가 더 이상 메모리를 주지 않으면 0 으로 반환한다.
4. 현재 메모리 영역이 “사용 불가”라면 다음 영역으로 이동하고 2 단계로 돌아간다.
5. 현재 메모리 영역이 요청된 크기보다 작으면 2 단계로 돌아간다.
6. 메모리 영역이 사용 가능하고 충분히 크면 “사용 불가”로 표시하고 반환한다.

이 개요를 옆두에 두고 코드 전체를 다시 살펴보자. 주석을 읽으면 각 레지스터에 어떤 값이 들어 있는지 알 수 있다.

이제 코드를 한 줄씩 살펴본다. 시작 부분은 다음과 같다.

```
pushl %ebp
movl %esp, %ebp
movl ST_MEM_SIZE(%ebp), %ecx
movl heap_begin, %eax
movl current_break, %ebx
```

앞의 두 줄은 표준적인 함수 프롤로그이다. 다음 줄은 스택에서 할당된 메모리 크기를 가져온다. 이것이 함수의 유일한 매개변수이다. 그 다음 힙의 시작 주소와 끝 주소를 레지스터로 옮긴다. 이제 처리를 시작할 준비가 되었다.

다음 부분은 alloc_loop_begin 으로 표시된 루프이다. 이 루프에서는 사용 가능한 메모리 영역을 찾거나, 더 많은 메모리가 필요하다는 것을 알 때까지 메모리 영역들을 검사한다. 첫 번째로 하는 일은 더 많은 메모리가 필요한지 확인하는 것이다.

```
cmpl %ebx, %eax
je move_break
```

%eax 에는 현재 검사 중인 메모리 영역의 주소가 있고, %ebx 에는 힙의 다음 주소가 있다. 따라서 검사할 다음 영역이 힙의 끝을 넘어섰다면, 이 크기의 메모리 영역을 할당하려면 더 많은 메모리가 필요하다는 뜻이다. 이제 move_break 로 이동해서 무슨 일이 일어나는지 살펴보자.

```
move_break:
addl $HEADER_SIZE, %ebx
addl %ecx, %ebx
```


이 시점에서 %ebx 는 우리가 다음 메모리 영역을 두고 싶은 위치를 가지고 있다. 헤더 크기와 영역 크기를 %ebx 에 더한 값이 우리가 시스템을 break 를 설정하고 싶은 위치이다. 그런 다음 저장해야 할 레지스터들을 스택에 푸시하고 brk 시스템 콜을 호출한다.

```
pushl %eax
pushl %ecx
pushl %ebx
movl $SYS_BRK, %eax
int $LINUX_SYSCALL
```

그 다음 오류가 발생했는지 확인한다.

```
cmpl $0, %eax
je error
```

오류가 없었다면 레지스터를 다시 스택에서 꺼내고, 메모리를 사용 불가로 표시하고, 메모리 크기를 기록하며, %eax 가 실제로 사용 가능한 메모리의 시작을 가리키도록 한다(헤더 뒤):

```
popl %ebx
popl %ecx
popl %eax
movl $UNAVAILABLE, HDR_AVAIL_OFFSET(%eax)
movl %ecx, HDR_SIZE_OFFSET(%eax)
addl $HEADER_SIZE, %eax
```

그 다음 새 program break 를 저장하고 할당된 메모리의 포인터를 반환한다.

```
movl %ebx, current_break
movl %ebp, %esp
popl %ebp
ret
```

error 코드는 단순히 %eax 에 0 을 반환하므로 여기서는 따로 설명하지 않는다.

이제 루프의 나머지 부분을 본다. 현재 검사 중인 메모리가 힙의 끝이 아니라면 어떻게 될까?

```
movl HDR_SIZE_OFFSET(%eax), %edx
cmpl $UNAVAILABLE, HDR_AVAIL_OFFSET(%eax)
je next_location
```

먼저 메모리 영역의 크기를 %edx 에 넣는다. 그 다음 사용 가능 플래그가 UNAVAILABLE 인지 확인한다. 만약 그렇다면, 이 메모리 영역은 이미 사용 중이므로 건너뛰어야 한다. 따라서 next_location 으로 이동한다. 사용 가능 플래그가 AVAILABLE 이면 계속 진행한다.

공간이 사용 가능하다고 가정하자. 이제 이 공간이 요청된 크기를 담을 만큼 충분히 큰지 확인한다. 현재 영역의 크기는 %edx 에 있으므로 다음과 같이 비교한다.

```
cmpl %edx, %ecx
jle allocate_here
```

요청된 크기가 현재 영역 크기보다 작거나 같으면 이 블록을 사용할 수 있다. 현재 영역이 요청보다 더 크더라도 상관없다. 남은 공간은 그냥 사용되지 않을 뿐이다. 이제 allocate_here 로 가서 무슨 일이 일어나는지 보자

```
movl $UNAVAILABLE, HDR_AVAIL_OFFSET(%eax)
addl $HEADER_SIZE, %eax
movl %ebp, %esp
popl %ebp
ret
```

이 코드는 메모리를 사용 불가로 표시한 뒤, %eax 를 헤더 뒤로 이동시켜 함수의 반환값으로 사용한다. 이 함수를 사용하는 쪽은 메모리 헤더의 존재를 전혀 알 필요가 없다. 단지 사용 가능한 메모리의 포인터만 받으면 된다.

그렇다면 영역이 충분히 크지 않다면 어떻게 될까? 그 경우 next_location 으로 이동한다. 이 코드는 현재 메모리 영역이 할당에 적합하지 않다고 판단될 때마다 실행된다. 하는 일은 %eax 를 다음 메모리 영역으로 이동시키고 루프의 처음으로 돌아가는 것이다. %edx 에는 현재 영역의 크기가 들어 있고, HEADER_SIZE 는 헤더 크기이다. 따라서 다음 코드는 다음 메모리 영역으로 이동한다.

```
addl $HEADER_SIZE, %eax
addl %edx, %eax
jmp alloc_loop_begin
```

이제 함수는 다시 루프를 한 번 더 실행한다.

루프가 있다면 반드시 끝나야 한다. 가장 좋은 방법은 모든 경우를 살펴보고, 그 모든 경우가 결국 루프 종료 조건으로 이어지는지 확인하는 것이다. 이 경우 가능한 상황은 다음과 같다.

- 힙의 끝에 도달한다.
- 사용 가능하고 충분히 큰 메모리 영역을 찾는다.
- 다음 위치로 이동한다.

앞의 두 경우는 루프를 종료시키는 조건이다. 세 번째 경우는 루프를 계속 돌게 한다. 그러나 설령 사용 가능한 영역을 차지 못하더라도, 힙은 유한한 크기이므로 결국 힙의 끝에 도달하게 된다. 따라서 어떤 경우든 결국 루프는 종료 조건을 만나게 된다.

[7-4] deallocate 함수

deallocate 함수는 allocate 함수보다 훨씬 간단하다. 이 함수는 단순히 메모리 영역을 AVAILABLE 로 표시하기만 하면 되고, 다음에 allocate 가 호출되면 이 영역을 다시 찾게 된다. 코드는 다음과 같다.

```
movl ST_MEMORY_SEG(%esp), %eax
subl $HEADER_SIZE, %eax
movl $AVAILABLE, HDR_AVAIL_OFFSET(%eax)
ret
```

이 함수에서 %ebp 나 %esp 를 저장할 필요가 없다. 값을 변경하지 않기 때문이다. 또한 마지막에 복원할 필요도 없다. 스택에서 메모리 영역의 주소를 읽어 온 뒤, 헤더의 시작 위치로 되돌아가 해당 영역을 사용 가능으로 표시하기만 하면 된다. 이 함수는 반환값이 없으므로 %eax 에 무엇이 남아 있든 상관 없다.

[7-5] 성능 문제 및 기타 문제점

이 단순한 메모리 관리자는 학습 목적 외에는 거의 쓸모가 없다. 이 절에서는 이러한 단순한 할당기의 문제점을 살펴본다.

가장 큰 문제는 속도이다. 할당이 몇 번 안된다면 큰 문제가 없겠지만, 만약 1000 번의 할당이 이루어진다면 상황이 달라진다. 1000 번째 할당에서는 999 개이 메모리 영역을 검사한 뒤에야 더 많은 메모리를 요청해야 한다는 사실을 알게 된다. 이는 매우 느리다.

또한 Linux 는 메모리 페이지를 실제 메모리 대신 디스크에 저장할 수도 있다. 그러므로 프로그램의 모든 메모리 영역을 검사해야 한다면, Linux 는 디스크에 있던 메모리를 다시 불러와 사용 가능 여부를 확인해야 한다. 이것이 얼마나 느려질 수 있는지 쉽게 알 수 있다.

이 방식은 선형 시간(linear time)으로 동작한다고 말한다. 즉, 관리해야 할 요소가 늘어날수록 실행 시간이 늘어난다.

반면 상수 시간(constant time)으로 동작하는 프로그램은 관리하는 요소 수와 상관없이 항상 같은 시간이 걸린다. 예를 들어, `deallocate` 함수는 관리 중인 요소 수나 메모리 위치와 관계없이 항상 4 개의 명령어만 실행한다. 실제로 우리의 `allocate` 함수는 가장 느린 메모리 관리자 중 하나이지만, `deallocate` 함수는 가장 빠른 측에 속한다.

또 다른 성능 문제는 `brk` 시스템 콜을 너무 자주 호출한다는 점이다. 시스템 콜은 일반 함수 호출보다 훨씬 느리다. 프로세서는 사용자 모드에서 커널 모드로 전환해야 하고, Linux 커널이 메모리를 매핑한 뒤 다시 사용자 모드로 돌아와야 한다. 이를 컨텍스트 스위치라고 하며, x86 프로세서에서는 특히 느리다. 일반적으로 정말 필요하지 않다면 커널 호출은 피하는 것이 좋다.

또 다른 문제는 Linux 가 실제로 `break` 를 어디에 설정했는지를 기록하지 않는다는 점이다. Linux 는 우리가 요청한 위치보다 더 뒤쪽에 `break` 를 설정할 수도 있다. 이 프로그램에서는 Linux 가 실제로 설정한 `break` 를 전혀 확인하지 않고, 우리가 요청한 위치에 설정되었다고 가정한다. 이는 엄밀히 말해 버그는 아니지만, 이미 매핑된 메모리가 있음에도 불필요하게 `brk` 시스템 콜을 호출하게 만든다.

또 하나의 문제는, 예를 들어, 5 바이트 메모리를 요청했는데 처음 발견한 사용 가능한 영역이 1000 바이트라면, 전체 1000 바이트를 모두 할당해 버린다는 점이다. 이로 인해 995 바이트가 사용되지 않지만 할당된 상태로 남게 된다. 이런 경우에는 메모리 영역을 분할해서 나머지 공간을 나중에 사용할 수 있도록 하는 것이 바람직하다. 또한 큰 할당을 위해 연속된 빈 공간을 합치는 기능도 있으면 좋을 것이다.

[7-6] 할당기 사용하기

이 책에서 다루는 프로그램들은 메모리 관리자가 꼭 필요할 정도로 복잡하지 않다. 따라서 우리는 메모리 관리자를 단지 파일 입출력 프로그램 중 하나에서 버퍼를 할당하는 데 사용해 보겠다. `.bss` 에 버퍼를 두는 대신, 동적으로 할당된 버퍼를 사용한다.

우리가 시연할 프로그램은 6 장의 `read-records.s` 이다. 이 프로그램은 입출력을 처리하기 위해 `record_buffer` 라는 버퍼를 사용한다. 우리는 이것을 `.bss` 에 정의된 버퍼에서, 메모리 관리자를 통해 동적으로 할당되는 버퍼 포인터로 변경할 것이다. 이 절에서는 변경된 부분만 설명하므로 해당 프로그램의 기존 코드를 준비해 두어야 한다.

첫 번째 변경은 선언 부분이다. 기존에는 다음과 같았다.

```
.section .bss
.lcomm record_buffer, RECORD_SIZE
```

이제 실제 버퍼가 아니라 버퍼에 대한 포인터가 되므로 같은 이름을 쓰는 것은 부적절하다. 또한 이제는 포인터 하나만 저장하면 되므로 한 워드만 필요하다. 새 선언은 .data 섹션에 다음과 같이 둔다.

```
record_buffer_ptr:
.long 0
```

다음으로 프로그램 시작 직후 메모리 관리자를 초기화해야 한다. 따라서 스택이 설정된 직후에 다음 호출을 추가한다.

```
call allocate_init
```

이제 메모리 관리자는 메모리 할당 요청을 처리할 준비가 되었다. 레코드를 저장할 충분한 메모리를 할당해야 하므로 allocate 를 호출하고, 반환된 포인터를 record_buffer_ptr 에 저장한다.

```
pushl $RECORD_SIZE
call allocate
movl %eax, record_buffer_ptr
```

이제 read_record 를 호출할 때는 포인터가 필요하다. 이전 코드에서는 \$record_buffer 라는 즉시 값 주소를 사용했다. 하지만 이제 record_buffer_ptr 는 버퍼 자체가 아니라 포인터를 담고 있다.

따라서 다음 줄을 제거해야 한다.

```
pushl $record_buffer
```

그리고 대신 다음 줄을 사용해야 한다.

```
pushl record_buffer_ptr
```

다음 변경은 레코드의 firstname 필드 주소를 구할 때이다. 기존 코드에서는 \$RECORD_FIRSTNAME + record_buffer 를 사용했다. 이는 상수 주소에 대한 상수 오프셋이기 때문에 가능했다. 하지만 이제는 record_buffer_ptr 에 저장된 주소에 대한 오프셋이다. 따라서 포인터를 레지스터로 옮긴 뒤 오프셋을 더해야 한다. 기존 코드는 다음과 같았다면

```
pushl $RECORD_FIRSTNAME + record_buffer
```

이를 다음과 같이 바꿔야 한다.

```
movl record_buffer_ptr, %eax
addl $RECORD_FIRSTNAME, %eax
pushl %eax
```

마찬가지로 다음 코드도

```
movl $RECORD_FIRSTNAME + record_buffer, %ecx
```

다음과 같이 변경해야 한다.

```
movl record_buffer_ptr, %ecx
addl $RECORD_FIRSTNAME, %ecx
```

마지막으로 프로그램이 끝날 때 메모리를 해제해야 한다(이 프로그램에서는 꼭 필요하지는 않지만 좋은 습관이다.). 종료 직전에 record_buffer_ptr 를 deallocate 함수에 넘긴다.

```
pushl record_buffer_ptr
call deallocate
```

이제 다음 명령으로 프로그램을 빌드할 수 있다.

```
as read-records.s -o read-records.o
ld alloc.o read-record.o read-records.o write-newline.o count-chars.o -o read-records
```

그 다음 ./read-records 로 프로그램을 실행하면 된다.

동적 메모리 할당의 용도가 지금 당장은 분명하지 않을 수도 있지만, 학습용 예제를 넘어 실제 프로그램을 작성하게 되면 지속적으로 사용하게 될 것이다.

문제

[1] 개념 이해하기

- {1} Linux 프로그램이 시작될 때 메모리의 배치는 어떻게 구성되는가
- {2} 힙(heap)이란 무엇인가
- {3} 현재 break(current break)란 무엇인가
- {4} 스택은 어느 방향으로 성장하는가
- {5} 힙은 어느 방향으로 성장하는가
- {6} 매핑되지 않은 메모리에 접근하면 어떤 일이 발생하는가
- {7} 운영체제는 프로세스들이 서로의 메모리를 덮어쓰지 못하도록 어떻게 방지하는가
- {8} 사용 중인 메모리의 일부가 현재 디스크에 존재할 경우 어떤 과정이 발생하는지 설명하라

[2] 개념 사용하기

- {1} 메모리 관리자가 아직 초기화되지 않았을 경우, 자동으로 `allocate_init` 을 호출하도록 메모리 관리자를 수정하라
- {2} 요청된 메모리 크기가 선택된 메모리 영역보다 작을 경우 해당 영역을 여러 부분으로 나누도록 메모리 관리자를 수정하라. 이때 새로운 헤더 레코드의 크기도 반드시 고려해야 한다.
- {3} 버퍼를 사용하는 프로그램 중 하나를 수정하여, `.bss` 를 사용하는 대신 메모리 관리자를 통해 버퍼 메모리를 할당하도록 하라.

[3] 심화

- {1} 가비지 컬렉션(garbage collection)을 조사하라. 이것이 여기에서 사용한 메모리 관리 방식과 비교했을 때 가지는 장점과 단점은 무엇인가
- {2} 참조 카운팅(reference counting)을 조사하라. 이것이 여기에서 사용한 메모리 관리 방식과 비교했을 때 가지는 장점과 단점은 무엇인가
- {3} 함수 이름을 `malloc` 과 `free` 로 변경하고, 이를 공유 라이브러리로 만들어라. `LD_PRELOAD` 를 사용하여 기본 메모리 관리자 대신 이 메모리 관리자가 사용되도록 강제하라. 기본 메모리 관리자 대신 당신의 메모리 관리자가 사용되고 있음을 확인하기 위해 `write` 시스템 콜을 사용해 `STDOUT` 에 추가하라.

10 장 컴퓨터처럼 세기

[1] 세기(Counting)

[1-1] 인간처럼 세기(Counting Like a Human)

여러 면에서 컴퓨터는 인간과 비슷한 방식으로 센다. 따라서 컴퓨터가 어떻게 세는지를 배우기 전에, 먼저 우리가 어떻게 세는지를 좀 더 깊이 살펴본다.

인간은 (보통) 손가락이 열 개이다. 이것이 중요한 이유는 우리의 숫자 체계를 보면 된다. 한 자리 숫자가 두 자리 숫자가 되는 시점은 10 이다.

인간은 10 진법을 사용해 세고 계산한다. 10 진법이란 모든 것을 10 씩 묶는다는 뜻이다. 예를 들어, 양을 센다고 한다. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10.

왜 갑자기 두 자리가 되었고, 다시 1 을 쓰게 되었을까? 이는 숫자를 10 단위로 묶기 때문이며, 양이 10 마리로 한 묶음이 되었기 때문이다.

그 다음 숫자인 11 을 보면, 이는 10 마리짜리 묶음이 1 개 있고, 묶이지 않은 양이 1 마리 있다는 뜻이다. 그래서 계속해서 12, 13, 14, 15, 16, 17, 18, 19, 20 으로 간다. 이제는 10 마리 묶음이 2 개이다. 21 은 10 마리 묶음이 2 개, 남은 양이 1 마리라는 뜻이고, 22 는 2 개의 묶음에 남은 양 2 마리라는 뜻이다.

이렇게 계속 세다 보면 97, 98, 99, 그리고 100 에 도달한다. 또 같은 일이 발생한다. 100 에서는 무엇이 일어날까? 이제는 10 마리 묶음이 10 개가 된다. 101 은 10 마리 묶음이 10 개와 남은 양 1 마리이다.

따라서 어떤 숫자든 이런식으로 해석할 수 있다. 예를 들어, 60879 마리의 양을 셸다면, 이는
10 의 네제곱 묶음 6 개,
10 의 세제곱 묶음 0 개,
10 의 제곱 묶음 8 개,
10 의 일제곱 묶음 7 개,
그리고 남은 양 9 마리를 의미한다.

그렇다면 10 씩 묶는 것이 특별한 의미가 있을까? 아니다. 단지 우리가 손가락이 10 개이기 때문에 그렇게 해온 것 뿐이다. 9 나 11 로 묶을 수도 있었을 것이다.(그 경우에는 새로운 숫자 기호가 필요했을 것이다.)

숫자를 묶는 방식의 차이는 곱셈, 덧셈, 뺄셈, 나눗셈 표를 다시 배워야 한다는 점뿐이다. 규칙 자체는 변하지 않고, 표현 방식만 달라진다. 또한 우리가 능숙하게 사용하던 몇몇 요령도 더 이상 적용되지 않을 수도 있다.

예를 들어, 10 진법 대신 9 진법을 사용한다고 가정한다. 소수점을 오른쪽으로 한 자리 이동하는 것이 더 이상 10 을 곱하는 것이 아니라 9 를 곱하는 것이 된다. 9 진법에서 500 은 50 의 9 배일 뿐이다.

[1-2] 컴퓨터처럼 세기(Counting Like a Computer)

그렇다면 컴퓨터는 몇 개의 손가락으로 셀까? 컴퓨터는 단 두 개의 손가락만 가지고 있다. 따라서 모든 묶음은 2 개씩 묶인다.

이제 이진법으로 세어본다.

0(영),
1(일),
10(이 - 2 의 묶음 1 개),
11(삼 - 2 의 묶음 1 개와 남은 1),
100(사 - 2 의 묶음 2 개),
101(오 - 2 의 묶음 2 개와 남은 1),
110(육 - 2 의 묶음 3 개)

이와 같은 방식이다.

2 진법에서는 소수점을 오른쪽으로 한 자리 이동하면 2 를 곱하는 것이고, 왼쪽으로 이동하면 2 로 나누는 것이다. 2 진법은 binary(이진법)라고도 부른다.

2 진법의 장점은 기본적인 계산 표가 매우 단순하다는 것이다. 10 진법에서는 곱셈표가 가로 10 칸, 세로 10 칸이지만, 2 진법에서는 매우 간단하다.

이진 덧셈 표

```
+ | 0 | 1
--+---+---
0 | 0 | 1
--+---+---
1 | 1 | 10
```

이진 곱셈 표

```
* | 0 | 1
--+---+---
0 | 0 | 0
--+---+---
1 | 0 | 1
```

이제 숫자 10010101 과 1100101 을 더한다.

10010101
+ 1100101

11111010

이제 곱한다.

10010101
* 1100101

10010101
00000000
10010101
00000000
00000000
10010101
10010101

11101011001001

[2] 이진수와 십진수 사이의 변환

이제 숫자를 이진수(2 진법)에서 십진수(10 진법)로 변환하는 방법을 배워보자. 이는 실제로 꽤 간단한 과정이다. 앞에서 배운 것처럼, 각 자릿수는 2 의 어떤 거듭제곱 단위를 의미한다. 따라서 각 자릿수가 나타내는 값을 모두 더하면 십진수가 된다.

이진수 10010101 을 예로 들어보자. 이것을 십진수로 바꾸기 위해 다음과 같이 분해한다.

1	0	0	1	0	1	0	1
							1의 자리 (2^0)
						0개의 2 (2^1)	
					1개의 4 (2^2)		
				0개의 8 (2^3)			
			1개의 16 (2^4)				
		0개의 32 (2^5)					
	0개의 64 (2^6)						
1개의 128 (2^7)							

이제 각각을 더하면 다음과 같다.

$1 \times 128 + 0 \times 64 + 0 \times 32 + 1 \times 16 + 0 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 =$ $128 + 16 + 4 + 1 = 149$

따라서 이진수 10010101 은 십진수 149 이다.

이제 1100101 을 보자. 다음과 같이 쓸 수 있다.

$1 \times 64 + 1 \times 32 + 0 \times 16 + 0 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 =$ $64 + 32 + 4 + 1 = 101$

즉, 이진수 1100101 은 십진수 101 이다.

이제 하나 더 살펴보자. 11101011001001 을 십진수로 변환하면 다음과 같다.

$1 \times 8192 + 1 \times 4096 + 1 \times 2048 + 0 \times 1024 + 1 \times 512 + 0 \times 256 + 1 \times 128 + 1 \times 64 + 0 \times 32 + 0 \times 16 +$ $1 \times 8 + 0 \times 4 + 0 \times 2 + 1 \times 1 =$ $8192 + 4096 + 2048 + 512 + 128 + 64 + 8 + 1 = 15049$
--

앞에서 곱셈 예제로 사용했던 숫자들이라는 점을 눈치챘을 것이다. 결과를 확인해보면,

$101 \times 149 = 15049$

이제 십진수에서 이진수로 변환하는 방법을 살펴보자. 이 변환을 위해 숫자를 2 로 계속 나누어야 한다. 예를 들어, 숫자 17 을 생각해 보자.

17 을 2 로 나누면 몫은 8 이고 나머지는 1 이다. 이는 오른쪽 끝 자릿수가 1 이라는 뜻이다.

이제 남은 8 을 다시 2 로 나눈다.

- $8 \div 2 = 4$, 나머지 0 → 다음 자릿수는 0
- $4 \div 2 = 2$, 나머지 0 → 다음 자릿수는 0
- $2 \div 2 = 1$, 나머지 0 → 다음 자릿수는 0
- $1 \div 2 = 0$, 나머지 1 → 다음 자릿수는 1

더 이상 나눌 수 없으므로 종료한다. 결과적으로 얻어진 이진수는 10001 이다.

앞에서 이진수 11101011001001 을 십진수 15049 로 변환했다. 이제 반대로 다시 변환해보자.

15049 / 2 = 7524	나머지 1
7524 / 2 = 3762	나머지 0
3762 / 2 = 1881	나머지 0
1881 / 2 = 940	나머지 1
940 / 2 = 470	나머지 0
470 / 2 = 235	나머지 0
235 / 2 = 117	나머지 1
117 / 2 = 58	나머지 1
...	

이 나머지들은 아래에서 위로 일으면 원래의 숫자인 11101011001001 이 된다. 첫 번째 나머지가 가장 오른쪽 비트라는 점을 기억하자.

이진수의 각 자릿수는 비트(bit)라고 하며, 이는 binary digit 의 줄임말이다. 컴퓨터는 메모리를 바이트(byte)라는 저장 단위로 나눈다. x86 프로세서(그리고 대부분의 프로세서)에서 하나의 저장 위치는 8 비트이다.

앞에서 바이트 하나에 0 부터 255 까지의 숫자를 저장할 수 있다고 했는데, 그 이유는 8 비트로 표현할 수 있는 가장 큰 숫자가 255 이기 때문이다.

이를 직접 확인해보면 다음과 같다.

11111111 =
$(1 \times 2^7) + (1 \times 2^6) + (1 \times 2^5) + (1 \times 2^4)$
$+ (1 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) =$
$128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$

16 비트로 표현할 수 있는 최대 숫자는 65535 이고,
32 비트는 4,294,967,295(약 40 억),
64 비트는 18,446,744,073,709,551,615,

128 비트는 340,282,366,920,938,463,463,374,607,431,768,211,456 이다.

x86 프로세서에서는 대부분 4 바이트(32 비트) 숫자를 다루게 되는데, 이는 레지스터의 크기가 그만큼이기 때문이다.

[3] 참, 거짓, 그리고 이진수

이제 컴퓨터가 모든 것을 1 과 0 의 연속으로 저장한다는 것을 보았다. 이제 이것의 다른 활용을 살펴보자. 비트들의 나열을 숫자로 보는 대신, 스위치들의 집합으로 본다면 어떨까?

예를 들어, 집 안의 조명을 제어하는 네 개의 스위치가 있다고 가정해 보자. 바깥 조명, 복도 조명, 거실 조명, 침실 조명을 위한 스위치가 있다. 어떤 조명이 켜져 있고 꺼져 있는지를 다음과 같은 표로 나타낼 수 있다.

바깥	복도	거실	침실
켜짐	꺼짐	켜짐	켜짐

이 표를 보면 복도 조명만 꺼져 있고 나머지는 모두 켜져 있다는 것을 쉽게 알 수 있다. 이제 “켜짐(on)”과 “꺼짐(Off)” 대신 숫자 1 과 0 을 사용해 보자. 1 은 켜짐, 0 은 꺼짐을 의미하도록 한다. 같은 정보는 다음과 같이 표현할 수 있다.

바깥	복도	거실	침실
1	0	1	1

이제 각 스위치가 어떤 위치에 해당하는지 외워두기만 한다면, 같은 정보는 다음과 같이 표현할 수 있다.

1 0 1 1

이것은 컴퓨터의 저장 공간을 숫자 이상의 의미로 사용하는 여러 방법 중 하나일 뿐이다. 컴퓨터의 메모리는 단지 숫자만 볼 뿐이지만, 프로그래머는 이 숫자를 사용해 상상할 수 있는 어떤 것이든 표현할 수 있다. 다만 가장 적절한 표현 방식을 찾기 위해 창의적으로 생각해야 할 때도 있다.

이진수는 일반적인 연산 뿐만 아니라, 이진 연산(논리 연산)이라고 불리는 고유한 연산들도 가지고 있다. 표준 이진 연산은 다음과 같다.

- AND
- OR
- NOT
- XOR

예제를 보기 전에 각각을 설명하겠다.

AND 는 두 개의 비트를 받아 하나의 비트를 반환한다. 두 비트가 모두 1 일 때만 1 을 반환하고, 그 외에는 0 을 반환한다.

- $1 \text{ AND } 1 = 1$
- $1 \text{ AND } 0 = 0$
- $0 \text{ AND } 1 = 0$
- $0 \text{ AND } 0 = 0$

OR 는 두 개의 비트를 받아 하나의 비트를 반환한다. 둘 중 하나라도 1 이면 1 을 반환한다.

- $1 \text{ OR } 1 = 1$
- $1 \text{ OR } 0 = 1$
- $0 \text{ OR } 1 = 1$
- $0 \text{ OR } 0 = 0$

NOT 은 하나의 비트만을 받아 반대값을 반환한다.

- $\text{NOT } 1 = 0$
- $\text{NOT } 0 = 1$

마지막으로 XOR 은 OR 과 비슷하지만, 두 비트가 모두 같으면 0 을 반환한다.

컴퓨터는 이러한 연산을 레지스터 전체에 대해 한 번에 수행할 수 있다. 예를 들어, 한 레지스터에 다음 값이 있고

10100010101010010101101100101010

다른 레지스터에 다음 값이 있다면,

10001000010101010101010101111010

이 두 레지스터 전체에 대해 AND 연산을 수행할 수 있다. 컴퓨터는 첫 번째 비트부터 32 번째 비트까지 각각 AND 연산을 수행한다.

10100010101010010101101100101010 AND
10001000010101010101010101111010

100000000000000010101000100101010

결과를 보면 두 숫자 모두에서 1 이었던 위치에만 1 이 있고, 나머지는 모두 0 이다.

이제 OR 연산을 살펴보자.

```
10100010101010010101101100101010 OR
10001000010101010101010101111010
-----
1010101011111101010111101111010
```

이 경우, 어느 한쪽이라도 1 이 있으면 결과는 1 이 된다.

NOT 연산은 다음과 같다.

```
NOT 10100010101010010101101100101010
-----
01011101010101101010010011010101
```

각 비트가 반전된 것을 볼 수 있다.

마지막으로 XOR 연산이다.

```
10100010101010010101101100101010 XOR
10001000010101010101010101111010
-----
00101010111111000000111001010000
```

이는 OR 과 비슷하지만, 두 비트가 서로 같은 경우에는 0 이 반환되고 다른 경우에는 1 이 반환된다.

또한 어떤 수를 자기 자신과 XOR 하면 항상 0 이 된다.

```
10100010101010010101101100101010 XOR
10100010101010010101101100101010
-----
00000000000000000000000000000000
```

이 연산들이 유용한 이유는 두 가지이다.

- 컴퓨터가 매우 빠르게 수행할 수 있다.
- 여러 개의 참/거짓 값을 한 번에 비교할 수 있다.

명령어마다 실행 속도가 다르다는 사실을 알고 있었을지도 모른다. 실제로 그렇다. 그리고 이 논리 연산들은 대부분의 프로세서에서 가장 빠르다.

예를 들어, 어떤 수를 자기 자신과 XOR 하면 0 이 된다는 것을 보았다. XOR 연산은 메모리에서 값을 불러오는 연산보다 빠르기 때문에, 많은 프로그래머들이 레지스터를 0 으로 만들기 위해 이를 사용한다.

```
movl $0, %eax
```

는 종종 다음과 같이 대체된다.

```
xorl %eax, %eax
```

속도에 대해서는 12 장에서 더 자세히 다루겠지만, 여기서는 프로그래머들이 성능을 위해 이진 연산자를 사용해 얼마나 기교적인 방법을 쓰는지 보여주고 싶었다.

이제 이러한 연산자를 사용해 참/거짓 값을 다루는 방법을 살펴보자. 앞에서 이진수가 다양한 것을 표현할 수 있다고 말했다. 이번에는 내가 좋아하는 것과 아버지가 좋아하는 것을 이진수로 표현해 보자.

먼저 내가 좋아하는 것들이다.

- 음식: 예
- 헤비 메탈 음악: 예
- 격식 있는 옷 입기: 아니오
- 미식축구: 예

아버지가 좋아하는 것들은 다음과 같다.

- 음식: 예
- 헤비 메탈 음악: 아니오
- 격식 있는 옷 입기: 예
- 미식축구: 예

이제 좋아함을 1, 좋아하지 않음을 0 으로 표현하면 다음과 같다.

나

- 음식: 1
- 헤비 메탈 음악: 1
- 격식 있는 옷: 0
- 미식축구: 1

아버지

- 음식: 1
- 헤비 메탈 음악: 0
- 격식 있는 옷: 1
- 미식축구: 1

각 항목의 위치를 외우고 있다면, 다음과 같이 표현할 수 있다.

나: 1101
아버지: 1011

이제 아버지와 내가 둘 다 좋아하는 것을 알고 싶다면 AND 연산을 사용한다.

1101 AND
1011

1001

이는 다음을 의미한다.

- 음식: 예
- 헤비 메탈 음악: 아니오
- 격식 있는 옷: 아니오
- 미식축구: 예

컴퓨터는 이 1 과 0 이 무엇을 의미하는지 전혀 모른다. 그것을 해석하는 것은 전적으로 프로그램과 프로그래머의 책임이다. 만약 컴퓨터가 “두 사람이 동의하는 것”에 대한 답으로 1001 을 출력한다면, 사용자에게는 전혀 도움이 되지 않을 것이다.

이번에는 서로 의견이 다른 것을 알고 싶다고 하자. 이때는 XOR 연산을 사용한다. XOR 는 한 쪽만 1 일 때 1 을 반환한다.

1101 XOR
1011

0110

이 결과의 해석은 직접 해보면 된다.

AND, OR, NOT, XOR 연산은 불(Boolean)연산자라고 불린다. 이 연산들은 조지 불(George Boole)에 의해 처음 연구되었기 때문이다. 이제 누군가 불 연산자나 불 대수라고 말한다면 그것이 무엇을 의미하는지 알 수 있을 것이다 .

불 연산자 외에도 이진 연산자에는 시프트(shift)와 로테이트(rotate)가 있다. 이 연산자들은 이름 그대로 비트를 왼쪽이나 오른쪽으로 이동시킨다.

왼쪽 시프트는 각 비트를 왼쪽으로 한 칸 이동시키고, 가장 오른쪽에 0 을 채우며, 가장 왼쪽 비트는 버린다.

왼쪽 로테이트는 비슷하지만, 가장 왼쪽 비트를 오른쪽 끝으로 다시 가져온다.

예를 들어,

왼쪽 시프트 $10010111 = 00101110$
왼쪽 로테이트 $10010111 = 00101111$

32 비트 숫자를 32 번 로테이트하면 원래 숫자로 돌아오지만, 32 번 시프트하면 결과는 0 이 된다.

이 시프트는 무엇에 유용할까? 이진수가 여러 값을 담고 있을 때, 각 개별 값을 하나씩 확인하는 데 사용된다.

예를 들어, 아버지의 취향이 레지스터에 저장되어 있다면 다음과 같을 것이다.

00000000000000000000000000001011

이 값을 그대로 출력하는 것은 의미가 없으므로, 출력하려면 시프트와 마스킹(masking)을 사용해야 한다.

마스킹이란 필요 없는 부분을 제거하는 과정이다. 확인하고 싶은 값이 가장 오른쪽 비트에 오도록 시프트한 뒤, AND 연산으로 해당 비트만 남긴다.

예를 들어, 아버지가 격식 있는 옷을 좋아하는지 알고 싶다면, 이는 오른쪽에서 두 번째 비트이다. 따라서 오른쪽으로 1 비트 시프트하면 다음과 같다.

0000000000000000000000000000101

이제 가장 오른쪽 비트만 보기 위해 다음 값과 AND 연산을 한다.

0000000000000000000000000000101 AND
0000000000000000000000000000001

0000000000000000000000000000001

이 결과가 1 이면 좋아하는 것이고, 0 이면 좋아하지 않는 것이다.

해당 코드는 다음과 같다.

```
# NOTE - 레지스터 %ebx 에 아버지의 취향이 들어 있다고 가정
movl %ebx, %eax    # 원본 데이터를 잃지 않기 위해 복사
shrl $1, %eax      # 오른쪽으로 1 비트 시프트
andl $0b00000000000000000000000000000001, %eax
cmpl $0b00000000000000000000000000000001, %eax
je yes_he_likes_dressy_clothes
jmp no_he_doesnt_like_dressy_clothes
```

0b 표기는 이진수임을 의미한다. 여기서는 필요하지는 않지만 명확성을 위해 사용되었다. 또한 32 비트임을 강조하기 위해 0 을 모두 적어 두었다.

숫자가 함수나 시스템 호출의 옵션 집합을 나타낼 때, 각각의 참/거짓 요소를 플래그(flag)라고 부른다. 많은 시스템 호출은 이러한 방식을 사용한다.

예를 들어, open 시스템 호출의 두 번째 인자는 파일을 어떻게 열지에 대한 플래그들이다.

O_WRONLY

- 쓰기 전용
- 이진수: 00000000000000000000000000000001
- 8 진수: 01

O_RDWR

- 읽기/쓰기 모드
- 이진수: 00000000000000000000000000000010
- 8 진수: 02

O_CREAT

- 파일이 없으면 생성
- 이진수: 000000000000000000000000000010000000
- 8 진수: 0100

O_TRUNC

- 파일이 이미 존재하면 내용을 삭제
- 이진수: 00000000000000000000000000001000000000
- 8 진수: 01000

O_APPEND

- 파일 끝에 추가
- 이진수: 0000000000000000000000000000100000000000
- 8 진수: 02000

이 플래그들은 OR 연산으로 결합해서 사용한다. 예를 들어 쓰기 전용으로 열고, 파일이 없으면 생성하려면

`O_WRONLY (01) OR O_CREAT (0100) = 0101`

O_WRONLY 나 O_RDWR 를 설정하지 않으면 기본적으로 읽기 전용(O_RDONLY)로 열린다.

O_RDONLY 는 값이 0 이므로 실제로는 플래그가 아니다. 이와 같은 방식은 하나의 32 비트 값에 최대 32 개의 옵션을 담을 수 있기 때문에, 많은 함수와 시스템 호출에서 사용된다.

[3] 프로그램 상태 레지스터(Program Status Register)

우리는 레지스터의 비트들이 예/아니오, 참/거짓과 같은 답을 나타내는 데 사용될 수 있다는 것을 보았다. 컴퓨터에는 프로그램 상태 레지스터라는 레지스터가 있다. 이 레지스터는 계산 과정에서 발생하는 다양한 정보들을 담고 있다.

예를 들어, 두 수를 더했을 때 결과가 레지스터에 담을 수 있는 크기를 초과하면 어떻게 되는지 궁금했던 적이 있을 것이다. 프로그램 상태 레지스터에는 캐리 플래그(carry flag)라는 플래그가 있으며, 이를 통해 마지막 계산이 레지스터의 범위를 넘쳤는지를 확인할 수 있다.

이 외에도 다양한 상태를 나타내는 플래그들이 있다. 실제로 비교 명령어(cmpl)를 수행하면, 그 결과는 이 상태 레지스터에 저장된다. 조건 분기 명령어(jge, jne 등)는 이 결과를 사용해 점프를 할지 말지를 결정한다. 반면, 무조건 점프 명령어인 jmp 는 조건이 없기 때문에 상태 레지스터의 내용을 전혀 신경 쓰지 않는다.

만약 32 비트보다 큰 수를 저장해야 한다면 어떻게 해야 할까? 예를 들어, 숫자 두개의 레지스터를 사용하는 64 비트 수라고 가정해 보자. 두 개의 64 비트 수를 더하려면 먼저 하위 비트(가장 덜 중요한 레지스터)를 더한다. 그 결과 캐리가 발생했다면, 상위 비트 레지스터에 1 을 더해 주면 된다.

이 방식은 우리가 10 진수 덧셈을 배울 때 사용했던 방법과 같다. 한 자리에서 결과가 9 를 넘으면, 다음 자리로 올림(carry)을 한다. 예를 들어 65 와 37 을 더하면, 먼저 5 와 7 을 더해 12 를 얻는다. 2 는 해당 자리에 남기고 1 을 다음 자리로 올린다. 그 다음 6, 3, 그리고 올림된 1 을 더해 10 을 얻는다. 0 을 남기고 다시 1 을 다음 자리로 올린다. 다음 자리는 비어 있으므로 그대로 1 을 적는다.

다행히도 32 비트는 대부분 우리가 일상적으로 사용하는 숫자를 저장하기에 충분하다. 프로그램 상태 레지스터의 추가 플래그들은 부록 B 에서 다룬다.

[4] 다른 숫자 체계

지금까지 우리가 살펴본 내용은 양의 정수에만 적용된다. 하지만 현실 세계의 숫자는 항상 양의 정수만 있는 것은 아니다. 음수와 소수점이 있는 숫자도 사용된다.

[4-1] 부동소수점 수(Floating-point Numbers)

지금까지 다룬 숫자는 모두 정수, 즉 소수점이 없는 숫자였다. 컴퓨터는 소수점이 있는 수자를 다루는데 일반적인 어려움을 겪는다. 그 이유는 컴퓨터가 고정된 크기의 유한한 값만 저장할 수 있기 때문이다. 소수는 길이가 무한할 수도 있다. (예: $1/3$ 과 같은 순환 소수)

컴퓨터는 소수를 고정된 정밀도로 저장한다. 부동소수점 수는 두 부분으로 저장된다.

- 가수(mantissa)
- 지수(exponent)

가수는 실제 숫자의 자릿수를 담고 있고, 지수는 수자의 크기(규모)를 나타낸다. 예를 들어 12345.2 는 다음과 같이 저장된다.

$$1.23452 \times 10^4$$

여기서 가수는 1.23452 이고, 지수는 4 이다. 모든 숫자는 다음과 같은 형태로 저장된다.

$$X.XXXXX \times 10^{XXXX}$$

숫자 1 은 다음과 같이 저장된다.

$$1.00000 \times 10^0$$

가수와 지수는 길이가 제한되어 있기 때문에 흥미로운 문제가 발생한다. 예를 들어, 정수의 경우, 1 을 더하면 결과는 항상 정확히 1 만큼 증가한다. 하지만 부동소수점 수에서는 그렇지 않을 수 있다.

숫자가 매우 크다면, 예를 들어 5.234×10^{5000} 같은 수에 1 을 더해도 전혀 반영되지 않을 수 있다(두 부분 모두 길이가 제한되어 있기 때문이다.). 이는 연산 순서(order of operations)에 특히 영향을 미친다.

예를 들어, 5.234×10^{5000} 에 1 을 수십억 번, 수조 번 더해도 값은 변하지 않을 수 있다. 하지만 1 을 자기 자신에게 충분히 많이 더한 후, 그 결과를 원래 숫자에 더하면 변화가 생길 수도 있다.

또한 대부분의 컴퓨터에서 부동소수점 연산은 정수 연산보다 훨씬 느리다. 따라서 속도가 매우 중요한 프로그램에서는 대부분 정수가 사용된다.

[4-2] 음수(Negative Numbers)

컴퓨터에서 음수는 어떻게 표현될까? 한 가지 방법은 가장 왼쪽 비트를 부호 비트로 사용하는 것이다. 예를 들어,

00000000000000000000000000000001	→ 1
10000000000000000000000000000001	→ -1

이 방식은 직관적이며 실제로 일부 오래된 프로세서에서는 이런 방식을 사용했다. 하지만 몇 가지 문제가 있다.

첫째, 이런 방식의 부호 있는 숫자의 덧셈과 뺄셈을 처리하기 위해 훨씬 더 복잡한 회로가 필요하다. 더 큰 문제는 0 의 표현이다. 이 방식에서는 양의 0 과 음의 0 이 모두 존재하게 된다. 그러면 “음의 0 과 양의 0 은 같은가?”, “상황에 따라 0 의 부호는 어떻게 해야하는가?”와 같은 문제가 생긴다.

이러한 문제는 2 의 보수(two's complement) 표현 방식을 사용함으로써 해결되었다.

2 의 보수 방식에서 어떤 수의 음수를 구하려면 다음 단계를 따른다.

- | |
|------------------------|
| 1. 해당 수에 NOT 연산을 수행한다. |
| 2. 그 결과에 1 을 더한다. |

예를 들어 다음 수의 음수를 구해보자.

00000000000000000000000000000001

먼저 NOT 연산을 하면,

11111111111111111111111111111110

여기에 1 을 더하면,

11111111111111111111111111111111

이 값이 -1 이다.

이 표현 방식을 사용하면, 양수와 음수를 구분하지 않고도 덧셈을 정상적으로 수행할 수 있다. 예를 들어, 1 과 -1 을 더하면 모든 비트가 0 이 된다.

또한, 가장 왼쪽 비트는 여전히 부호 비트 역할을 하므로, 숫자가 양수인지 음수인지 쉽게 판단할 수 있다. 음수는 항상 가장 왼쪽 비트가 1 이다.

이 방식은 비트 수에 따라 표현 가능한 값의 범위도 바꾼다. 예를 들어, 바이트(8 비트)는 일반적으로 0 부터 255 까지 표현할 수 있다. 하지만 부호 있는 바이트는 -128 부터 127 까지 표현할 수 있다.

2 의 보수 표현에서 한 가지 주의할 점은, 비트 수를 늘릴 때 단순히 왼쪽에 0 을 추가하면 안된다는 것이다.(부호 없는 수와는 다르다.)

예를 들어, 4 비트에서 -3 은 다음과 같이 표현된다.

1101

이를 8 비트로 확장할 때 다음과 같이 하면 안된다.

00001101 (이는 13 을 의미함)

부호 있는 수를 확장할 때는 부호 확장(sign extension)을 해야 한다. 부호 확장이란, 새로운 비트를 추가할 때 기존 부호 비트와 같은 값으로 채우는 것이다.

- | |
|--|
| <ul style="list-style-type: none">- 음수라면 왼쪽을 1 로 채운다.- 양수라면 왼쪽을 0 으로 채운다. |
|--|

따라서 -3 을 4 비트에서 8 비트로 확장하면 다음과 같다.

11111101

x86 프로세서는 연산 대상이 부호 있는 수인지, 부호 없는 수인지에 따라 서로 다른 명령어 형태를 제공한다. 이들은 부록 B 에 나열되어 있다.

예를 들어, x86 프로세서에는 다음과 같은 명령어가 있다.

- sarl: 부호를 유지하는 산술적 오른쪽 시프트
- shrl: 부호를 유지하지 않는 논리적 오른쪽 시프트

이처럼 부호 여부에 따라 동작이 달라지는 명령어들이 존재한다.

[5] 8 진수와 16 진수 숫자

지금까지 다룬 수 체계는 10 진수와 2 진수였다. 하지만 컴퓨터에서는 두 가지가 더 널리 사용되는데, 바로 8 진수(octal)와 16 진수(hexadecimal)이다. 사실 이 둘은 2 진수보다 더 자주 쓰인다. 8 진수는 0 부터 7 까지만 사용하는 표현 방식이다. 따라서 8 진수에서의 10 은 실제로 10 진수의 8 을 의미하는데, 이는 8 하나의 묶음을 뜻하기 때문이다. 8 진수 121 은 81 인데, 이는 $64(8^2)$ 한 묶음, 8 두 묶음, 그리고 나머지 1 을 의미한다.

8 진수의 장점은 2 진수 3자리가 8 진수 1자리와 정확히 대응된다는 점이다(10 진수에는 이런 대응 관계가 없다). 예를 들어, 0 은 000, 1 은 001, 2 는 010, 3 은 011, 4 는 100, 5 는 101, 6 은 110, 7 은 111 에 해당한다.

리눅스의 파일 권한은 8 진수를 사용한다. 이는 리눅스 권한이 읽기(read), 쓰기(write), 실행(execute)의 세 가지로 구성되기 때문이다. 첫 번째 비트는 읽기 권한, 두 번째는 쓰기 권한, 세 번째는 실행 권한을 의미한다. 따라서 0(000)은 권한 없음, 6(110)은 읽기와 쓰기 권한, 5(101)는 읽기와 실행 권한을 의미한다.

이 숫자들은 파일 권한의 세 가지 집합에 사용된다. 즉, 소유자(owner), 그룹(group), 기타 사용자(everyone else)에 대한 권한이다.

예를 들어, 0644 는 소유자는 읽기와 쓰기 권한을 가지며, 그룹과 기타 사용자는 읽기만 가능하다는 뜻이다. 첫 번째 권한 집합은 파일 소유자, 두 번째는 그룹 소유자, 세 번째는 기타 사용자에게 해당한다.

또한, 0751 은 소유자는 읽기, 쓰기, 실행이 가능하고, 그룹 사용자는 읽기와 실행만 가능하며, 기타 사용자는 실행만 가능하다는 의미이다.

이처럼 8 진수는 비트(2 진수)를 세 개씩 묶는 데 사용된다. 어셈블러는 숫자 앞에 0 이 붙어 있으면 8 진수로 인식한다. 예를 들어, 010 은 8 진수 10, 즉 10 진수 8 을 의미한다. 반면 10 이라고 쓰면 이는 10 진수이다. 따라서 10 진수 앞에 실수로 0 을 붙이면 8 진수로 해석되므로 주의해야 한다.

16 진수(보통 “hex”라고 부름)는 한 자리당 1~15 의 값을 사용한다. 하지만 10~16 에 해당하는 숫자가 없기 때문에, 16 진수에서는 a 부터 f까지의 문자를 사용한다. 예를 들어, a 는 10, b 는 11 을 의미한다. 16 진수에서의 10 은 10 진수 16 이다.

8 진수에서 한 자리는 3 비트를 나타내지만, 16 진수에서 한 자리는 4 비트를 나타낸다. 두 자리는 1 바이트, 여덟 자리는 32 비트 워드에 해당한다. 따라서 16 진수는 2 진수보다 훨씬 짧고 쓰기 쉽다(자릿수가 1/4 밖에 되지 않는다).

16 진수에서 가장 중요한 숫자는 f 인데, 이는 모든 비트가 1 로 설정되었음을 의미한다. 예를 들어 레지스터의 모든 비트를 1 로 설정하려면 다음과 같이 쓸 수 있다.

```
movl $0xFFFFFFFF, %eax
```

이는 다음과 같이 2 진수로 쓰는 것보다 훨씬 쉽고 오류도 적다.

```
movl $0b11111111111111111111111111111111, %eax
```

또한 16 진수는 항상 0x 로 시작한다. 따라서 우리가 호출하는 것은 인터럽트 번호 128(16 이 8 번 묶인 값), 즉 0b00000000000000000000000010000000 에 해당하는 인터럽트이다.

16 진수와 8 진수는 익숙해지는 데 시간이 걸리지만, 컴퓨터 프로그래밍에서는 매우 많이 사용된다. 직접 몇 개의 16 진수 숫자를 만들어서 2 진수, 10 진수, 8 진수로 변환해 보는 것도 도움이 될 것이다 .

[6] 워드에서의 바이트 순서

비트와 바이트를 저수준에서 다룰 때 많은 사람이 헷갈리는 점 중 하나는, 레지스터에서 메모리로 데이터를 쓸 때 바이트가 가장 하위 바이트부터 저장된다는 점이다.

일반적으로는 레지스터에 0x5d 23 ef ee(바이트 구분을 위해 띄어 씀)가 있다면, 메모리에도 그 순서대로 저장될 것이라 생각하기 쉽다. 하지만 x86 프로세서에서는 반대 순서로 저장된다. 즉, 메모리에는 0xee ef 23 5d 의 순서로 저장된다. 바이트 순서는 거꾸로 저장되지만, 각 바이트 내부의 비트 순서는 정상적으로 유지된다.

모든 프로세서가 이렇게 동작하는 것은 아니다. x86 프로세서는 리틀 엔디언(little-endian) 방식의 프로세서로, 워드에서 가장 하위 바이트(least-significant byte)를 먼저 저장한다.

반면, 다른 프로세서들은 빅 엔디언(big-endian) 방식으로 동작하며, 우리가 숫자를 읽는 방식처럼 가장 상위 바이트(most-significant byte)를 먼저 저장한다.

이 차이는 보통 큰 문제가 되지 않는다(수년간 많은 기술적 논쟁을 불러일으키긴 했지만). 메모리에서 다시 레지스터로 읽어올 때 바이트 순서가 다시 뒤집히거나(혹은 빅 엔디언에서는 그대로 유지되므로) 프로그래머는 대부분 이를 인식하지 못한다. 이런 바이트 순서 처리는 레지스터-메모리 전송 과정에서 자동으로 처리된다.

하지만 다음과 같은 경우에는 문제가 발생할 수 있다.

- `movl` 로 여러 바이트를 한 번에 읽어온 뒤, `%al` 을 사용하거나 시프트 연산으로 바이트 단위 처리를 할 경우, 메모리에 보이는 순서와 다를 수 있다.
- 서로 다른 아키텍처에서 작성된 파일을 읽거나 쓸 때는 바이트 순서를 고려해야 할 수 있다 .
- 네트워크 소켓을 통해 데이터를 주고 받을 때, 프로토콜의 바이트 순서를 고려해야 할 수 있다.

이 문제를 인식하고만 있다면, 대부분 큰 문제는 되지 않는다. 바이트 순서에 대해 더 깊이 알고 싶다면 Cary A. Devorak 의 Endian FAQ 를 참고하라. 특히 Danial Cohen 의 “On Holy Wars and a Plea for Peace”라는 글이 도움이 된다.

[7] 숫자를 화면에 표시하기 위한 변환

지금까지 우리는 사용자에게 저장된 숫자를 보여주는 방법이 거의 없었는데, 그나마 가능한 방법은 `exit` 코드로 전달하는 매우 제한적인 방식뿐이었다. 이 절에서는 양의 정수를 문자열로 변환하여 화면에 표시하는 방법을 다룬다.

이 함수의 이름은 `integer2string` 이며, 두 개의 매개변수를 받는다.

하나는 변환할 정수 값이고, 다른 하나는 널 문자(0)로 채워진 문자열 버퍼이다.

이 버퍼는 숫자를 문자열로 저장하기에 충분히 크다고 가정한다.(마지막 널 문자를 포함해 최소 11 바이트).

우리가 숫자를 보는 방식은 10 진수이므로, 숫자의 각 자릿수를 얻기 위해서는 10 을 나누고 나머지를 사용하는 방식이 필요하다. 전체 과정은 다음과 같다.

- 숫자를 10 으로 나눈다.
- 나머지가 현재 자릿수이다. 이를 문자로 변환하여 저장한다.
- 몫이 0 이면 작업을 종료한다.
- 그렇지 않으면 몫을 다음 값으로 사용하고, 버퍼의 다음 위치로 이동해 과정을 반복한다.

문제는 이 과정이 1 의 자리부터 처리하기 때문에, 결과 문자열이 거꾸로 저장된다는 점이다. 따라서 마지막에 문자의 순서를 뒤집어야 한다. 이를 위해 변환 중에 문자를 스택에 저장한다. 이렇게 하면 스택에서 하나씩 꺼내 버퍼에 저장할 때, 자연스럽게 올바른 순서가 도난다.

이 함수의 코드는 `integer-to-string.s` 라는 파일에 작성하며, 다음과 같다.

이 함수의 목적은 정수 값을 10 진수 문자열로 변환하여 출력 가능하게 만드는 것이다. 입력으로는 충분히 큰 버퍼와 변환할 정수를 받으며, 출력으로는 해당 버퍼에 문자열이 덮어써진다.

사용되는 레지스터의 역할은 다음과 같다.

- %ecx: 처리된 무나 수
- %eax: 현재 처리 중인 값
- %edi: 기준값(10)

변환 과정에서는 나눗셈 연산을 사용하며, x86 에서 나눗셈은 %edx:%eax 를 하나의 값을 취급하므로 %edx 를 먼저 0 으로 초기화한다.

나눗셈 결과에서 몫은 %eax, 나머지는 %edx 에 저장된다.

나머지는 0 부터 9 사이의 값이므로, ASCII 코드에서 '0'을 더하면 숫자에 대응하는 문자 코드가 된다. 이렇게 얻은 문자는 스택에 저장된다.

모든 자릿수 처리가 끝나면, 스택에서 문자를 하나씩 꺼내 버퍼에 복사하면서 순서를 뒤집는다. 마지막으로 널 문자를 추가하고 함수는 종료된다.

이 함수를 실제 프로그램에서 사용하는 예제를 보기 위해, 이전 장에서 작성한 count_chars 함수와 write_newline 함수를 함께 사용한다.

다음 코드는 conversion-program.s 파일에 작성한다.

integer-to-string.s

```
# PURPOSE: Convert an integer number to a decimal string for display
#
# INPUT:
#   - Integer value
#   - Buffer large enough to hold the result (at least 11 bytes)
#
# OUTPUT:
#   - Buffer overwritten with decimal string

.equ ST_VALUE, 8
.equ ST_BUFFER, 12

.globl integer2string
.type integer2string, @function
```

```

integer2string:
    # Function prologue
    pushl %ebp
    movl %esp, %ebp

    # %ecx = character count
    movl $0, %ecx

    # %eax = value to convert
    movl ST_VALUE(%ebp), %eax

    # divisor = 10
    movl $10, %edi

conversion_loop:
    # clear %edx before division
    movl $0, %edx

    # divide %edx:%eax by 10
    divl %edi

    # remainder in %edx (0–9), convert to ASCII
    addl $'0', %edx

    # push character onto stack
    pushl %edx

    # increment digit count
    incl %ecx

    # if quotient is zero, stop
    cmpl $0, %eax
    je end_conversion_loop

    jmp conversion_loop

end_conversion_loop:
    # %edx = buffer address
    movl ST_BUFFER(%ebp), %edx

```

```

copy_reversing_loop:
    # pop character
    popl %eax

    # store only low byte
    movb %al, (%edx)

    # advance buffer pointer
    incl %edx

    # decrement counter
    decl %ecx

    # done?
    cmpl $0, %ecx
    je end_copy_reversing_loop

    jmp copy_reversing_loop

end_copy_reversing_loop:
    # null-terminate string
    movb $0, (%edx)

    # Function epilogue
    movl %ebp, %esp
    popl %ebp
    ret

```

conversion-program.s

```
.include "linux.s"

.section .data
# Buffer for converted string
tmp_buffer:
    .ascii "W0W0W0W0W0W0W0W0W0W0W0W0"

.section .text
.globl _start

_start:
    movl %esp, %ebp

    # push buffer
    pushl $tmp_buffer
    # push integer to convert
    pushl $824
    call integer2string
    addl $8, %esp

    # get string length
    pushl $tmp_buffer
    call count_chars
    addl $4, %esp

    # write syscall
    movl %eax, %edx    # length
    movl $SYS_WRITE, %eax
    movl $STDOUT, %ebx
    movl $tmp_buffer, %ecx
    int $LINUX_SYSCALL

    # newline
    pushl $STDOUT
    call write_newline

    # exit
    movl $SYS_EXIT, %eax
```

```
movl $0, %ebx  
int $LINUX_SYSCALL
```

이 프로그램은 다음과 같은 과정을 수행한다.

- 변환 결과를 저장할 임시 버퍼를 준비한다.
- 정수 값 824 를 integer2string 함수로 전달하여 문자열로 변환한다.
- 변환된 문자열의 길이를 count_chars 로 구한다.
- write 시스템 호출을 사용해 문자열을 표준 출력으로 출력한다.
- 줄바꿈을 출력한 후 프로그램을 종료한다.

프로그램을 빌드하려면 다음 명령어를 실행한다.

```
as integer-to-string.s -o integer-to-number.o  
as count-chars.s -o count-chars.o  
as write-newline.s -o write-newline.o  
as conversion-program.s -o conversion-program.o  
ld integer-to-number.o count-chars.o write-newline.o conversion-program.o -o  
conversion-program
```

프로그램을 실행하려면 다음과 같이 입력한다.

```
./conversion-program
```

출력 결과는 다음과 같이 표시된다.

```
824
```

문제

[1] 개념 이해하기

- {1} 십진수 5,294 를 이진수로 변환하라
- {2} 0x0234aeff 는 어떤 수를 나타내는가. 이 값을 이진수, 8 진수, 10 진수로 명시하라.
- {3} 이진수 10111001 과 101011 을 더하라.
- {4} 이진수 1100 과 1010110 을 곱하라
- {5} 앞의 두 문제에서 얻은 결과를 십진수로 변환하라
- {6} AND, OR, NOT, XOR 가 어떻게 동작하는지 설명하라.
- {7} 마스킹(masking)은 무엇을 위해 사용하는가
- {8} 파일을 쓰기 모드로 열고, 파일이 존재하지 않으면 생성하려면 open 시스템 호출의 flags 값으로 어떤 수를 사용해야 하는가
- {9} 32 비트 레지스터에서 -55 를 어떻게 표현하겠는가
- {10} 앞에서 표현한 값을 64 비트 레지스터로 부호 확장(sign-extension)하라.
- {11} 메모리에서 워드를 저장할 때 리틀 엔디언과 빅 엔디언 방식의 차이를 설명하라.

[2] 개념 사용하기

- {1} 이전에 exit 상태 코드를 통해 숫자 결과를 반환하던 프로그램들을 다시 살펴보고, 정수 → 문자열 변환 함수를 사용하여 결과를 출력하도록 수정하라.
- {2} integer2string 코드를 수정하여 10 진수 대신 8 진수로 결과를 반환하도록 하라.
- {3} integer2string 코드를 수정하여 변환 기준(base)이 하드코딩되지 않고 매개변수로 전달되도록 하라.
- {4} 하나의 정수를 매개변수로 받아, 그 값이 음수이면 1, 양수이면 0 을 반환하는 is_negative 라는 함수를 작성하라.

[3] 심화

- {1} integer2string 코드를 수정하여 변환 기준이 10 보다 클 수 있도록 하라(9 를 넘는 숫자에는 문자를 사용해야 한다.)
- {2} integer2string 의 반대 역할을 하는 nuber2integer 함수를 작성하라. 이 함수는 문자열을 받아 레지스터 크기의 정수로 변환해야 한다. 변환한 결과를 다시 integer2string 함수에 넣어 출력하여 테스트하라.
- {3} 좋아요와 싫어요를 하나의 머신 워드에 저장하고, 두 개의 좋아요/싫어요 집합을 비교하여 공통점을 찾는 프로그램을 작성하라.
- {4} STDIN 에서 문자열을 읽어 숫자로 변환하는 프로그램을 작성하라.

11 장 고급언어(High-Level Languages)

이 장에서는 우리가 처음으로 “실제 세계에서 사용되는” 프로그래밍 언어를 살펴보게 된다. 어셈블리 언어는 컴퓨터의 기계 수준에서 사용되는 언어이지만, 대부분의 사람들에게는 일상적인 프로그래밍에 사용하기에는 너무 번거롭다. 그래서 프로그래밍 작업을 더 쉽게 만들기 위해 다양한 프로그래밍 언어들이 만들어졌다.

여러 언어를 아는 것이 유용한 이유는 다음과 같다.

- 서로 다른 언어들은 서로 다른 개념을 기반으로 만들어져 있기 때문에, 다양한 프로그래밍 방법과 사고 방식을 배우는 데 도움이 된다.
- 언어마다 잘 어울리는 프로젝트 유형이 다르다.
- 회사마다 표준으로 사용하는 언어가 다르기 때문에, 여러 언어를 알수록 취업 경쟁력이 높아진다.
- 이미 많은 언어를 알고 있을수록 새로운 언어를 더 쉽게 배울 수 있다.

프로그래머는 새로운 언어를 배우는 일을 자주 하게 된다.

전문 프로그래머라면 보통 일주일 정도의 공부와 연습만으로도 새로운 언어를 기본적으로 사용할 수 있다. 프로그래밍 언어는 결국 도구일 뿐이며, 새로운 도구를 배우는 것을 두려워해서는 안 된다.

특히 컴퓨터 컨설팅을 하는 경우에는, 생계를 유지하기 위해 즉석에서 새로운 언어를 배워야 하는 상황도 자주 발생한다. 이때 어떤 언어를 사용할지는 보통 프로그래머가 아니라 고객이 결정한다.

이 장에서는 사용할 수 있는 여러 언어들을 간단히 소개한다. 저자는 개인적으로 몇 달에 한 번씩 새로운 언어를 배우려고 노력한다고 말한다.

[1] 컴파일 언어와 인터프리터 언어

컴파일 언어

- 어셈블리 언어에서는 한 줄의 코드가 정확히 하나의 기계어 명령어로 변환된다.
- 반면, 고급 언어에서는 한 문장이 하나 또는 수백 개의 기계어 명령어로 변환될 수 있다.
- 컴파일러가 충분히 똑똑하다면, 코드 일부를 더 빠르게 실행되도록 재구성하기도 한다.

인터프리터 언어

인터프리터 언어는 프로그램을 실행하기 위해 인터프리터라는 별도의 프로그램이 필요하다.

- 인터프리터는 프로그램을 한 줄씩 읽고 해석하며 실행한다.
- 이 때문에 일반적으로 컴파일된 프로그램보다 느리다.
- 하지만 잘 만들어진 인터프리터의 경우, 성능 차이는 거의 느껴지지 않을 수도 있다.

하이브리드 언어

일부 언어는 실행 전에 부분적으로 컴파일하여 바이트코드(byte-code)로 만든다.

- 바이트 코드는 사람이 쓰는 언어보다 인터프리터가 훨씬 빠르게 읽을 수 있다.
- Java, Python 일부 구현 등이 여기에 해당한다.

[1-1] 어떤 방식을 선택해야 할까?

컴파일 언어의 장점

- 사용자가 인터프리터를 설치할 필요가 없다.
- 배포가 간단하다.

인터프리터 언어의 단점

- 사용자의 컴퓨터에 인터프리터가 설치되어 있어야 한다.
- 어떤 인터프리터로 실행할지 시스템이 알아야 한다.

하지만 실제로는 컴파일/인터프리터 여부보다

- 사용 가능한 도구
- 프로그래밍 방법 지원
- 라이브러리와 생태계

가 언어 선택의 핵심 요소가 된다.

많은 언어는 컴파일과 인터프리터 방식을 모두 지원하기도 한다.

[1-2] 고급 언어의 특징

고급 언어는 컴퓨터가 아니라 프로그래머 중심으로 설계되어 있다.

이로 인해 다음과 같은 기능들이 가능해진다 .

- 여러 연산을 하나의 표현식으로 묶을 수 있다.
- 4 바이트 정수가 아니라, 문자열과 리스트 같은 큰 개념의 값을 하나의 값처럼 다룰 수 있다.
- 단순한 jump 가 아닌 고급 제어 구조(if, while, for 등) 제공
- 컴파일러가 타입 검사와 오류 검사를 수행
- 메모리 관리 자동화
- 하드웨어가 아니라 문제 영역(problem domain)에 가까운 언어로 작업 가능

[1-3] 언어 선택의 이유

그렇다면 왜 사람들은 서로 다른 언어를 선택할까? Perl 은 거의 모든 프로토콜과 데이터 형식을 처리할 수 있는 방대한 함수 라이브러리를 가지고 있어 많이 사용된다. 반면 Python 은 문법이 더 깔끔하고, 보다 직관적인 해결책을 제시하는 경우가 많다. 또한 크로스 플랫폼 GUI 도구도 매우 뛰어나다.

PHP 는 웹 애플리케이션을 쉽게 작성할 수 있게 해준다. Common LISP 는 배우려는 의지가 있는 사람에게 어떤 환경보다 강력한 기능과 표현력을 제공한다. Scheme 은 단순함과 강력함이 결합된 언어의 대표적인 예이다. C 언어는 다른 언어들과의 연동이 쉽다.

각 언어는 모두 다르며, 아는 언어가 많을수록 더 나은 프로그래머가 된다. 다양한 언어의 개념을 알고 있으면 문제에 더 잘 맞는 언어를 선택할 수 있고, 사용할 수 있는 도구의 폭도 넓어진다. 어떤 기능이 특정 언어에서 직접 지원되지 않더라도, 종종 다른 방식으로 구현(시뮬레이션)할 수 있다. 하지만 다양한 언어 경험이 없다면, 선택할 수 있는 가능성 자체를 알지 못하게 된다.

[2] 첫 번째 C 프로그램

다음은 화면에 “Hello world”를 출력하고 종료하는 여러분의 첫 번째 C 프로그램이다. 이 코드를 입력하고 파일 이름을 Hello-World.c 로 저장하라.

```
#include <stdio.h>

/* PURPOSE: This program is meant to show a basic */
/* C program. All it does is print */
/* "Hello World!" to the screen and */
/* exit. */

/* Main Program */
int main(int argc, char **argv)
{
    /* Print our string to standard output */
    puts("Hello World!\n");

    /* Exit with status 0 */
    return 0;
}
```

보는 것처럼 꽤 간단한 프로그램이다. 컴파일하려면 다음 명령을 실행한다.

```
gcc -o HelloWorld Hello-World.c
```

프로그램을 실행하려면 다음과 같이 한다.

```
./HelloWorld
```

이제 이 프로그램이 어떻게 구성되었는지 살펴보자.

C 에서 주석은 `/*`로 시작해서 `*/`로 끝난다. 주석은 여러 줄에 걸칠 수 있지만, 많은 사람들은 혼동을 피하기 위해 한 줄 안에서 시작하고 끝내는 방식을 선호한다.

`#include <stdio.h>`는 프로그램의 첫 부분이다. 이것은 전처리기 지시문이다. C 컴파일 과정은 두 단계로 나뉘는데, 전처리기(preprocessor)와 메인 컴파일러이다. 이 지시문은 전처리기에 `stdio.h` 파일을 찾아서 그 내용을 프로그램 안에 그대로 붙여 넣으라고 지시한다. 전처리기는 프로그램의 텍스트를 구성하는 역할을 한다. 여러 파일을 하나로 합치고, 매크로를 처리하는 등의 작업을 수행한다. 텍스트 구성이 끝나면 전처리기의 작업은 종료되고, 메인 컴파일러가 작업을 시작한다.

이제 `stdio.h` 에 들어 있는 모든 내용은 마치 여러분이 직접 타이핑하 것처럼 프로그램 안에 포함된다. 파일 이름을 꺾쇠 괄호(`<>`)로 감싸면 컴파일러는 표준 경로(보통 `/usr/include` 와 `/usr/local/include`)에서 해당 파일을 찾는다. 어쨌든 `stdio.h` 에는 표준 입력과 출력 함수 및 변수들의 선언이 들어 있다. 이 선언들은 컴파일러에게 어떤 입출력 함수들이 사용 가능한지를 알려준다. 그 다음 몇 줄은 단순히 프로그램에 대한 설명 주석이다.

그 다음에 나오는 `int main(int argc, char **argv)`는 함수의 시작이다. C 에서 함수는 이름, 매개변수, 반환 타입으로 선언된다. 이 선언은 함수 이름이 `main` 이고, `int` 타입의 값을 반환하며, 두 개의 매개변수를 가진다는 뜻이다. 첫 번째는 `argc` 라는 `int` 이고, 두 번째는 `argv` 라는 `char **`이다. 매개변수들이 스택의 어디에 위치하는지 걱정할 필요가 없다. C 컴파일러가 이를 알아서 처리해준다. 레지스터에 값을 넣고 빼는 것도 마찬가지로 컴파일러가 처리해 준다.

`main` 함수는 C 언어에서 특별한 함수로, 모든 C 프로그램의 시작점이다.(어셈블리 언어에서의 `_start` 와 비슷하다.) 이 함수는 항상 두 개의 매개변수를 받는다. 첫 번째는 명령줄에서 전달된 인자의 개수이고, 두 번째는 전달된 인자들의 목록이다.

다음 줄은 함수 호출이다. 어셈블리 언어에서는 함수의 인자를 스택에 직접 푸시한 다음 함수를 호출해야 했다. C 에서는 이런 복잡한 작업을 신경 쓸 필요가 없다. 단순히 괄호 안에 매개변수를 넣어서 함수를 호출하면 된다. 여기서는 `puts` 함수를 하나의 매개변수와 함께 호출한다. 이 매개변수는 출력하려는 문자열이다. 문자열을 따옴표로 입력하기만 하면 컴파일러가 저장 공간을 정의하고 해당 저장 공간의 포인터를 스택에 올린 뒤 함수를 호출해 준다. 훨씬 적은 작업으로 같은 일을 할 수 있는 것이다.

마지막으로 함수는 0 을 반환한다. 어셈블리 언어에서는 반환 값을 %eax 에 저장했지만, C 에서는 return 문을 사용하면 알아서 처리된다. main 함수의 반환 값은 프로그램의 종료 코드로 사용된다.

보듯이, 고급 언어를 사용하면 프로그래밍이 훨씬 쉬워진다. 또한 프로그램을 여러 플랫폼에서 더 쉽게 실행할 수 있다. 어셈블리 언어에서는 프로그램이 운영체제와 하드웨어 플랫폼에 강하게 종속되지만, 컴파일 언어나 인터프리터 언어에서는 동일한 코드가 여러 운영체제에서 실행되는 경우가 많다. 예를 들어, Linux, Windows, UNIX 등 대부분의 운영체제를 실행하는 x86 하드웨어에서 빌드하고 실행할 수 있다. 또한 여러 운영체제를 실행하는 Macintosh 하드웨어에서도 실행할 수 있다.

C 프로그래밍 언어에 대한 추가 정보는 부록 E 에서 확인할 수 있다.

[3] Perl

Perl 은 인터프리터 언어로, 주로 Linux 와 UNIX 기반 플랫폼에서 사용된다. 사실 거의 모든 플랫폼에서 실행되지만, Linux 와 UNIX 환경에서 가장 자주 볼 수 있다. 다음은 Perl 로 작성한 프로그램이며, Hello-World.pl 이라는 파일에 입력해야 한다.

```
#!/usr/bin/perl
print("Hello world!\n");
```

Perl 은 인터프리터 언어이기 때문에 컴파일이나 링크 과정이 필요 없다. 다음 명령으로 바로 실행된다.

```
perl Hello-World.pl
```

보는 것처럼 Perl 버전은 C 버전보다 훨씬 짧다. Perl 에서는 함수나 프로그램의 시작 지점을 선언할 필요가 없다. 명령을 입력하기만 하면 인터프리터가 순서대로 실행한다. 사실 이 프로그램은 두 줄의 코드로 이루어져 있는데, 그 중 한 줄은 선택 사항이다.

첫 번째 선택적인 줄은 UNIX 시스템에서 어떤 인터프리터를 사용할지 알려주는 역할이다. #!는 이 파일이 인터프리터 프로그램임을 알리고, /usr/bin/perl 은 /usr/bin/perl 프로그램을 사용해 이 파일을 해석하라는 의미이다. 하지만 perl Hello-World.pl 처럼 이미 Perl 인터프리터를 명시해서 실행했기 때문에 이 줄은 필요하지 않았다.

다음 줄은 Perl 의 내장 함수인 print 를 호출한다. 이 함수는 출력할 문자열 하나를 매개변수로 받는다. 이 프로그램에는 명시적인 반환 문이 없다. 파일의 끝까지 실행되면 자동으로 종료되기 때문이다. 또한 실행 중 오류가 없었기 때문에 자동으로 0 을 반환한다. 인터프리터 언어는 이처럼 불필요한 준비 작업을 줄이고, 가능한 한 빨리 동작하는 코드를 작성할 수 있도록 초점을 둔다.

이 예제에서는 잘 드러나지 않았지만, Perl 의 중요한 특징 중 하나는 문자열을 하나의 값으로 취급한다는 점이다. 어셈블리 언어에서는 문자열을 여러 값의 연속으로 다루고, 첫 글자를 가리키는 포인터를 사용해야 했다. Perl 은 문자열이 하나의 값으로 직접 저장되는 것처럼 동작하며, 문자열을 다루는 복잡함을 숨겨 준다. 실제로 Perl 의 가장 큰 강점 중 하나는 텍스트를 처리하는 능력과 속도이다.

[4] Python

Python 버전은 프로그램은 Perl 버전과 거의 동일해 보인다. 하지만 이 간단한 예제만으로는 알기 어렵지만, Python 은 Perl 과 매우 다른 언어이다. 다음 프로그램을 Hello-World.py 라는 파일에 입력하라.

```
#!/usr/bin/python
print "Hello World"
```

이 프로그램의 각 줄이 어떤 역할을 하는지는 충분히 짐작할 수 있을 것이다.

문제

[1] 개념 이해하기

- {1} 인터프리터 언어와 컴파일 언어의 차이점은 무엇인가
- {2} 어떤 이유들로 인해 새로운 프로그래밍 언어를 배워야 할 필요가 생길 수 있는가

[2] 개념 활용하기

- {1} 새로운 프로그래밍 언어 하나를 선택하여 기본 문법을 익힌 후, 이 책에 나오는 프로그램 중 하나를 그 언어로 다시 작성하라.
- {2} 위 질문에서 작성한 프로그램에서, 선택한 프로그래밍 언어가 자동으로 처리해 주는 구체적인 기능들은 무엇인가
- {3} 작성한 프로그램을 10,000 번 연속 실행되도록 수정하라. 이를 어셈블리 언어와 새로 배운 프로그래밍 언어 각각으로 구현한 뒤, time 명령어를 사용하여 실행 시간을 비교하라. 어느 쪽이 더 빠르나. 그 이유는 무엇인가
- {4} 선택한 프로그래밍 언어의 입출력 방식은 Linux 시스템 호출(system call)을 이용한 입출력 방식과 어떻게 다른가

[3] 심화

- {1} Perl 처럼 매우 간결한 언어들이 존재함에도 불구하고, 이 책이 어셈블리 언어처럼 장황한 언어부터 시작하는 이유는 무엇이라고 생각하는가
- {2} 고급 프로그래밍 언어는 프로그래밍 과정에 어떤 영향을 미쳤다고 생각하는가
- {3} 왜 이렇게 많은 프로그래밍 언어들이 존재한다고 생각하는가
- {4} 두 가지 새로운 고급 프로그래밍 언어를 학습하라. 이 두 언어는 서로 어떻게 다른가. 또한 어떤 점에서 유사한가. 각 언어는 문제 해결에 어떤 접근 방식을 취하는가

12 장 최적화(Optimization)

최적화란 애플리케이션이 더 효율적으로 실행되도록 만드는 과정이다. 최적화의 대상은 다양할 수 있는 데, 예를 들면 실행 속도, 메모리 사용량, 디스크 공간 사용량 등이 있다. 하지만 이장에서는 속도 최적화에 초점을 맞춘다.

[1] 언제 최적화해야 하는가

너무 이른 최적화는 차라리 하지 않는 것보다 못하다. 최적화를 하면 코드가 더 복잡해지고, 그 결과 가독성이 떨어진다. 코드를 읽는 사람은 왜 그런 방식으로 작성되었는지를 이해하기 어려워지고, 이는 유지보수 비용을 증가시킨다. 심지어 작성자 자신이 코드의 동작 원리를 잘 알고 있더라도, 최적화된 코드는 디버깅과 확장이 훨씬 어렵다. 또한 최적화 자체에 드는 시간과, 최적화된 코드를 수정하는 데 걸리는 시간 때문에 개발 속도도 크게 느려진다.

문제를 더 어렵게 만드는 점은, 프로그램에서 어디가 느린지 미리 알 수 없다는 것이다. 경험 많은 프로그래머조차 어떤 부분이 병목이 될지를 정확히 예측하기는 어렵다. 그 결과, 실제로는 중요하지 않은 부분을 최적화하는 데 시간을 낭비하게 되는 경우가 많다. “어디를 최적화 할 것인가”라는 절에서는 성능 문제가 발생하는 지점을 찾는 방법을 설명한다.

프로그램을 개발하는 동안에는 다음과 같은 우선순위를 가져야 한다.

- 모든 것이 문서화되어 있어야 한다.
- 모든 것이 문서에 적힌 대로 정확히 동작해야 한다.
- 코드는 모듈화되어 있고 쉽게 수정할 수 있어야 한다.

문서화는 특히 팀 단위 작업에서 필수적이다. 프로그램이 올바르게 동작하는 것은 가장 중요하다. 이 목록 어디에도 속도는 포함되어 있지 않다. 초기 개발 단계에서 최적화가 필요 없는 이유는 다음과 같다.

- 작은 속도 문제는 하드웨어 업그레이드로 해결할 수 있으며, 이는 보통 프로그래머의 시간보다 저렴하다.
- 애플리케이션은 개발 과정에서 크게 변경되므로, 초기 최적화 작업은 대부분은 헛수고가 된다.
- 성능 문제는 보통 코드의 일부에만 집중되어 있으며, 프로그램이 거의 완성되기 전에는 이를 찾기 어렵다.

따라서 최적화는 코드가 완성되고, 실제로 성능 문제가 있다는 것이 확인된 후에 수행해야 한다.

저자가 참여했던 한 웹 기반 전자상거래 프로젝트에서, 그는 정확성에만 집중했다. 동료들은 페이지 하나를 처리하는 데 12 초가 걸리는 것을 걱정했지만, 그는 먼저 올바르게 만드는 것을 최우선으로 삼았다. 3 개월 후 코드가 완성되었고, 병목을 찾고 제거하는 데는 단

3 일밖에 걸리지 않았다. 그 결과 평균 처리 시간은 0.25 초 이하로 줄었다. 올바른 순서를 지켰기 때문에 정확하면서도 빠른 프로그램을 완성할 수 있었다.

[2] 어디를 최적화해야 하는가

성능 문제가 있다는 것을 확인했다면, 코드의 어느 부분에서 문제가 발생하는지를 찾아야 한다. 이를 위해 프로파일러(profiler)를 사용할 수 있다. 프로파일러는 프로그램을 실행하면서 각 함수가 얼마나 자주 호출되는지, 그리고 얼마나 많은 시간이 소요되는지를 알려준다.

GNU/Linux 의 표준 프로파일링 도구는 gprof 이지만, 사용법에 대한 자세한 설명은 이 책의 범위를 벗어났다. 프로파일링 결과를 통해 가장 자주 호출되거나, 가장 많은 시간이 소비되는 함수들을 찾아낼 수 있다. 이러한 함수들이 최적화 주요 대상이다.

만약 어떤 함수에서 프로그램 실행 시간의 1%만 소비되면, 아무리 빨라져도 전체 성능 향상은 최대 1%에 불과하다. 하지만 실행 시간의 20%를 차지하는 함수라면, 작은 개선만으로도 전체 성능이 눈에 띄게 향상된다. 프로파일링은 어디에 시간을 투자해야 하는지를 알려주는 중요한 도구다.

함수를 최적화하려면, 그 함수가 어떤 방식으로, 언제 호출되는지를 이해해야 한다. 호출 방식에 대해 많이 알수록, 더 효과적인 최적화를 할 수 있다.

최적화는 크게 두 가지로 나뉜다.

- 지역 최적화(Local Optimization)
- 전역 최적화(Global Optimization)

지역 최적화는 특정 하드웨어나 코드 조각에 국한된 최적화이다. 예를 들면, 특정 연산을 가장 빠르게 수행하는 방법이나, 가장 자주 사용되는 경우를 기준으로 코드를 조정하는 것이다.

전역 최적화는 구조적인 최적화이다. 예를 들어, 서로 다른 도시에 있는 세 사람이 세인트루이스에서 만나야 한다고 할 때, 더 좋은 도로를 찾는 것은 지역 최적화이고, 아예 화상회의를하기로 결정한 것은 전역 최적화이다. 전역 최적화는 느린 경로를 빠르게 만드는 것이 아니라, 느린 경로 자체를 피하도록 구조를 바꾸는 것이다.

[3-1] 지역 최적화(Local Optimization)

다음은 코드 조각을 최적화하는 데 널리 알려진 방법들이다. 고급 언어를 사용할 경우, 이부는 컴파일러가 자동으로 수행한다.

[3-2] 계산 결과 미리 구하기

함수의 입력과 출력이 매우 제한적인 경우, 가능한 모든 결과를 미리 계산해두고 함수 호출 시 결과를 조회하는 방식이 있다. 저장 공간은 더 필요하지만, 계산 비용이 큰 경우 매우 효과적이다.

[3-3] 계산 결과 기억하기

이전 방식과 비슷하지만, 모든 결과를 미리 계산하지 않는다. 한 번 계산한 결과를 저장해 두었다가, 같은 입력이 다시 들어오면 저장된 결과를 반환한다. 저장 공간을 덜 사용하며, 이를 캐싱(caching) 또는 메모이제이션(memorizing)이라고 한다.

[3-4] 참조 지역성(Locality of Reference)

참조 지역성이란, 메모리에서 서로 가까운 위치의 데이터를 연속적으로 사용하는 성질을 말한다. 가상 메모리 환경에서는 메모리 페이지가 디스크에서 로드되는데, 데이터를 분산 접근하면 페이지 교체가 자주 발생한다. 반면, 하나의 데이터에 대해 모든 작업을 한 번에 처리하면 디스크 접근 횟수를 줄일 수 있다.

또한, 프로세서는 내부에 매우 빠른 캐시 메모리를 가지고 있다. 작은 메모리 영역만 집중적으로 사용하면, 메인 메모리조차 거치지 않고 캐시만으로 처리할 수 있다.

[3-5] 레지스터 사용

레지스터는 컴퓨터에서 가장 빠른 저장 공간이다. 메모리는 버스를 통해 접근해야 하지만, 레지스터는 CPU 내부에 있다. 가능한 한 많은 데이터를 레지스터에 유지하는 것이 중요하다. 고급 언어에서는 컴파일러가 이를 결정한다.

[3-6] 인라인 함수(Inline Functions)

함수 호출은 스택 처리와 점프 비용이 든다. 인라인 함수는 호출 대신 함수 코드를 호출 위치에 직접 삽입한다. 이는 속도를 향상시키지만 코드 크기를 증가시킨다. 재귀 함수는 인라인될 수 없다.

[3-7] 최적화된 명령어

같은 작업을 수행하는 여러 어셈블리 명령어가 존재하며, 속도는 프로세서마다 다를 수 있다. 예를 들어, 레지스터를 0으로 만드는 가장 빠른 방법은 mov가 아니라 자기 자신과 xor하는 것이다. 고급 언어에서는 컴파일러가 이를 처리한다.

[3-8] 주소 지정 방식(Addressing Modes)

즉시값과 레지스터 주소 지정이 가장 빠르고, 직접 -> 간접 -> 베이스 포인터/인덱스 방식 순으로 느려진다. 가능한 빠른 주소 지정 방식을 사용하는 것이 좋다.

[3-9] 데이터 정렬(Data Alignment)

일부 프로세서는 워드 단위로 정렬된 메모리를 더 빠르게 접근한다. 구조체를 설계할 때 워드 정렬을 유지하는 것이 좋다. 일부 비 x86 프로세서는 비정렬 접근을 아예 지원하지 않기도 한다.

[4-1] 전역 최적화(Global Optimization)

전역 최적화에는 두 가지 목표가 있다. 첫 번째 목표는 지역 최적화를 수행하기 쉬운 형태로 코드를 구성하는 것이다. 예를 들어, 여러 개의 느리고 복잡한 계산을 수행하는 큰 절차가 있다면, 그 절차의 일부를 별도의 함수로 분리할 수 있는지 살펴볼 수 있다. 그렇게 하면 값들을 미리 계산해 두거나 메모이제이션할 수 있는 가능성이 생긴다.

무상태 함수(stateless function) - 즉, 전역 변수나 시스템 호출 없이, 전달된 매개변수에만 의존하여 동작하는 함수 - 는 컴퓨터에서 가장 최적화하기 쉬운 형태의 함수이다. 프로그램에 무상태 부분이 많을수록 최적화를 할 수 있는 기회도 많아진다.

앞서 언급한 전자상거래 사례에서, 컴퓨터는 특정 재고 항목과 연관된 모든 부품을 찾아야 했다. 이 과정에는 약 12번의 데이터베이스 호출이 필요했고, 최악의 경우 약 20초가 걸렸다. 하지만 이 프로그램의 목적은 대화형(interactive)이었기 때문에, 긴 대기 시간은 치명적이었다.

그러나 재고 구성 정보는 자주 바뀌지 않는다는 점을 알고 있었다. 그래서 데이터베이스 호출들을 각각의 무상태 함수로 분리했다. 이후 이 함수들은 메모이제이션 할 수 있었다. 매일 하루가 시작될 때 함수 결과를 초기화하여 혹시 변경 사항이 있는 경우를 대비했고, 여러 재고 항목을 미리 로드했다. 그 결과, 하루 중 처음으로 특정 재고 항목에 접근할 때는 이전처럼 20 초가 걸렸지만, 그 이후에는 데이터베이스 결과가 메모이제이션되어 1 초도 걸리지 않게 되었다.

전역 최적화는 보통 다음과 같은 특성을 함수에 부여하는 것을 목표로 한다.

[4-2] 병렬화(Parallelization)

병렬화란 알고리즘을 여러 프로세스에 효과적으로 나누어 실행할 수 있음을 의미한다. 예를 들어, 임신은 병렬화가 거의 불가능하다. 아무리 많은 여성이 이어도 임신 기간은 9 개월이 걸린다. 반면 자동차 만드는 작업은 병렬화가 가능하다. 한 작업자는 엔진을 만들고, 다른 작업자는 내부를 동시에 작업할 수 있다. 대부분의 애플리케이션에는 병렬화에 한계가 있지만, 병렬화 가능성이 높을수록 다중 프로세서나 클러스터 환경을 더 잘 활용할 수 있다.

[4-3] 무상태성(Statelessness)

앞서 설명했듯이, 무상태 함수나 프로그램은 동작에 필요한 모든 데이터를 명시적으로 전파한다. 대부분의 프로그램은 완전히 무상태는 아니지만, 일정 범위 내에서 무상태처럼 동작할 수 있다. 전자상거래 예제에서 해당 함수는 완전히 무상태는 아니었지만, 하루 단위라는 범위 안에서는 무상태로 취급할 수 있었다. 그래서 낮 동안은 무상태 함수처럼 최적화되고, 밤에는 변경 가능성을 고려하도록 설계했다.

무상태성의 큰 장점 두 가지는 다음과 같다.

- 대부분의 무상태 함수는 병렬화가 가능하다.
- 메모이제이션의 효과를 크게 볼 수 있다.

전역 최적화는 무엇이 효과가 있고 무엇이 효과가 업는지를 알기까지 상당한 경험이 필요하다. 코드의 최적화 문제를 어떻게 해결할지 결정하려면 모든 요소를 고려해야 하며, 어떤 문제를 해결하면 다른 문제가 생길 수 있다는 점도 항상 염두에 두어야 한다.

문제

[1] 개념 이해하기

- {1} 프로그래밍에서 최적화는 다른 우선순위들과 비교했을 때 어느 정도의 중요도를 가지는가
- {2} 지역 최적화와 전역 최적화의 차이점은 무엇인가
- {3} 지역 최적화의 종류에는 어떤 것들이 있는가
- {4} 프로그램의 어떤 부분이 최적화가 필요한지 어떻게 판단할 수 있는가
- {4} 프로그래밍에서 최적화는 다른 우선순위들과 비교했을 때 어느 정도의 중요도를 가지는가. 이 질문을 반복해서 제시한 이유는 무엇이라고 생각하는가.

[2] 개념 사용하기

- {1} 이 책에서 나오는 각 프로그램을 다시 검토하고, 이 장에서 설명한 절차에 따라 최적화를 시도하라.
- {2} 앞선 과제에서 사용한 프로그램 중 하나를 선택하여, 특정 입력 조건에서 코드의 성능에 어떤 영향이 있는지 계산해 보라.

[3] 심화

- {1} 매우 빠르다고 느껴지는 오픈소스 프로그램 하나를 찾아라. 해당 프로그램의 개발자 중 한 명에게 연락하여, 성능 향상을 위해 어떤 종류의 최적화를 수행했는지 질문하라.
- {2} 상대적으로 느리다고 느껴지는 오픈소스 프로그램 하나를 찾아라. 먼저 그 프로그램이 느린 이유를 추측한 뒤, 코드를 다운로드하고 gprof 또는 유사한 도구를 사용하여 프로파일링하라. 코드에서 가장 많은 시간이 소비되는 부분을 찾아 최적화를 시도하라. 실제로 느린 이유는 처음에 예상했던 것과 달랐는가
- {3} 컴파일러는 지역 최적화의 필요성을 제거했는가. 그 이유는 무엇인가 또는 왜 그렇지 않은가
- {4} 컴파일러가 함수 호출 경계를 넘어 코드 최적화를 시도할 경우 어떤 문제에 직면할 수 있는가.

13 장 여기서부터 계속 나아가기

여기까지 온 것을 축하한다. 이제 여러분은 프로그래밍의 여러 영역에서 발생하는 문제들을 이해할 수 있는 기초를 갖추게 되었다. 비록 앞으로 어셈블리 언어를 다시 사용하지 않게 되더라도, 컴퓨터과학 전반을 이해하는 데 도움이 되는 소중한 관점과 사고의 틀을 얻게 된 것이다.

프로그래밍을 배우는 방법에는 기본적으로 세 가지가 있다.

- 하향식이 아닌 상향식(Bottom-Up): 이 책에서 사용한 방식이다. 저수준 프로그래밍부터 시작하여 점차 더 일반적이고 추상적인 개념으로 나아간다.
- 하향식(Top-down): 상향식과는 반대 방향이다. 컴퓨터로 무엇을 하고 싶은지에 초점을 맞추고, 그것을 점점 더 세분화하여 결국 저수준까지 내려가는 방식이다.
- 중간 단계 접근법(From the Middle): 특정 프로그램 언어나 API 를 가르치는 책들이 여기에 해당한다. 개념보다는 구체적인 사용법과 세부 사항에 더 집중한다.

사람마다 선호하는 접근 방식이 다르지만, 훌륭한 프로그래머는 이 세 가지 방식 모두 고려한다. 상향식 접근은 기계적인 측면을 이해하는 데 도움을 주고, 하향식 접근은 문제 영역 자체를 이해하는 데 도움을 주며, 중간 단계 접근은 실용적인 질문과 해답을 제공한다. 이 중 어느 하나라도 빠진다면 그것은 실수이다.

컴퓨터 프로그래밍은 매우 방대한 분야이다. 프로그래머로서 여러분은 끊임없이 배우고 자신의 한계를 넓혀 나갈 준비가 되어 있어야 한다. 여기 소개된 책들은 그런 과정에 도움이 될 것이다. 이 책들은 단순히 주제만 가르치는 것이 아니라, 다양한 사고 방식과 문제 해결 방법까지 함께 가르쳐 준다.

앨런 펄리스(Alan Perlis)는 다음과 같이 말했다.

“프로그래밍에 대한 사고방식을 바꾸지 못하는 언어는 배울 가치가 없다.”

<https://www.cs.yale.edu/homes/perlis-alan/quotes.html>

항상 더 나은 방법과 새로운 사고 방식을 찾고자 한다면, 여러분은 성공적인 프로그래머가 될 것이다. 반대로 자기 계발을 추구하지 않는다면,

“조그만 자고, 조금만 즐기고, 손을 모으고 쉬고 있으면 가난이 강도처럼, 궁핍이 무장한 자처럼 찾아올 것이다.” (잠언 24 장 33-34 절, NIV)

물론 실제로 그렇게까지 극단적이지 않겠지만, 그래도 항상 배우는 자세를 유지하는 것이 최선이다.

이 책들이 선택된 이유는 그 내용과 컴퓨터 과학 분야에서 받는 높은 평가 때문이다. 각각의 책은 고유한 가치를 가지고 있다. 목록에 많은 책들이 있으니, 온라인 리뷰를 살펴보고 여러분의 흥미를 끄는 지점부터 시작하는 것이 좋은 방법일 것이다.

[2] 하향식이 아닌 상향식 (From the Bottom Up)

Programming from the Ground Up

→ 기초부터 배우는 프로그래밍

Introduction to Algorithms

→ 알고리즘 개론

The Art of Computer Programming

→ 컴퓨터 프로그래밍의 예술

(1~3 권, 그중 1 권이 가장 중요)

Programming Languages

→ 프로그래밍 언어

Modern Operating Systems

→ 현대 운영체제

Linkers and Loaders

→ 링커와 로더

Computer Organization and Design: The Hardware/Software Interface

→ 컴퓨터 구조와 설계: 하드웨어와 소프트웨어의 인터페이스

[3] 하향식 접근 (From the Top Down)

How to Design Programs

→ 프로그램을 설계하는 방법

Simply Scheme: An Introduction to Computer Science

→ *Simply Scheme*: 컴퓨터 과학 입문

How to Think Like a Computer Scientist: Learning with Python

→ 컴퓨터 과학자처럼 사고하기: 파이썬으로 배우는 프로그래밍

Structure and Interpretation of Computer Programs

→ 컴퓨터 프로그램의 구조와 해석 (SICP)

Design Patterns

→ 디자인 패턴

What Not How: The Rules Approach to Application Development

→ 방법이 아닌 무엇을: 규칙 기반 애플리케이션 개발

The Algorithm Design Manual

→ 알고리즘 설계 매뉴얼

Programming Language Pragmatics

→ 프로그래밍 언어의 실제

Essentials of Programming Languages

→ 프로그래밍 언어의 핵심

[4] 중간 단계 접근 (From the Middle Out)

Programming Perl

→ 펄 프로그래밍

Common LISP: The Language

→ 커먼 리스프 언어

ANSI Common LISP

→ ANSI 커먼 리스프

The C Programming Language

→ C 프로그래밍 언어

[5] 제 13 장 이후 추천 (Moving On from Here)

C Primer Plus

→ C 프라이어머 플러스

The C++ Programming Language

→ C++ 프로그래밍 언어

Thinking in Java

→ 자바로 사고하기

The Scheme Programming Language

→ 스킴 프로그래밍 언어

Linux Assembly Language Programming

→ 리눅스 어셈블리 언어 프로그래밍

[6] 전문 주제 (Specialized Topics)

Programming Pearls

→ 프로그래밍 진주

More Programming Pearls

→ 더 많은 프로그래밍 진주

Understanding Relational Databases

→ 관계형 데이터베이스 이해하기

The Mythical Man-Month

→ 맨먼스 미신

The Art of UNIX Programming

→ 유닉스 프로그래밍의 예술

Advanced Programming in the UNIX Environment

→ 유닉스 환경에서의 고급 프로그래밍

UNIX Network Programming

→ 유닉스 네트워크 프로그래밍

Modern C++ Design

→ 모던 C++ 설계

The Art of Compiler Design: Theory and Practice

→ 컴파일러 설계의 예술: 이론과 실제

Advanced Compiler Design and Implementation

→ 고급 컴파일러 설계와 구현

Refactoring: Improving the Design of Existing Code

→ 리팩터링: 기존 코드의 설계 개선

Computers and Typesetting

→ 컴퓨터와 조판

Applied Cryptography

→ 응용 암호학

Professional Linux Programming

→ 전문가를 위한 리눅스 프로그래밍

Linux Device Drivers

→ 리눅스 디바이스 드라이버

The Cathedral and the Bazaar

→ 성당과 시장

Computer Architecture: A Quantitative Approach

→ 컴퓨터 구조: 정량적 접근

*부록은 원본을 참고하도록 한다.